

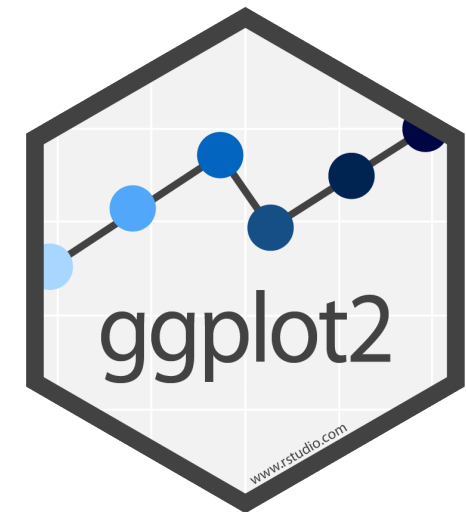
Visualización de datos en R con ggplot2

Fernando Martínez Plumed

ETSINF, UPV, fmartinez@dsic.upv.es

Carlos Monserrat Aranda

ETSINF, UPV, cmonserr@dsic.upv.es



Índice (1/2)

1. Introducción a ggplot2.
2. Construcción de un gráfico por capas.
 - Primeros pasos
 - Los datos
 - Las estéticas
 - GEOMS
 - STATS
3. Detalles de ggplot2.
 - Sobredibujado (overplotting)
 - Dibujado de mapas

Índice (2/2)

4. Posicionamiento

- Faceteado
- Sistemas de coordenadas

5. Escalas

- Introducción
- Escalas de posición
- Escalas de color

6. Puliendo los gráficos

- Los “temas”
- Múltiples gráficos
- Guardar los resultados

Índice

1. Introducción a ggplot2.

Introducción a GGPLOT2.

- ***ggplot2***, es una librería de R basada en la **Gramática para Gráficos** de Leland Wilkinson.
 - L. Wilkinson, The Grammar of Graphics, Ed. Springer, 2nd edition, ISBN: 978-1-4419-2033-1, 2005.
- Implementado por Hadley Wickham.
 - H. Wickham, ggplot2: Elegant Graphics for Data Analysis, Ed. Springer, ISBN: 978-0-387-98140-6, 2011.
- Es un **tercer sistema gráfico dentro de R** (siendo los dos primeros los gráficos de **base** y **lattice**).
- Está disponible desde CRAN mediante la instrucción ***install.packages()***.

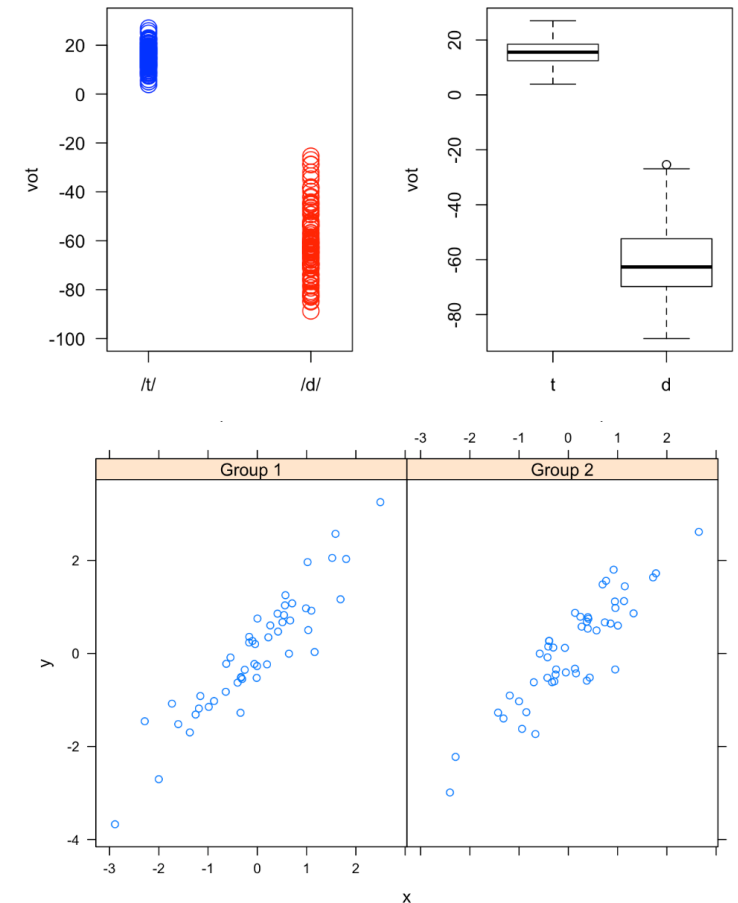
Introducción a GGPLOT2

BASE

- Más común, conceptualmente simple, contruidos paso a paso por medio de funciones.
- **No permiten ir atrás** una vez hemos comenzado el gráfico (necesidad de planificar por adelantado).
- Los gráficos de BASE **son difíciles de “enviar”** a terceros una vez creados (no hay un “lenguaje” gráfico).

LATTICE

- Creado con una sola llamada de función (*xyplot*, *bwplot*, ...). Todos los parámetros se deben especificar a la vez.
- Interfaz basada fórmulas $y \sim x$ (no generaliza correctamente para situaciones complicadas)
- La mayoría son útiles para el análisis de gráficos condicionales:
 - Como la y cambia con la x para diferentes niveles de z .
- La anotación de gráficos no es intuitiva.
- No se puede “añadir” elementos a la gráfica una vez creada.

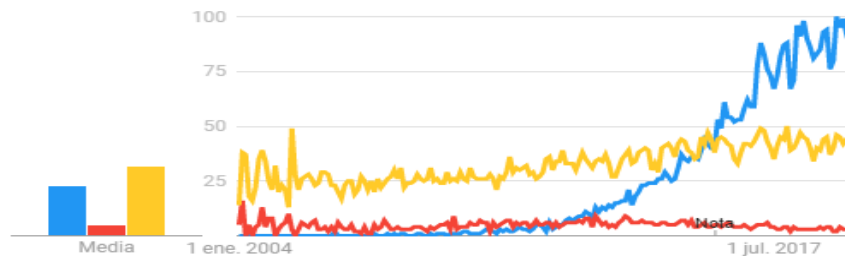


Introducción a GGPLOT2

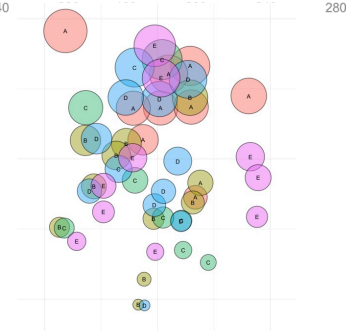
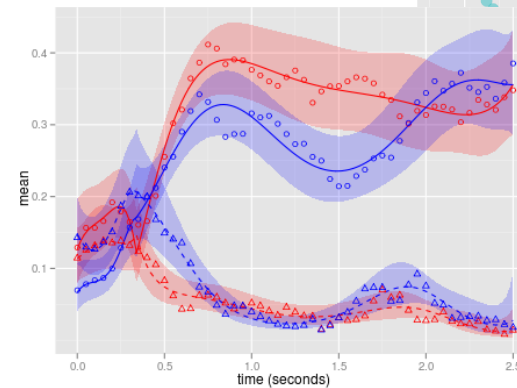
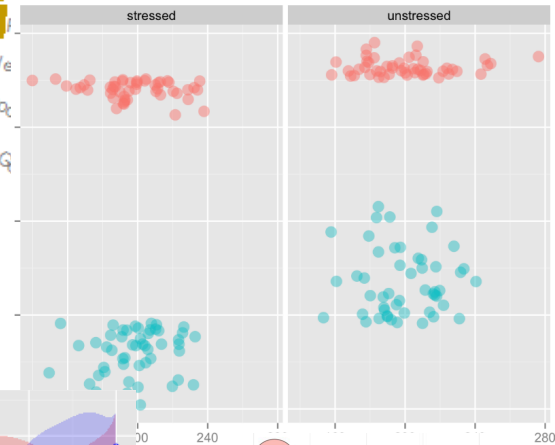
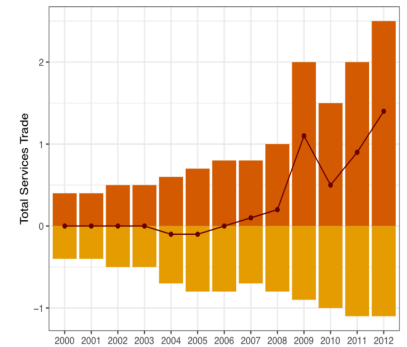
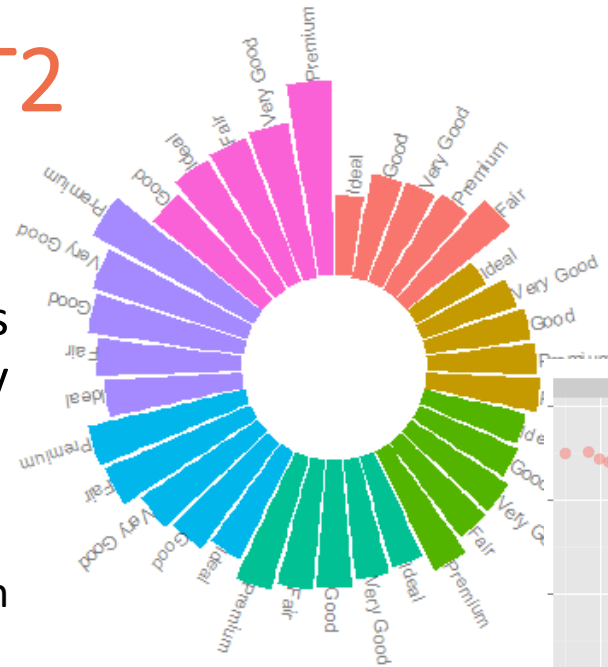
GGPLOT2

- Recoge las ventajas e intenta cubrir las deficiencias de los gráficos BASE y LATTICE.
- Basado en **capas**:
 - Representar los datos, superponer un resumen, añadir metadatos y anotación.

● R ggplot ● R lattice ● R Base



Todo el mundo. 1/1/04 - 25/6/19. Búsqueda web.

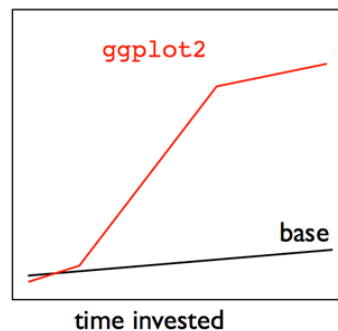


Introducción a GGPLOT2.

- Objetivos principales de los gráficos estadísticos:
 - Facilitar comparaciones
 - Identificar tendencias
- GGPLOT2 consigue esto con mucha facilidad:
 - *“The emphasis in ggplot2 is reducing the amount of thinking time by making it easier to go from the plot in your brain to the plot on the page.”* [Wickham, 2012]
 - *“Base graphics are good for drawing pictures; ggplot2 graphics are good for understanding the data.”* [Wickham, 2012]

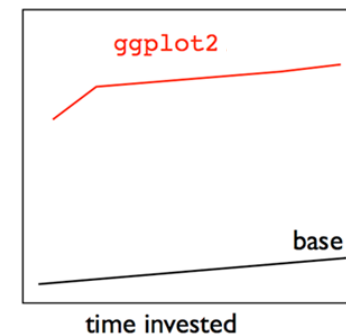
Week one...

quality of
output



After you've climbed the
steepest part of the learning
curve...

quality of
output



Introducción a GGPLOT2

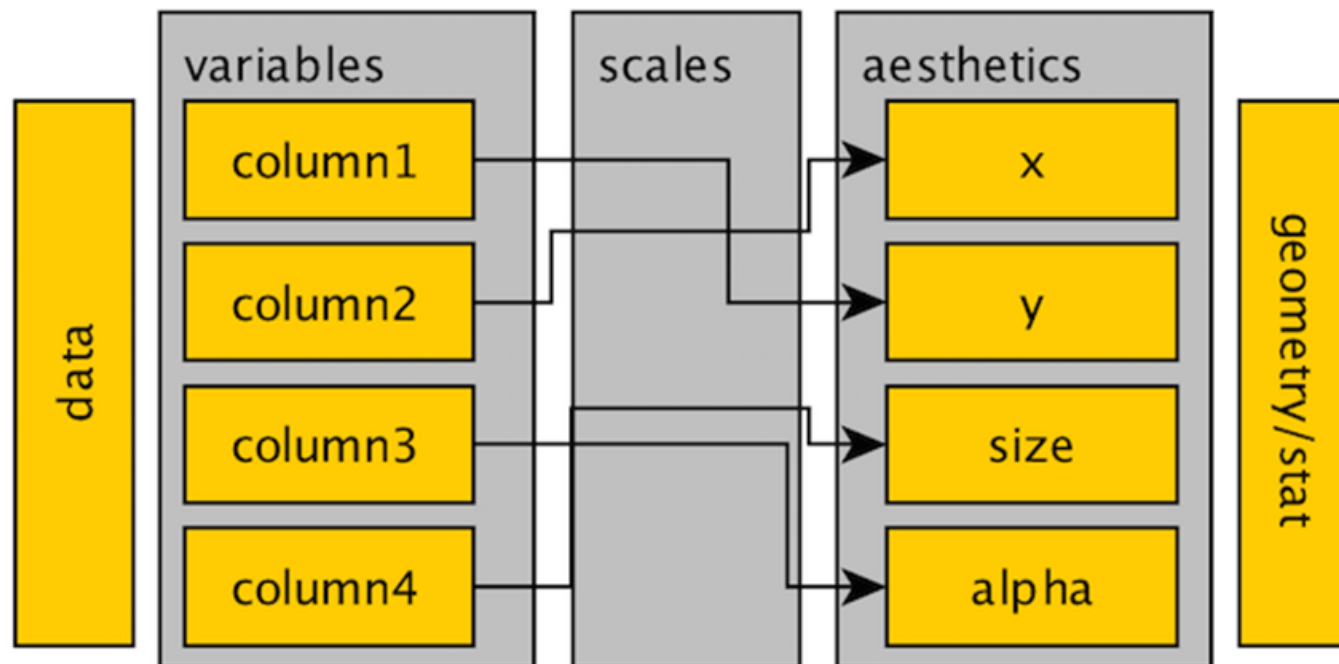
- La forma de construir un gráfico es como construir una frase: **sujeto**, **verbo**, **adjetivos**, ...
 - *The quick brown fox jumps over the lazy dog.*
 - *A rabid red fox bite the friendly dog.*



- . **data**: What you want to visualize, including variables (columns) to be mapped to aesthetic attributes.
- . **aes**: Aesthetics of the geometric and statistical objects, such as color, size, shape and position.
- . **geom**: Geometric objects that are drawn to represent the data: bars, lines, points, etc.
- . **stats**: Statistical transformations of the data, such as binning or averaging.
- . **scales**: Map values in the data space to values in an aesthetic space (color, shape, size...)
- . **coord**: Coordinate system; provides axes and gridlines to make it possible to read the graph.
- . **facets**: Breaking up the data into subsets, to be displayed independently on a grid.

Introducción a GGPLOT2

- “En resumen, la gramática nos dice que un gráfico estadístico es un mapeo de **información** sobre una **geometría** (puntos, líneas, barras, ...) con unos determinados **atributos** (color, forma, tamaño, ...). El gráfico también puede contener **transformaciones** estadísticas de la información y se dibuja en un sistema de **coordenadas** específico”



ÍNDICE

2. Construcción de un gráfico por capas.

- Primeros pasos
- Los datos
- Las estéticas
- GEOMS
- STATS

Construcción por capas: Primeros Pasos

- **mpg** data frame:

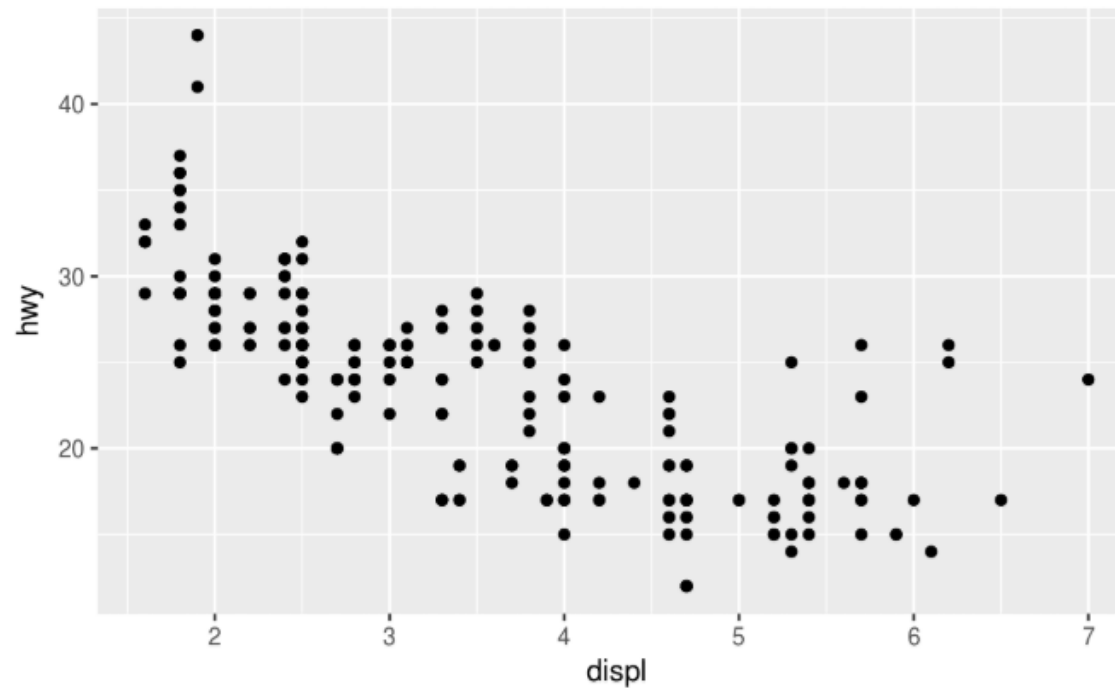
¿Los automóviles con motores de mayor cilindrada consumen más combustible que los automóviles con motores de menor cilindrada?

```
> mpg
# A tibble: 234 x 11
  manufacturer model      displ  year   cyl trans      drv    cty   hwy fl      class
  <chr>         <chr>    <dbl> <int> <int> <chr>    <chr> <int> <int> <chr> <chr>
1 audi         a4          1.8  1999     4 auto(l5)  f       18    29 p       compact
2 audi         a4          1.8  1999     4 manual(m5) f       21    29 p       compact
3 audi         a4          2    2008     4 manual(m6) f       20    31 p       compact
4 audi         a4          2    2008     4 auto(av)   f       21    30 p       compact
5 audi         a4          2.8  1999     6 auto(l5)  f       16    26 p       compact
6 audi         a4          2.8  1999     6 manual(m5) f       18    26 p       compact
7 audi         a4          3.1  2008     6 auto(av)   f       18    27 p       compact
8 audi         a4 quattro  1.8  1999     4 manual(m5) 4       18    26 p       compact
9 audi         a4 quattro  1.8  1999     4 auto(l5)   4       16    25 p       compact
10 audi        a4 quattro  2    2008     4 manual(m6) 4       20    28 p       compact
# ... with 224 more rows
```

¿Cómo es la relación entre el tamaño del motor y la eficiencia del combustible?
¿positiva? ¿negativa?
¿lineal? ¿no lineal?

Construcción por capas: Primeros Pasos

```
ggplot(data = mpg) +  
  geom_point(mapping = aes(x = displ, y = hwy))
```



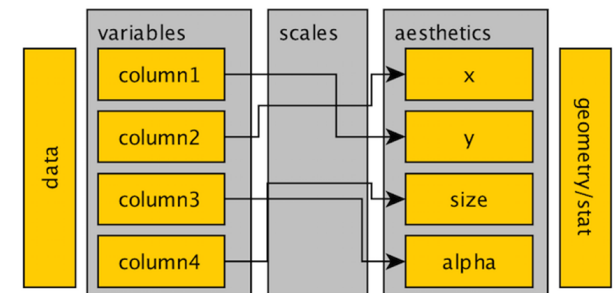
La gráfica muestra una relación negativa entre el tamaño del motor (*displ*) y la eficiencia del combustible (*hwy*).

Los coches con motores grandes usan más combustible. ¿Confirma o refuta esto la hipótesis sobre la eficiencia del combustible y el tamaño del motor?

Construcción por capas: Primeros Pasos

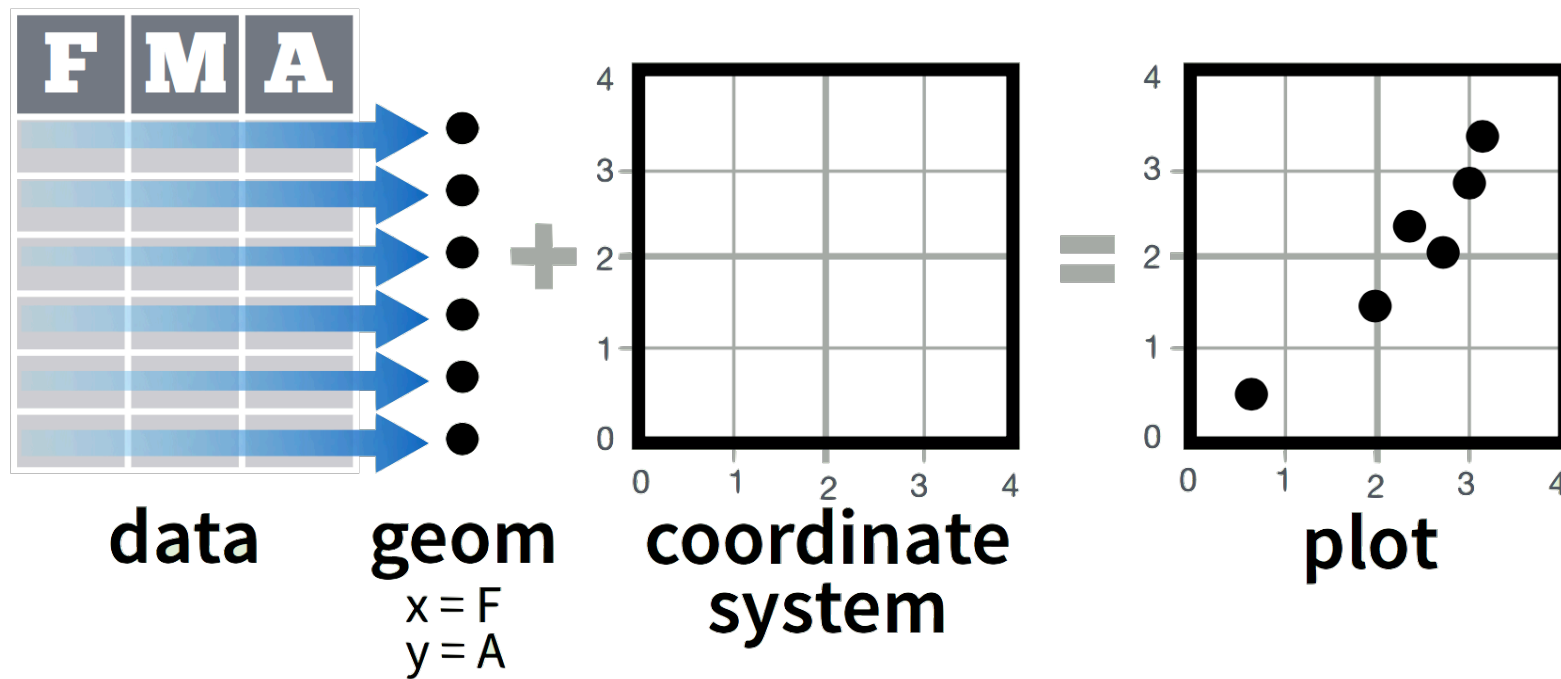
```
ggplot(data = mpg) +  
  geom_point(mapping = aes(x = displ, y = hwy))
```

- Con `ggplot2`, comienzas un gráfico con la función ***ggplot()***, la cual crea un sistema de coordenadas al que puedes agregar capas.
- Para completar el gráfico, agregamos una o más capas a ***ggplot()***. La función ***geom_point()*** agrega una capa de tipo “gráfico de dispersión”
- `ggplot` tiene dos elementos:
 - **datos**: el correspondiente “dataframe”.
 - **mapeado estético**: Cada función ***geom_X()*** en `ggplot2` toma un argumento de “mapping”. Esto define cómo se asignan las variables del dataframe a las propiedades visuales (x, y, color, shape, ...). El argumento de *mapping* siempre está emparejado con ***aes()***, donde los argumentos **x** e **y** especifican qué variables asignar a los ejes.



Construcción por capas: Primeros Pasos

```
ggplot(data = mpg) +  
  geom_point(mapping = aes(x = displ, y = hwy))
```



Construcción por capas: Primeros Pasos

- Plantilla ggplot:

```
ggplot(data = <DATA>) +  
  <GEOM_FUNCTION>(mapping = aes(<MAPPINGS>))
```

- Para hacer un gráfico, solo hay que reemplazar las secciones entre corchetes por un conjunto de ***datos***, una función ***geom*** y una colección de ***mappings***.

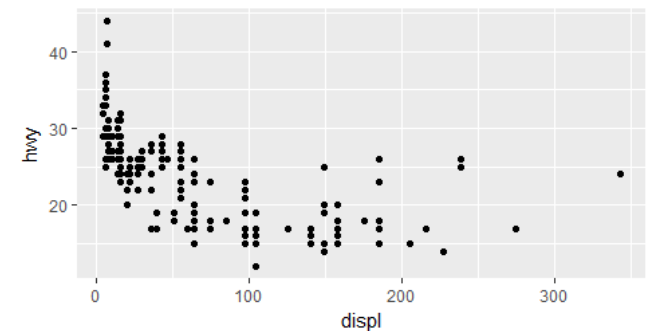
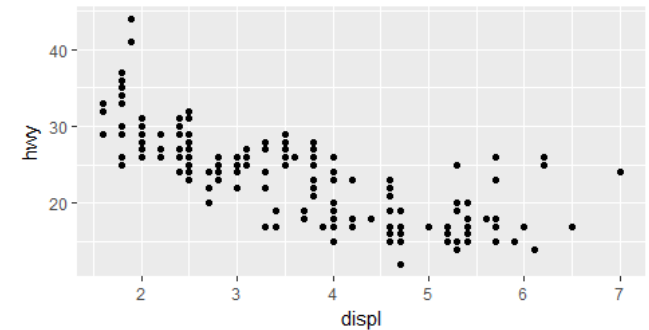
Construcción por capas: Primeros Pasos

- La mínima capa es simplemente una indicación de qué geometría se va a utilizar para representar la información.
- Muchas transformaciones estadísticas (**stats**) están asociadas a geometrías (**geoms**) y viceversa.
 - `<geom_FUNCTION>(mapping, data, stat, ...)`
- Donde:
 - **mapping** es el mapeado estético (con `aes()`).
 - **data** es el “dataset” (sobrescribe el original).
 - **stat** sobrescribe las transformaciones estadísticas por defecto.
 - ... parámetros de geom o de stat.
- Las capas son objetos regulares de R y, por tanto, pueden ser almacenados como variables.

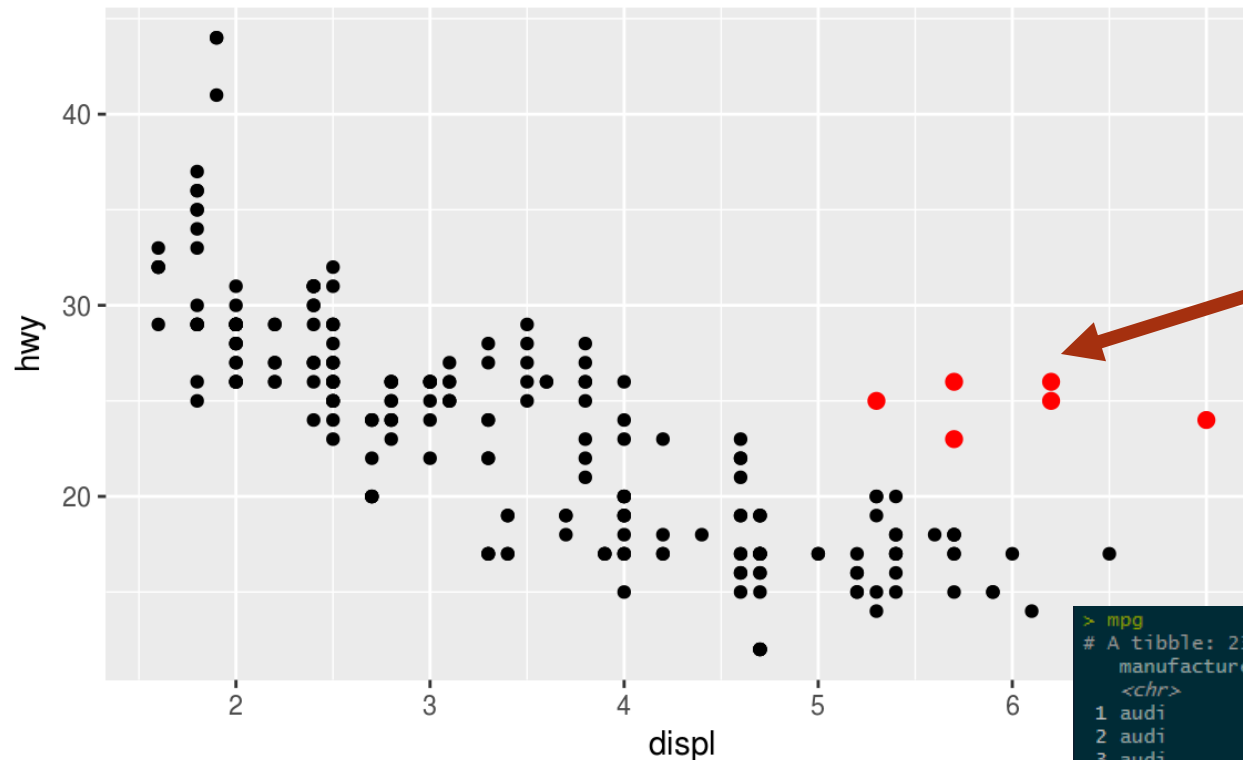
Construcción por capas: Los datos.

- Los datos a visualizar usando funciones de ggplot2 deben estar en estructura de ***data.frame***
- Se puede producir el mismo gráfico pero con diferentes datos: sustituye el ***data.frame***
- Ejemplo:

```
P <- ggplot(mpg) + geom_point(aes(displ, hwy))  
p  
mpg_2 <- transform(mpg, displ= displ^3)  
p %+% mpg_2
```



Construcción por capas: Las estéticas.



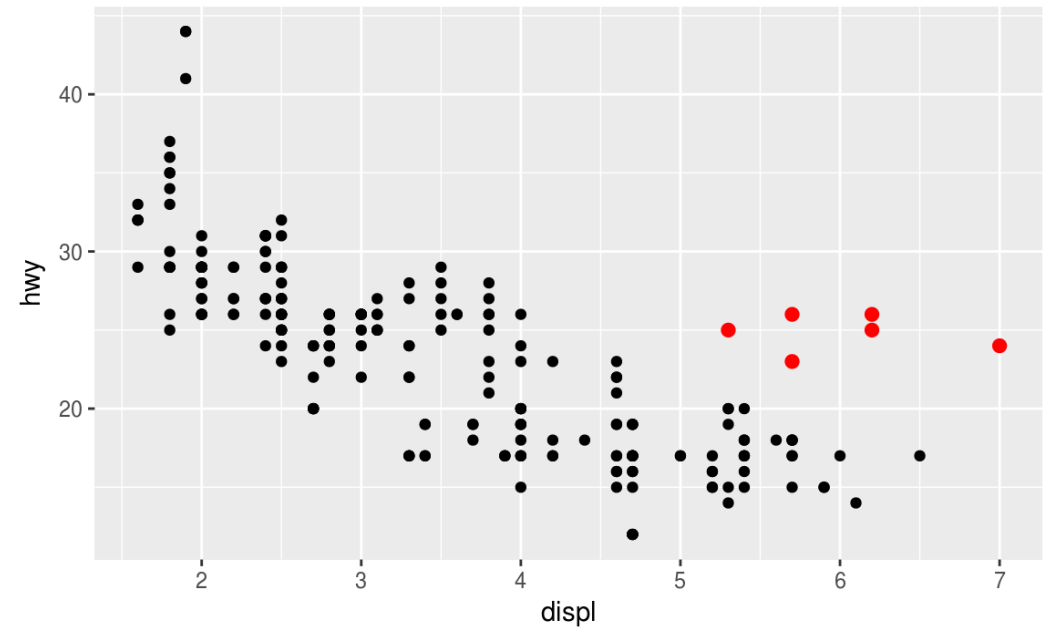
Este grupo parece quedar fuera de la tendencia general
¿Cómo podriamos “explicar” a estos coches?

¿Serán coches híbridos? Una forma de probar esta hipótesis es observar el valor de **clase** de cada automóvil.

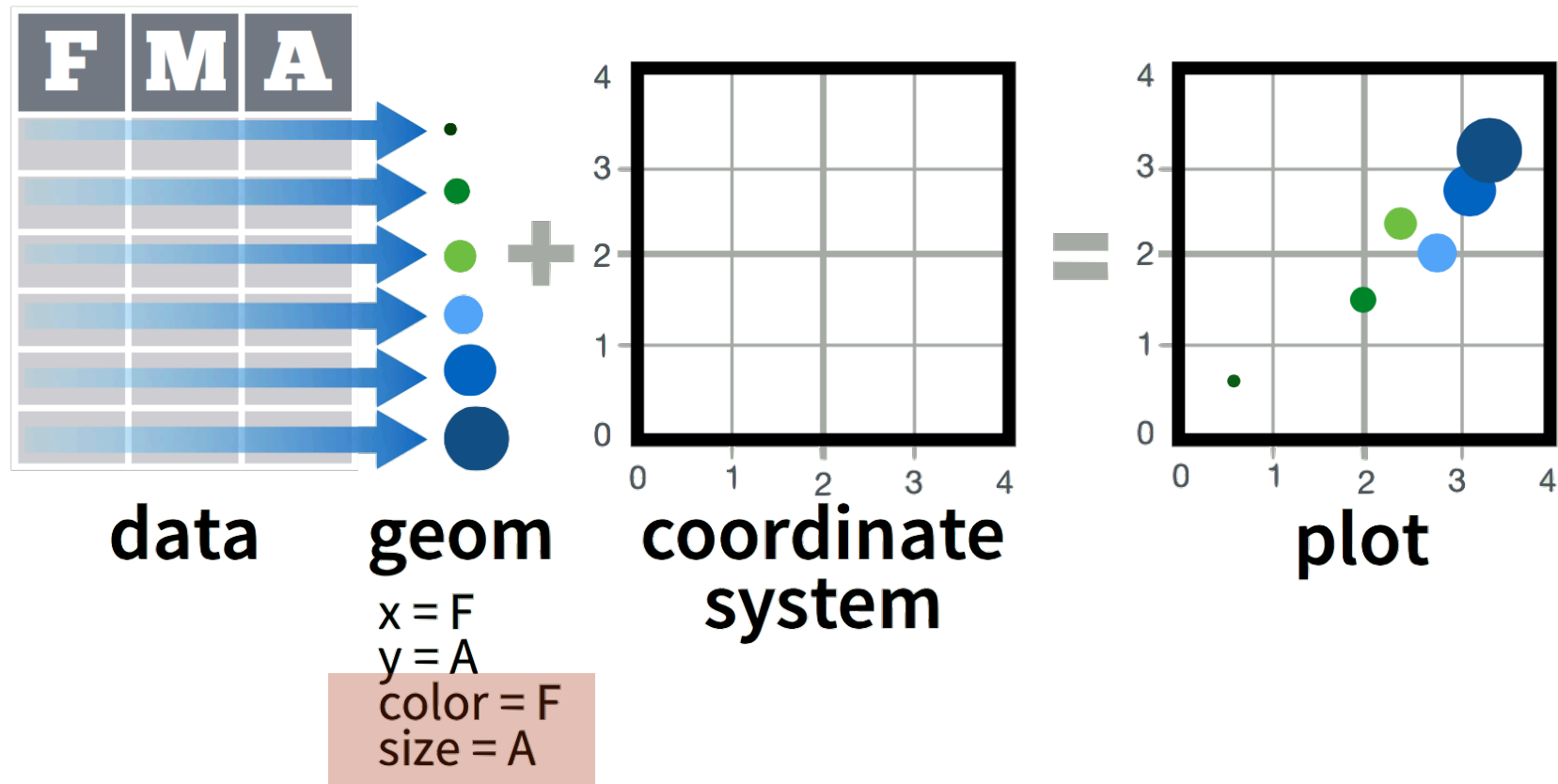
```
> mpg
# A tibble: 234 x 11
  manufacturer model      displ  year   cyl trans      drv      cty   hwy fl      class
  <chr>         <chr>    <dbl> <int> <int> <chr>    <chr> <int> <int> <chr>  <chr>
1 audi         a4          1.8  1999     4 auto(l5) f        18    29 p    compact
2 audi         a4          1.8  1999     4 manual(m5) f        21    29 p    compact
3 audi         a4          2    2008     4 manual(m6) f        20    31 p    compact
4 audi         a4          2    2008     4 auto(av) f        21    30 p    compact
5 audi         a4          2.8  1999     6 auto(l5) f        16    26 p    compact
6 audi         a4          2.8  1999     6 manual(m5) f        18    26 p    compact
7 audi         a4          3.1  2008     6 auto(av) f        18    27 p    compact
8 audi         a4 quattro  1.8  1999     4 manual(m5) 4        18    26 p    compact
9 audi         a4 quattro  1.8  1999     4 auto(l5) 4        16    25 p    compact
10 audi        a4 quattro  2    2008     4 manual(m6) 4        20    28 p    compact
# ... with 224 more rows
```

Construcción por capas: Las estéticas.

- La variable *class* del conjunto de datos de *mpg* clasifica los autos en grupos como *compacto*, *mediano* y *SUV*.
- En el ejemplo podemos agregar una tercera variable asignándolo a una **estética**.
 - Propiedad visual de los objetos en un gráfico (tamaño, forma, color,...).

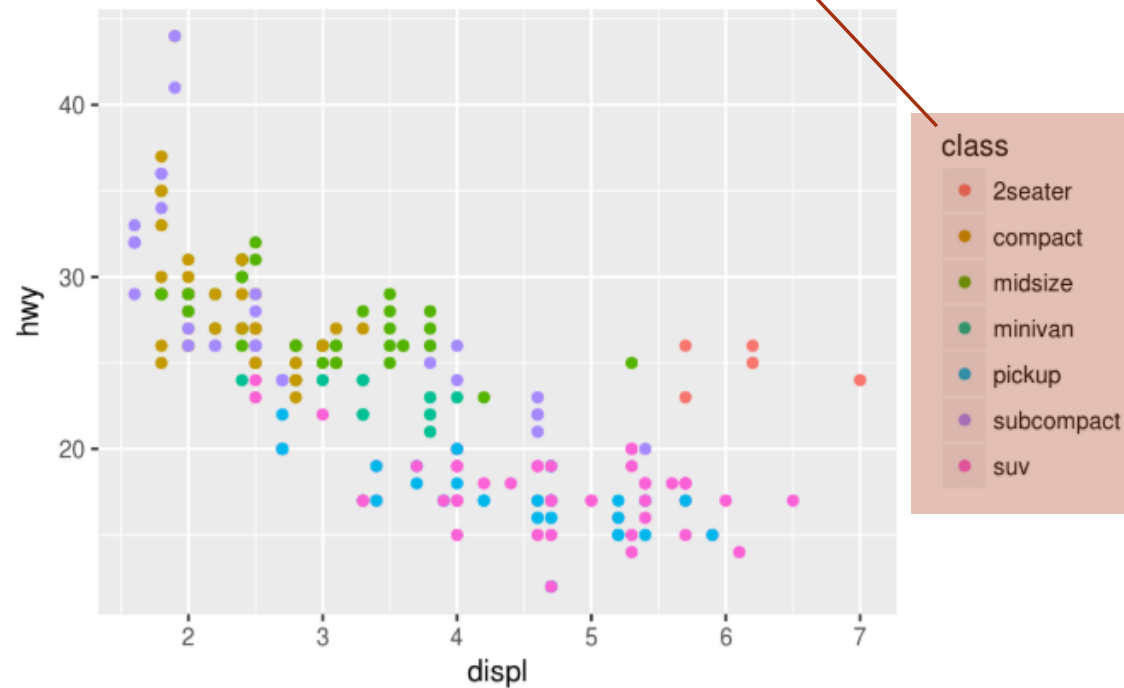


Construcción por capas: Las estéticas.



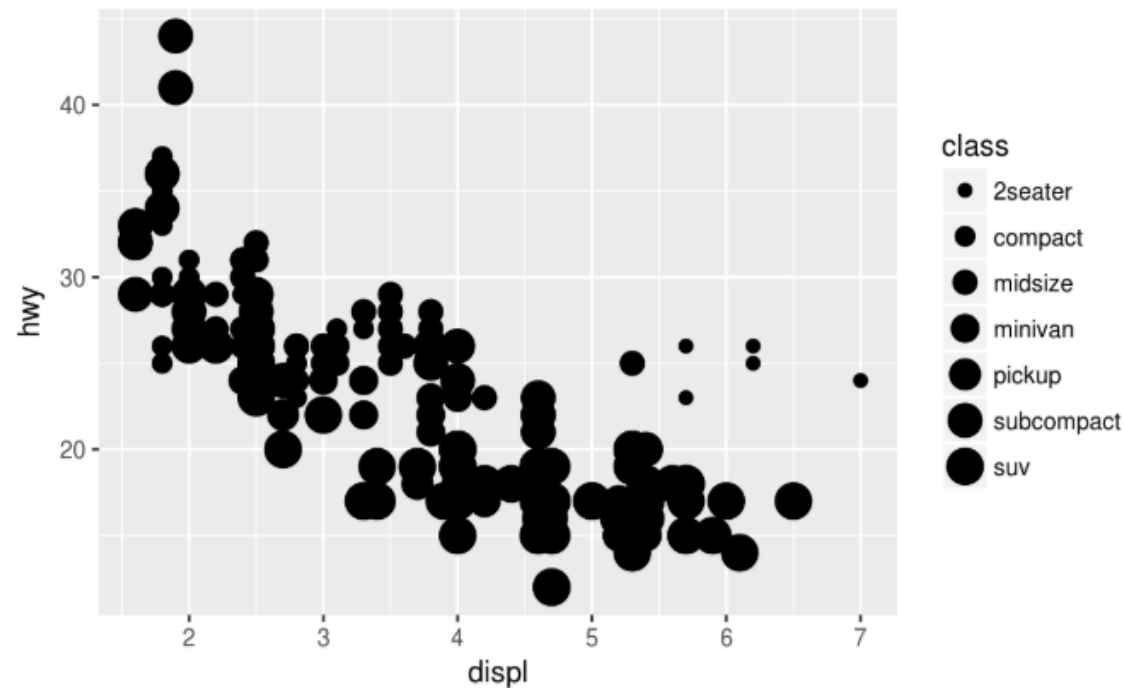
Construcción por capas: Las estéticas.

```
ggplot(data = mpg) +  
  geom_point(mapping = aes(x = displ, y = hwy, color = class))
```



Construcción por capas: Las estéticas.

```
ggplot(data = mpg) +  
  geom_point(mapping = aes(x = displ, y = hwy, size = class))  
#> Warning: Using size for a discrete variable is not advised.
```



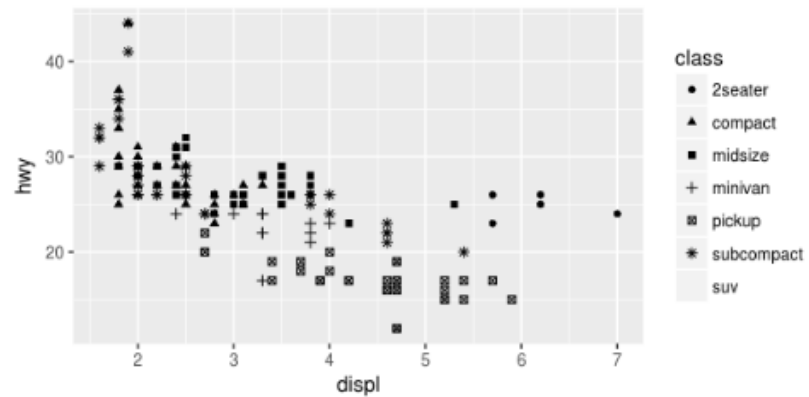
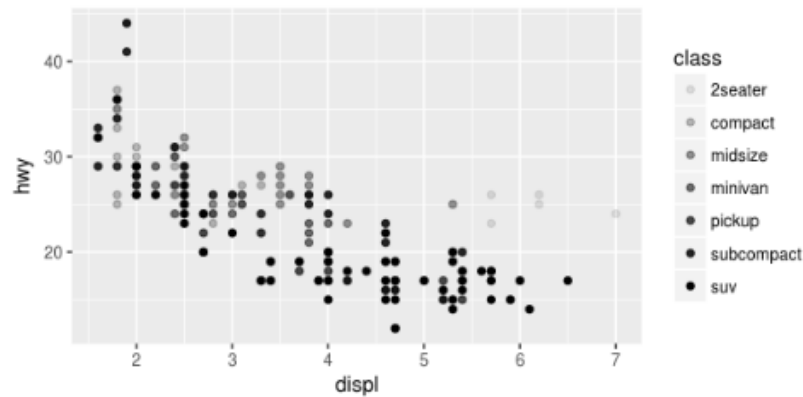
En el ejemplo anterior asignamos **class** a la estética “colour”, pero podríamos haber asignado **class** al tamaño (**size**) de la misma manera.

Construcción por capas: Las estéticas.

```
# Left
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy, alpha = class))

# Right
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy, shape = class))
```

O podríamos haber asignado **class** a la estética de transparencia (**alpha**) o forma de los puntos (**shape**).

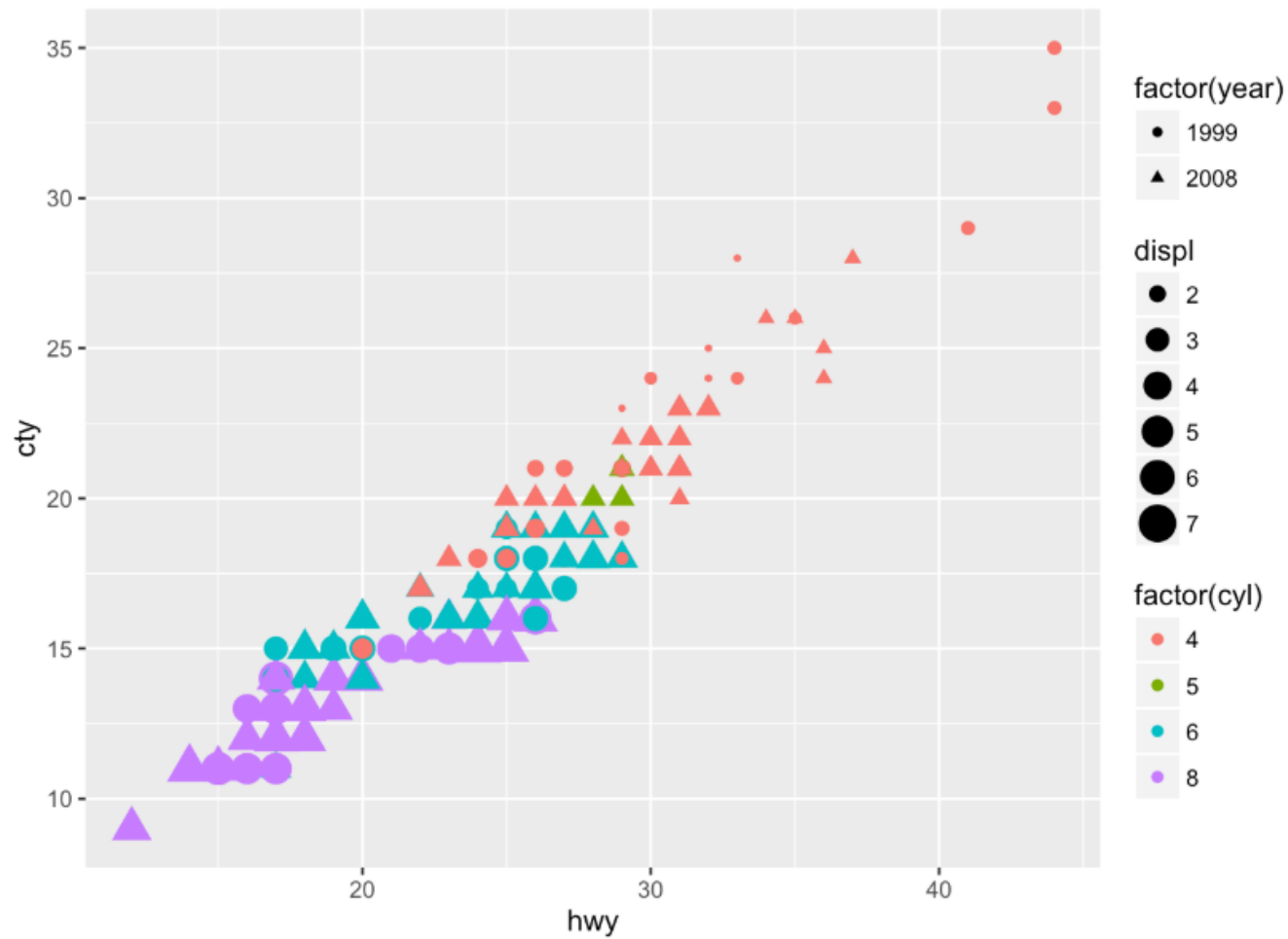


Construcción por capas: Las estéticas.

- Para mapear los datos sobre diferentes estéticas en el gráfico se utiliza la función ***aes()***.
- Una vez que mapea una estética, ***ggplot2*** se ocupa del resto seleccionando una escala razonable para usar con la estética y construye una leyenda que explica el mapeo entre niveles y valores.
- En ***aes()*** no se puede hacer referencia a variables fuera del ***data.frame***.
- Se pueden hacer uso de funciones de variables en la asignación de atributos a estéticas (e.g, ***aes(x = log(displ),...)***)
- Las estéticas más utilizadas son:
 - ***x, y, colour, size, shape, fill*** y ***group***.

Construcción por capas: Las estéticas.

TEST AULA 1



Construcción por capas: Las estéticas.

- Si asignamos una **variable** a una **estética**: la **estética** **variará** en cada **observación**.

```
shape = class))
```

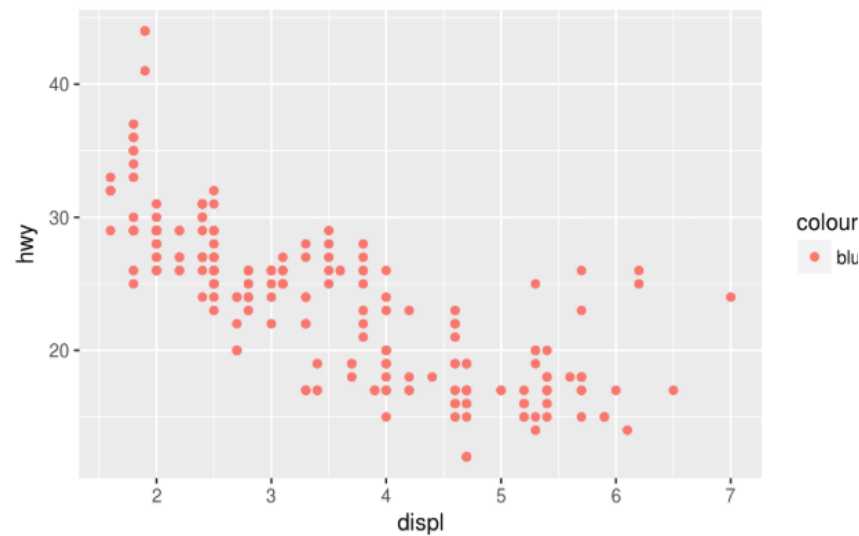
- Si asignamos una **constante** a una **estética**: esta **permanecerá constante** para todas las **observaciones**.

```
color = "blue"))
```

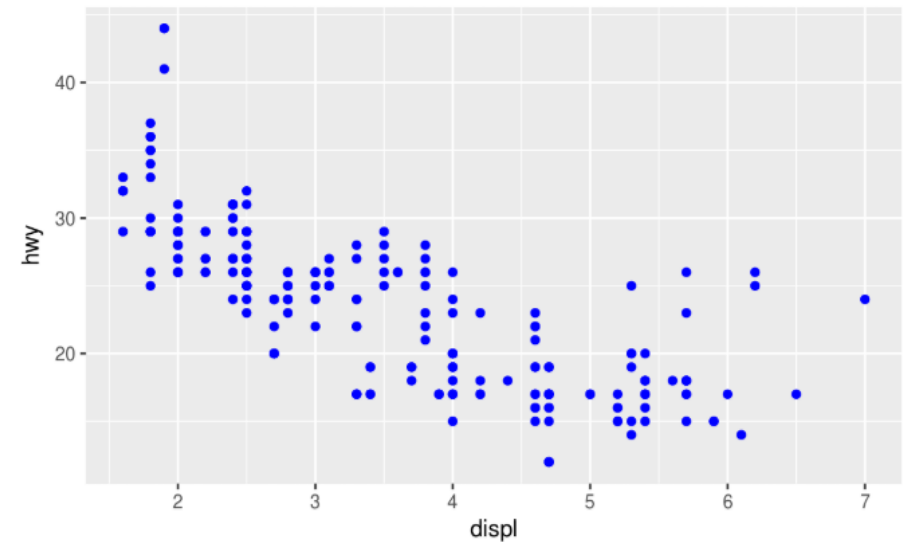
- **OJO**: no es lo mismo variar un atributo de la geometría que su estética.

Construcción por capas: Las estéticas.

```
ggplot(data = mpg) +  
  geom_point(mapping = aes(x = displ, y = hwy, color = "blue"))
```



```
ggplot(data = mpg) +  
  geom_point(mapping = aes(x = displ, y = hwy), color = "blue")
```

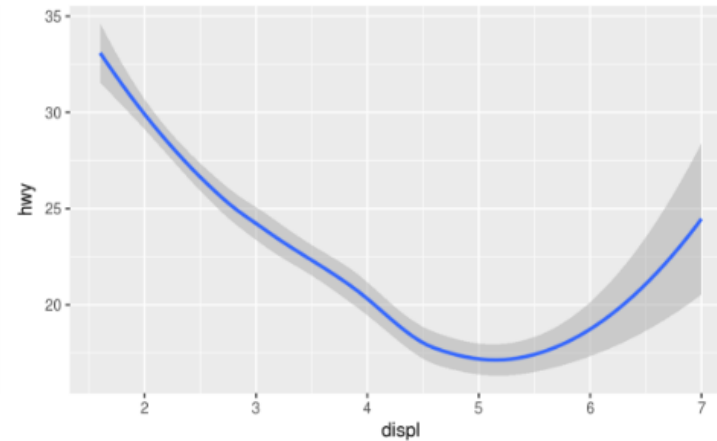
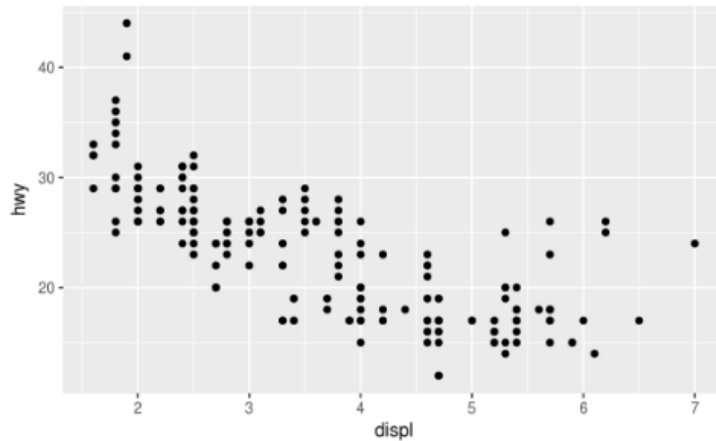


Construcción por capas: Las estéticas.

- El color en el ejemplo anterior no proporciona información sobre una variable, sino que solo cambia la apariencia de la misma.
- Para establecer una estética de forma manual, debemos configurar la estética por su nombre como un argumento fuera de ***aes()***, eligiendo el valor que tenga sentido para esa estética:
 - El nombre de un color como una cadena de caracteres.
 - El tamaño de un punto en mm.
 - La forma de un punto como un número:

 0	 4	 10	 15	 22
 1	 6	 11	 16	 21
 2	 7	 12	 17	 24
 5	 8	 13	 18	 23
 3	 9	 14	 19	 20

Construcción por capas: GEOMS.



- Ambos gráficos contienen la misma variable **x**, la misma variable **y**, y ambos describen los mismos **datos**.
- Sin embargo cada gráfico usa un objeto visual diferente para representar los datos. En la sintaxis **ggplot2**, decimos que usan diferentes **geoms**.
- Un **geom** es el objeto geométrico que un gráfico utiliza para representar datos. Como vemos arriba, es posible usar diferentes **geoms** para trazar los mismos datos.

```
# left
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy))
```

```
# right
ggplot(data = mpg) +
  geom_smooth(mapping = aes(x = displ, y = hwy))
```

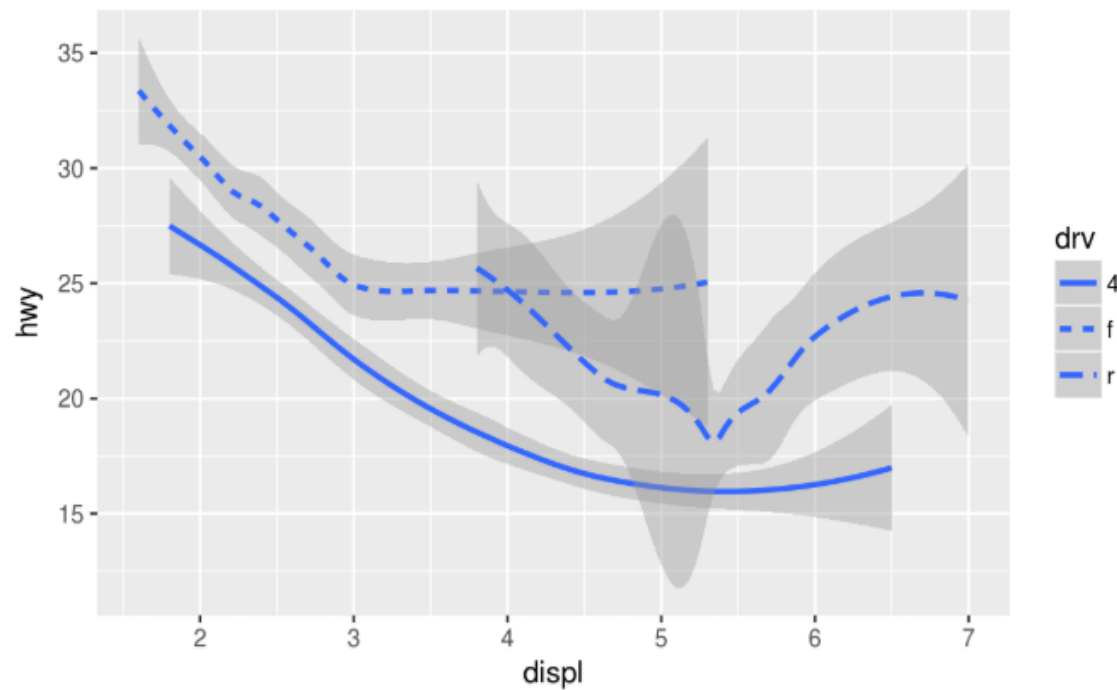
Construcción por capas: GEOMS.

- Los objetos geométricos (GEOMS) son los que realmente **controlan el tipo de gráfico que se va a crear**.
- Cada geometría tiene una **serie de estéticas que comprende**, algunas de las cuales son necesarias para la creación de la gráfica.
 - E.g., es posible establecer la forma (***shape***) de un punto, pero no para un gráfico de tipo línea (aunque sí el tipo de línea, ***linetype***).

Construcción por capas: GEOMS.

```
ggplot(data = mpg) +  
  geom_smooth(mapping = aes(x = displ, y = hwy, linetype = drv))
```

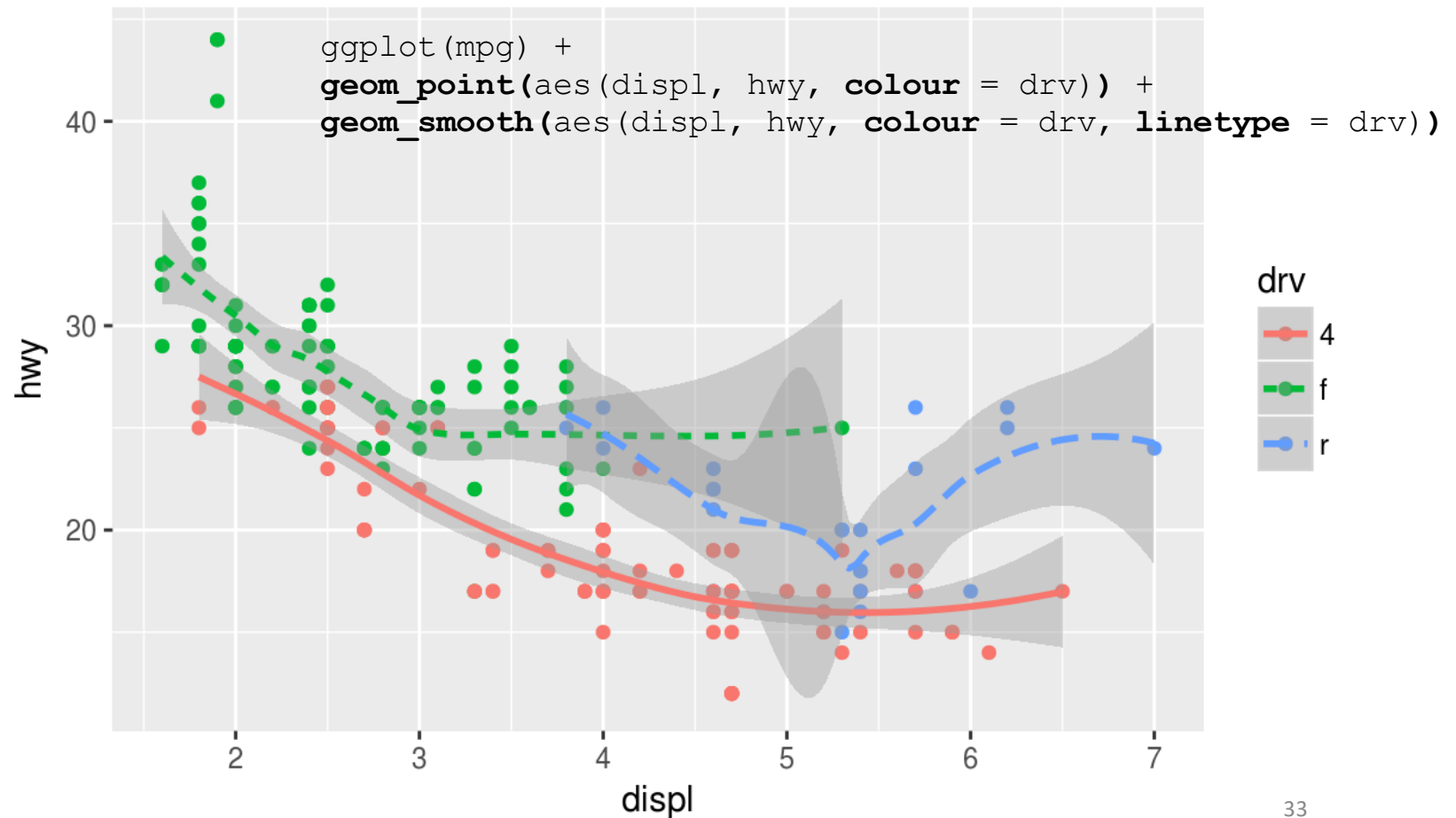
geom_smooth()
separa los coches
en tres líneas en
función de su valor
de ***drv***.



Construcción por capas: GEOMS.

Podríamos superponer las líneas sobre los datos brutos y luego colorear todo según *drv*.

¡Observar que este diagrama contiene dos *geoms* en el mismo gráfico!

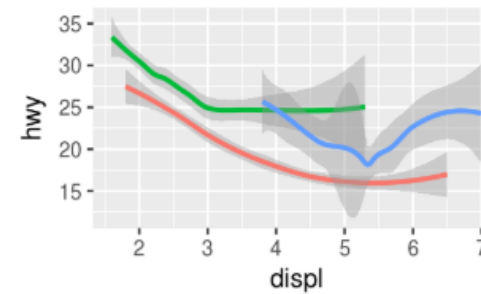
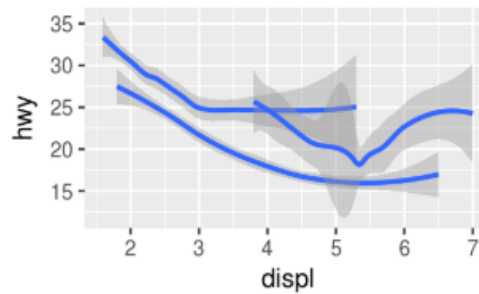
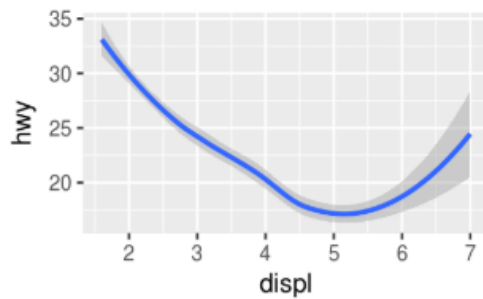


Construcción por capas: GEOMS.

- Es posible establecer la estética ***group*** a una variable categórica para dibujar múltiples objetos.
- ggplot2 dibujará un objeto separado para cada valor único de la variable de agrupamiento.
- En la práctica, ggplot2 agrupará automáticamente los datos para estos ***geoms*** siempre que asigne una estética a una variable discreta (como en el ejemplo del tipo de línea).

Construcción por capas: GEOMS.

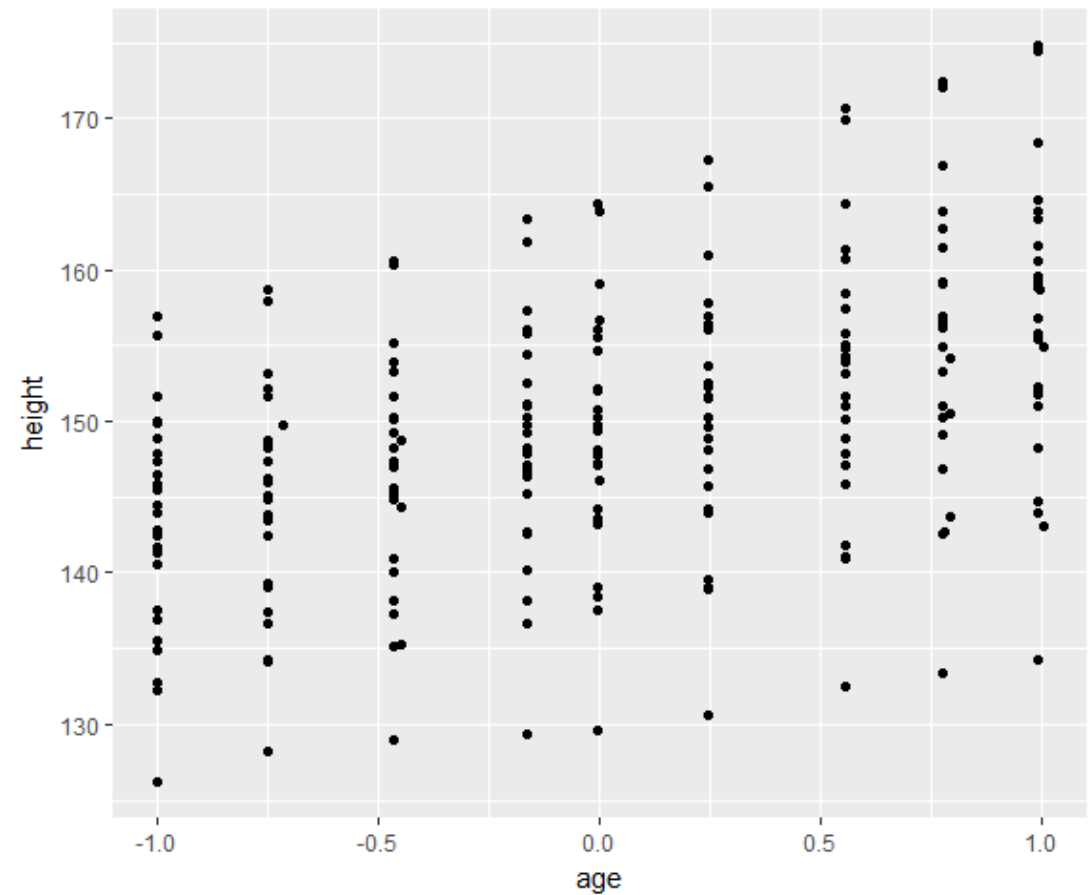
```
ggplot(data = mpg) +  
  geom_smooth(mapping = aes(x = displ, y = hwy))  
  
ggplot(data = mpg) +  
  geom_smooth(mapping = aes(x = displ, y = hwy, group = drv))  
  
ggplot(data = mpg) +  
  geom_smooth(  
    mapping = aes(x = displ, y = hwy, color = drv),  
    show.legend = FALSE  
  )
```



Construcción por capas: Grouping

	Subject	age	height	Occasion
1	1	-1.0000	140.50	1
2	1	-0.7479	143.40	2
3	1	-0.4630	144.80	3
4	1	-0.1643	147.10	4
5	1	-0.0027	147.70	5
6	1	0.2466	150.20	6
7	1	0.5562	151.70	7
8	1	0.7781	153.30	8
9	1	0.9945	155.80	9
10	2	-1.0000	136.90	1
11	2	-0.7479	139.10	2
12	2	-0.4630	140.10	3
13	2	-0.1643	142.60	4
14	2	-0.0027	143.20	5
15	2	0.2466	144.00	6
16	2	0.5562	145.80	7
17	2	0.7781	146.80	8
18	2	0.9945	148.30	9
19	3	-1.0000	150.00	1
20	3	-0.7479	152.10	2

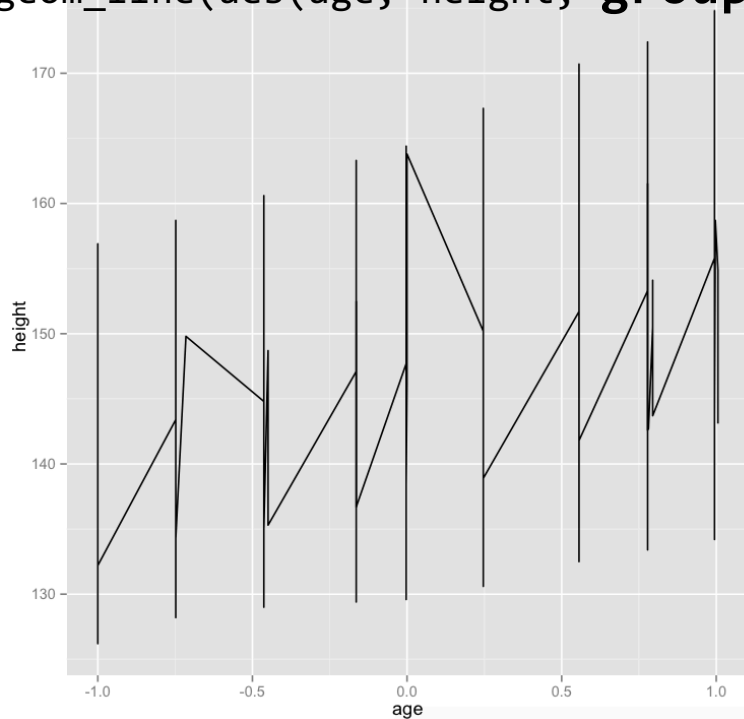
```
library(nlme)
ggplot(Oxboys) +
  geom_point(aes(age, height))
```



Construcción por capas: Grouping

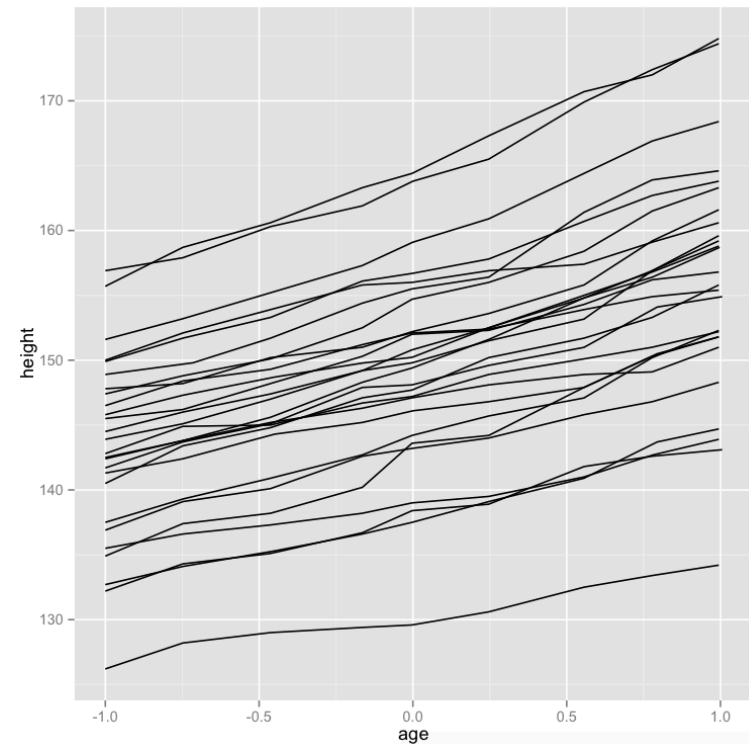
```
ggplot(Oxboys) +  
geom_line(aes(age, height))
```

```
ggplot(Oxboys) +  
geom_line(aes(age, height, group = 1))
```



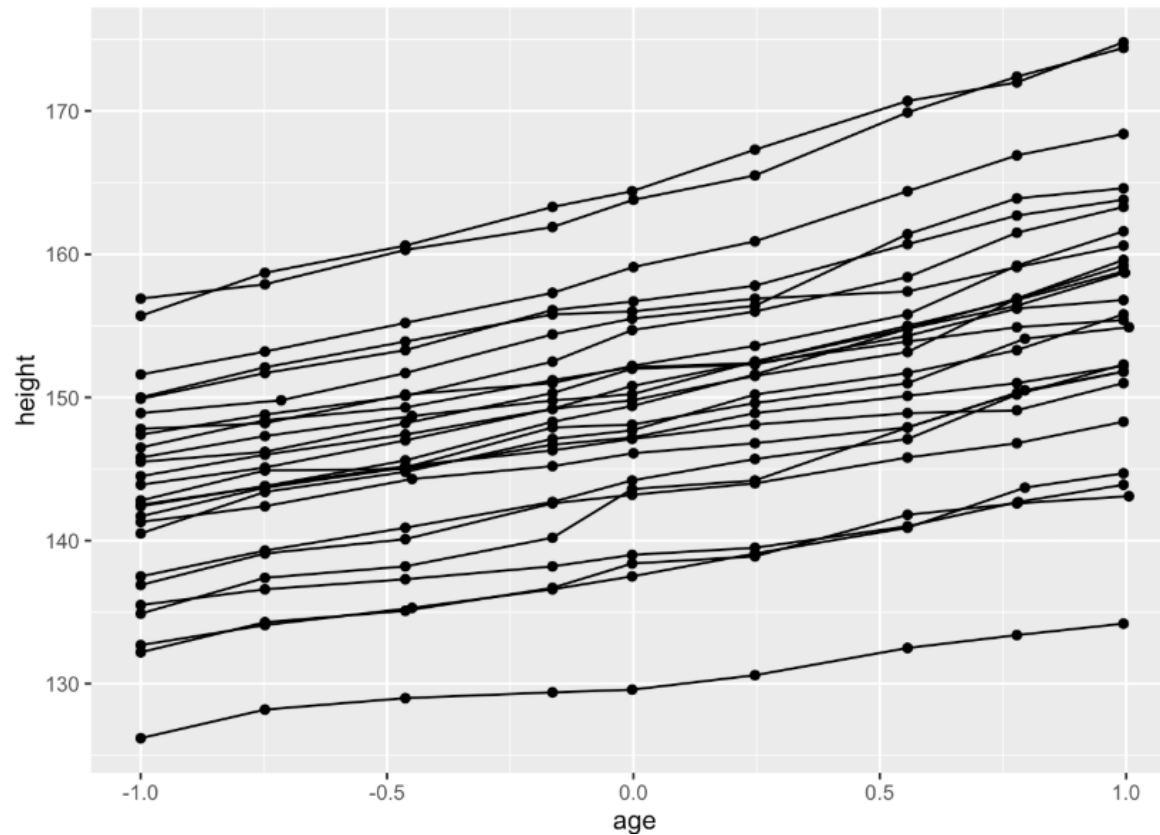
Por defecto!

```
ggplot(Oxboys) +  
geom_line(aes(age, height, group=Subject))
```



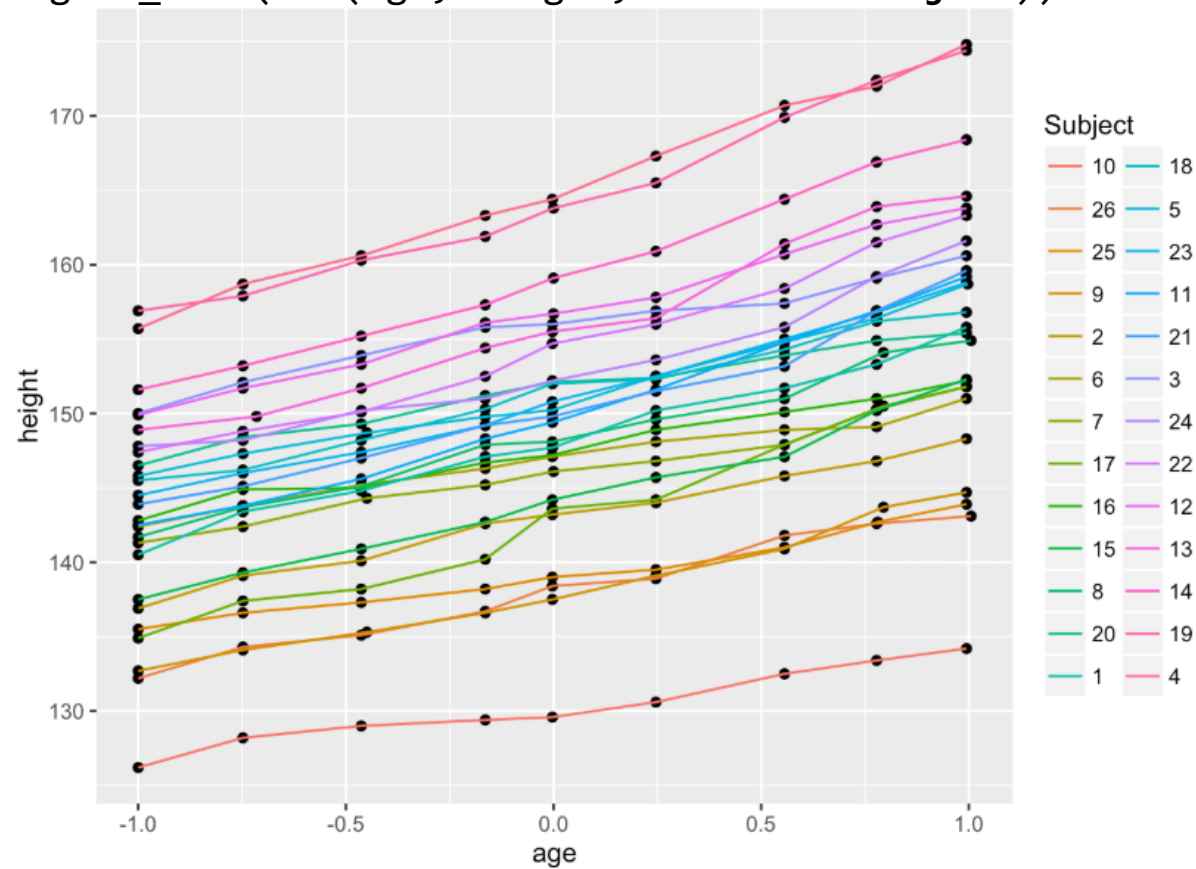
Construcción por capas: Grouping

```
ggplot(Oxboys) + geom_point(aes(age, height)) +  
  geom_line(aes(age, height, group = Subject))
```



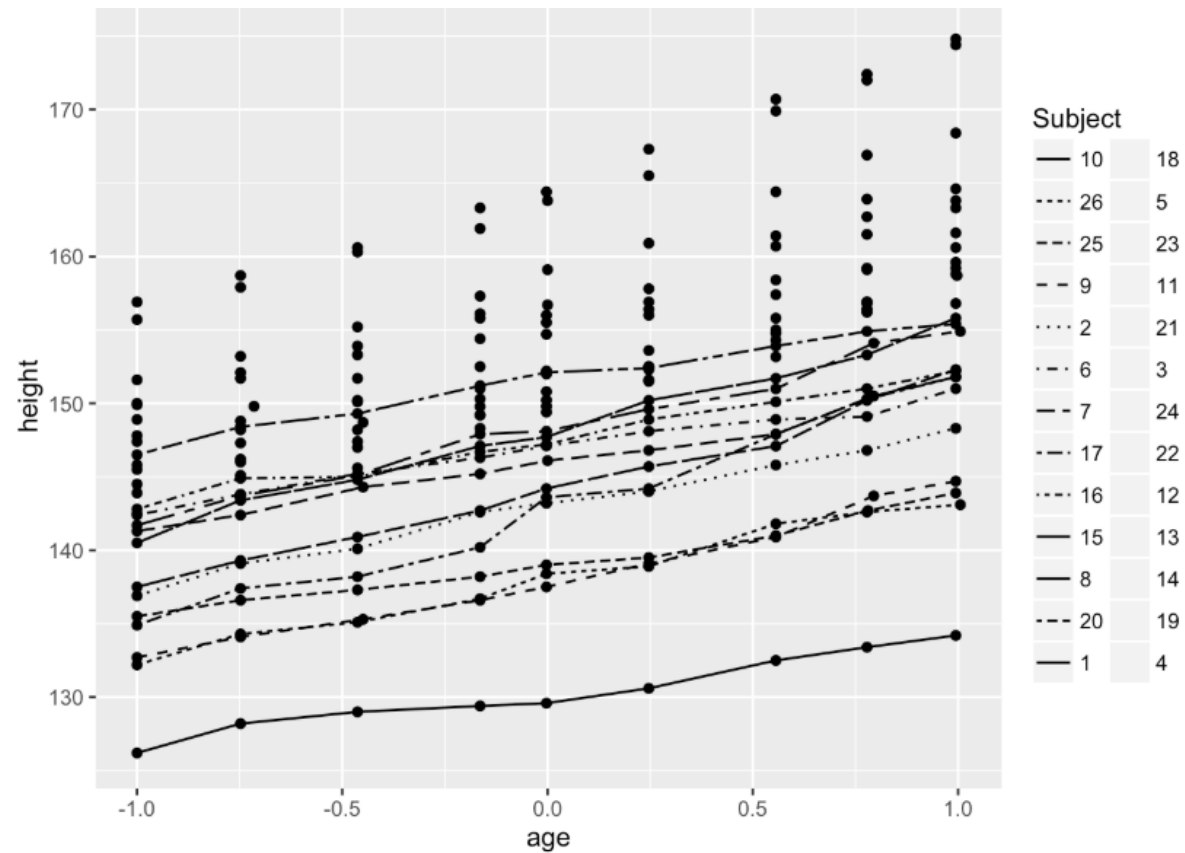
Construcción por capas: Grouping

```
ggplot(Oxboys) + geom_point(aes(age, height)) +  
geom_line(aes(age, height, colour = Subject))
```



Construcción por capas: Grouping

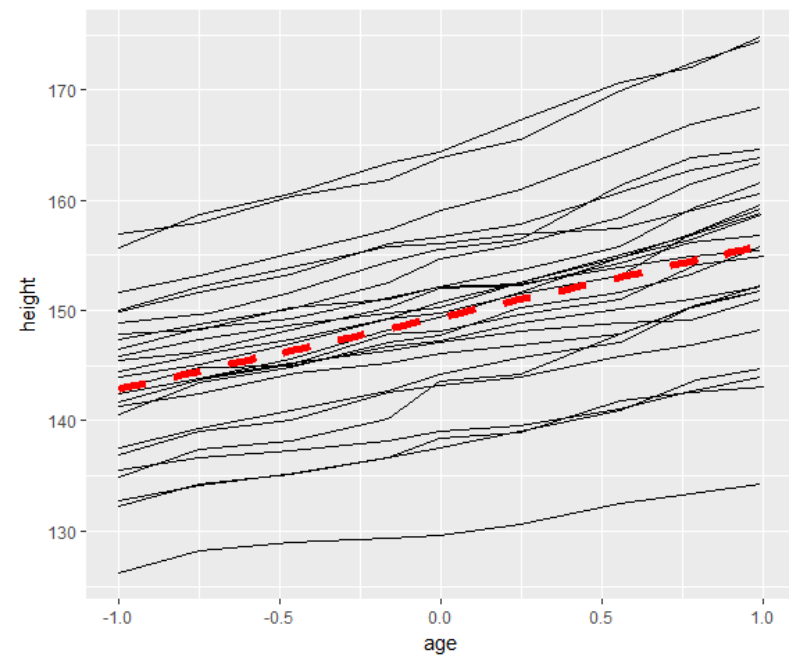
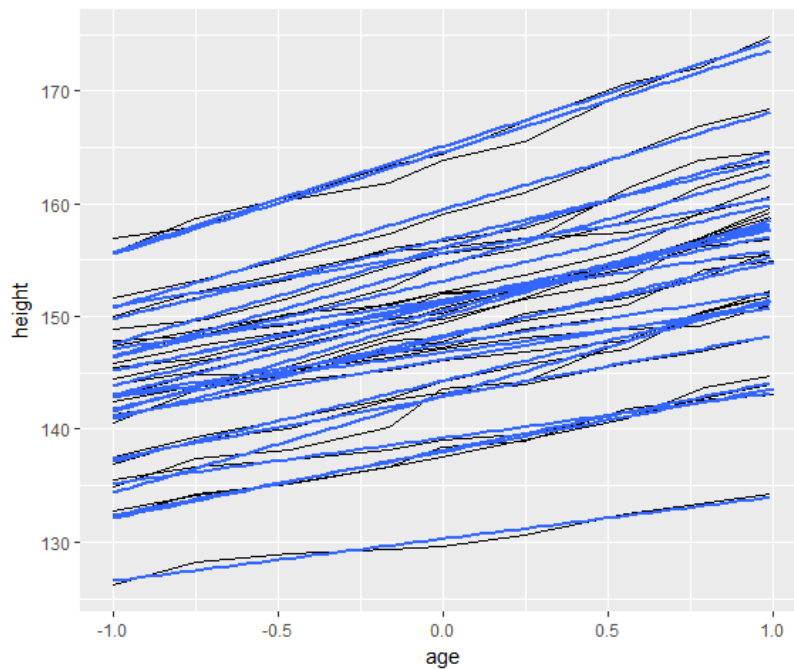
```
ggplot(Oxboys) + geom_point(aes(age, height)) +  
geom_line(aes(age, height, linetype = Subject))
```



Construcción por capas: Las estéticas.

- Algunas veces resulta interesante realizar resúmenes basados en diferentes niveles de agregación.
- ¿Cómo podríamos generar estos gráficos?

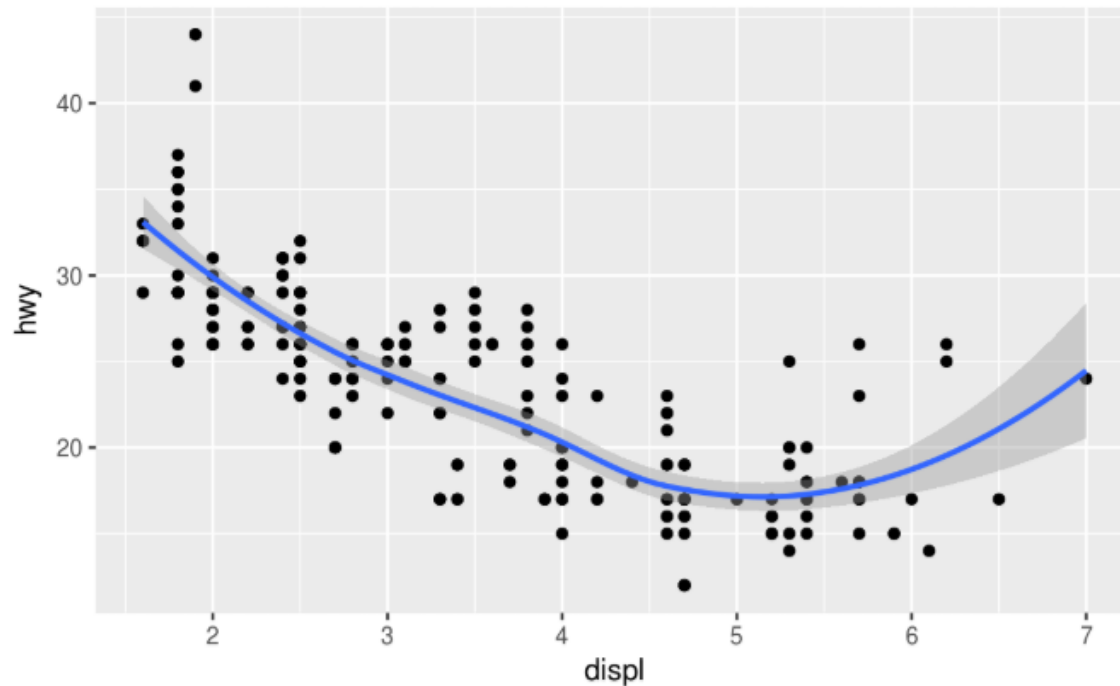
TEST
AULA 2



Construcción por capas: GEOMS.

```
ggplot(data = mpg) +  
  geom_point(mapping = aes(x = displ, y = hwy)) +  
  geom_smooth(mapping = aes(x = displ, y = hwy))
```

¡Observar que este diagrama contiene dos **geoms** en el mismo gráfico!



Construcción por capas: GEOMS.

```
ggplot(data = mpg) +  
  geom_point(mapping = aes(x = displ, y = hwy)) +  
  geom_smooth(mapping = aes(x = displ, y = hwy))
```

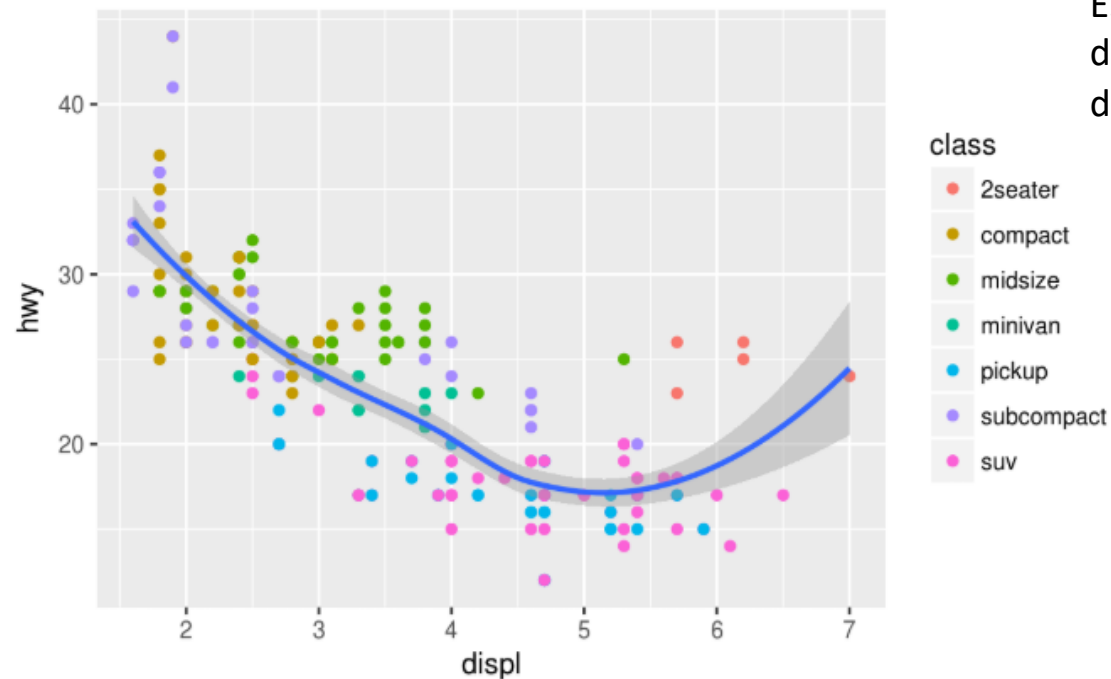
- Esto introduce cierta duplicación en nuestro código.
- Es posible evitar este tipo de repetición pasando el mapping (*aes()*) de forma global en *ggplot()*:

```
ggplot(data = mpg, mapping = aes(x = displ, y = hwy)) +  
  geom_point() +  
  geom_smooth()
```

Construcción por capas: GEOMS.

```
ggplot(data = mpg, mapping = aes(x = displ, y = hwy)) +  
  geom_point(mapping = aes(color = class)) +  
  geom_smooth()
```

Si a su vez incluyes mappings locales en una función **geom**, ggplot2 sobrescribirá las asignaciones globales solo para esa capa.

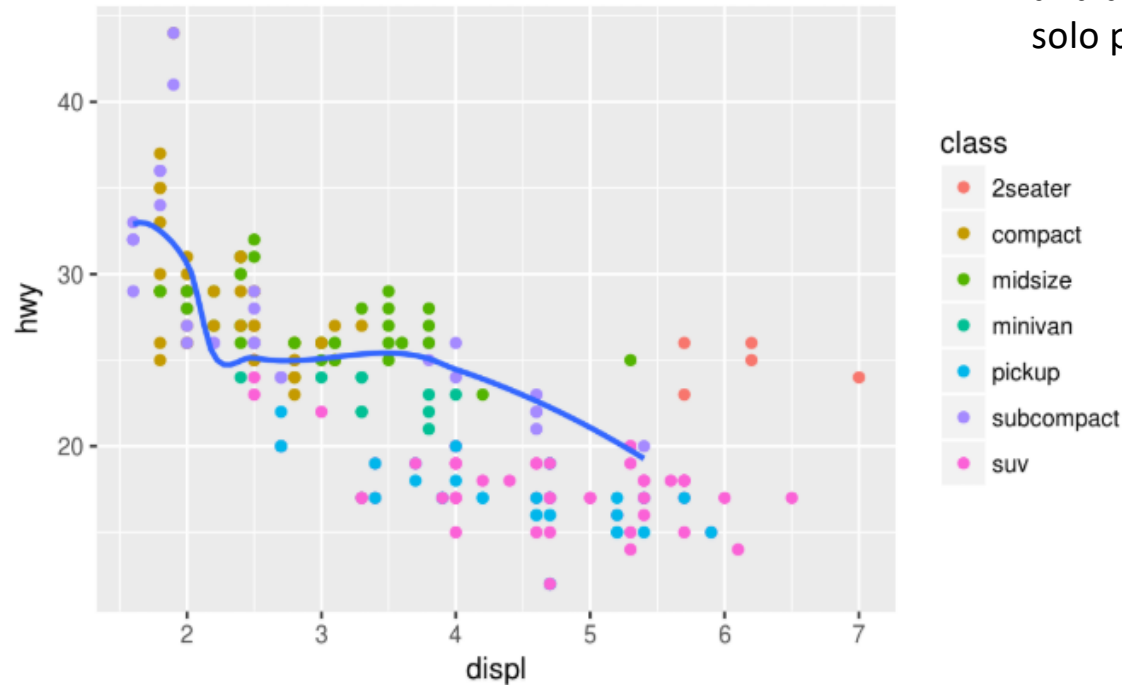


Esto hace posible mostrar diferentes estéticas en diferentes capas.

Construcción por capas: GEOMS.

```
ggplot(data = mpg, mapping = aes(x = displ, y = hwy)) +  
  geom_point(mapping = aes(color = class)) +  
  geom_smooth(data = filter(mpg, class == "subcompact"), se = FALSE)
```

Puedes usar la misma idea
para especificar datos
diferentes para cada capa.

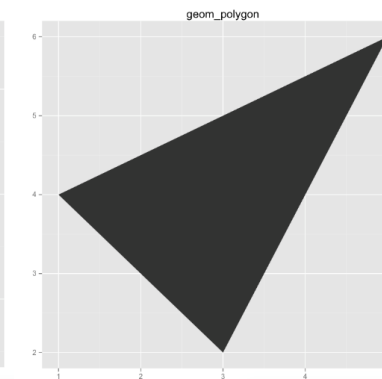
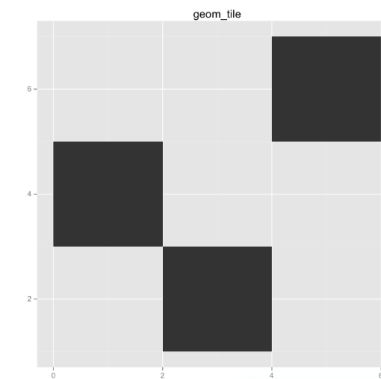
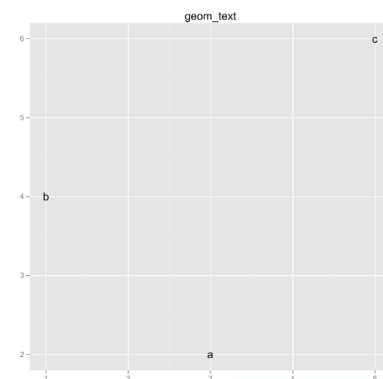
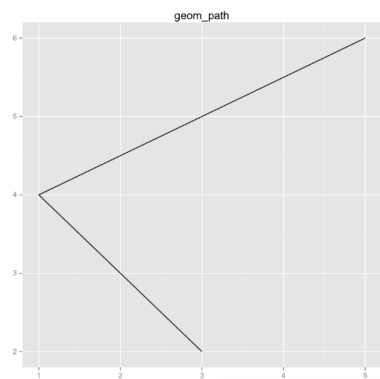
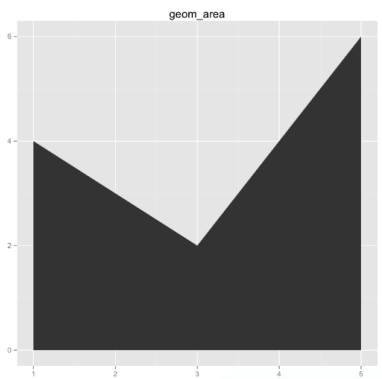
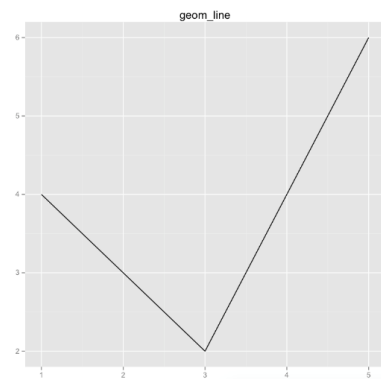
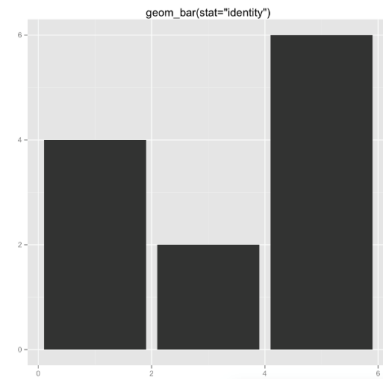
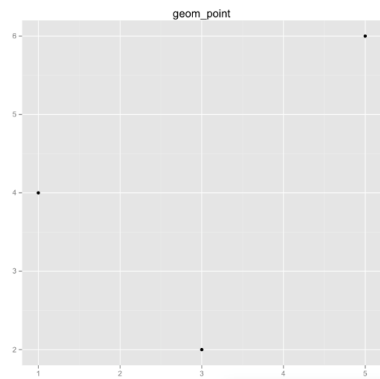


geom_smooth() muestra solo un subconjunto del conjunto de datos de *mpg*, los coches subcompactos. Las estéticas locales en ***geom_smooth()*** anulan las globales en ***ggplot()*** solo para esa capa.

Construcción por capas: GEOMS.

- ggplot2 proporciona más de 30 geoms, y los paquetes de extensión proporcionan aún más (<https://exts.ggplot2.tidyverse.org/>).
- La mejor forma de obtener una descripción completa es <https://rstudio.com/resources/cheatsheets/>, <https://github.com/rstudio/cheatsheets/raw/master/data-visualization-2.1.pdf> .
- Para obtener más información sobre cualquier **geom** individual, usar la ayuda: **?Geom_smooth**

Construcción por capas: GEOMS.



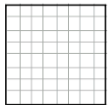
Construcción por capas: GEOMS.

Name	Description
abline	Line, specified by slope and intercept
area	Area plots
bar	Bars, rectangles with bases on y-axis
blank	Blank, draws nothing
boxplot	Box-and-whisker plot
contour	Display contours of a 3d surface in 2d
crossbar	Hollow bar with middle indicated by horizontal line
density	Display a smooth density estimate
density_2d	Contours from a 2d density estimate
errorbar	Error bars
histogram	Histogram
hline	Line, horizontal
interval	Base for all interval (range) geoms
jitter	Points, jittered to reduce overplotting
line	Connect observations, in order of x value
linrange	An interval represented by a vertical line
path	Connect observations, in original order
point	Points, as for a scatterplot
pointrange	An interval represented by a vertical line, with a point in the middle
polygon	Polygon, a filled path
quantile	Add quantile lines from a quantile regression
ribbon	Ribbons, y range with continuous x values
rug	Marginal rug plots
segment	Single line segments
smooth	Add a smoothed condition mean
step	Connect observations by stairs
text	Textual annotations
tile	Tile plot as densely as possible, assuming that every tile is the same size
vline	Line, vertical

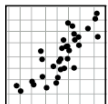
Construcción por capas: GEOMS.

Continuous X, Continuous Y

```
f <- ggplot(mpg, aes(cty, hwy))
```

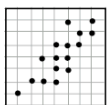


f + geom_blank()



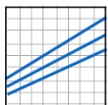
f + geom_jitter()

x, y, alpha, color, fill, shape, size



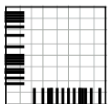
f + geom_point()

x, y, alpha, color, fill, shape, size



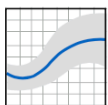
f + geom_quantile()

x, y, alpha, color, linetype, size, weight



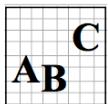
f + geom_rug(sides = "bl")

alpha, color, linetype, size



f + geom_smooth(model = lm)

x, y, alpha, color, fill, linetype, size, weight

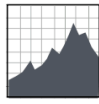


f + geom_text(aes(label = cty))

x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

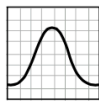
Continuous

```
a <- ggplot(mpg, aes(hwy))
```



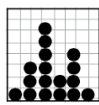
a + geom_area(stat = "bin")

x, y, alpha, color, fill, linetype, size



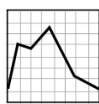
a + geom_density(kernel = "gaussian")

x, y, alpha, color, fill, linetype, size, weight



a + geom_dotplot()

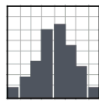
x, y, alpha, color, fill



a + geom_freqpoly()

x, y, alpha, color, linetype, size

b + geom_freqpoly(aes(y = ..density..))



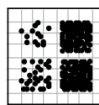
a + geom_histogram(binwidth = 5)

x, y, alpha, color, fill, linetype, size, weight

b + geom_histogram(aes(y = ..density..))

Discrete X, Discrete Y

```
h <- ggplot(diamonds, aes(cut, color))
```

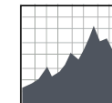


h + geom_jitter()

x, y, alpha, color, fill, shape, size

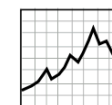
Continuous Function

```
j <- ggplot(economics, aes(date, unemploy))
```



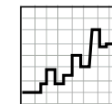
j + geom_area()

x, y, alpha, color, fill, linetype, size



j + geom_line()

x, y, alpha, color, linetype, size

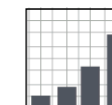


j + geom_step(direction = "hv")

x, y, alpha, color, linetype, size

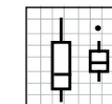
Discrete X, Continuous Y

```
g <- ggplot(mpg, aes(class, hwy))
```



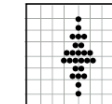
g + geom_bar(stat = "identity")

x, y, alpha, color, fill, linetype, size, weight



g + geom_boxplot()

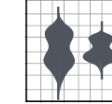
lower, middle, upper, x, ymax, ymin, alpha, color, fill, linetype, shape, size, weight



g + geom_dotplot(binaxis = "y",

stackdir = "center")

x, y, alpha, color, fill

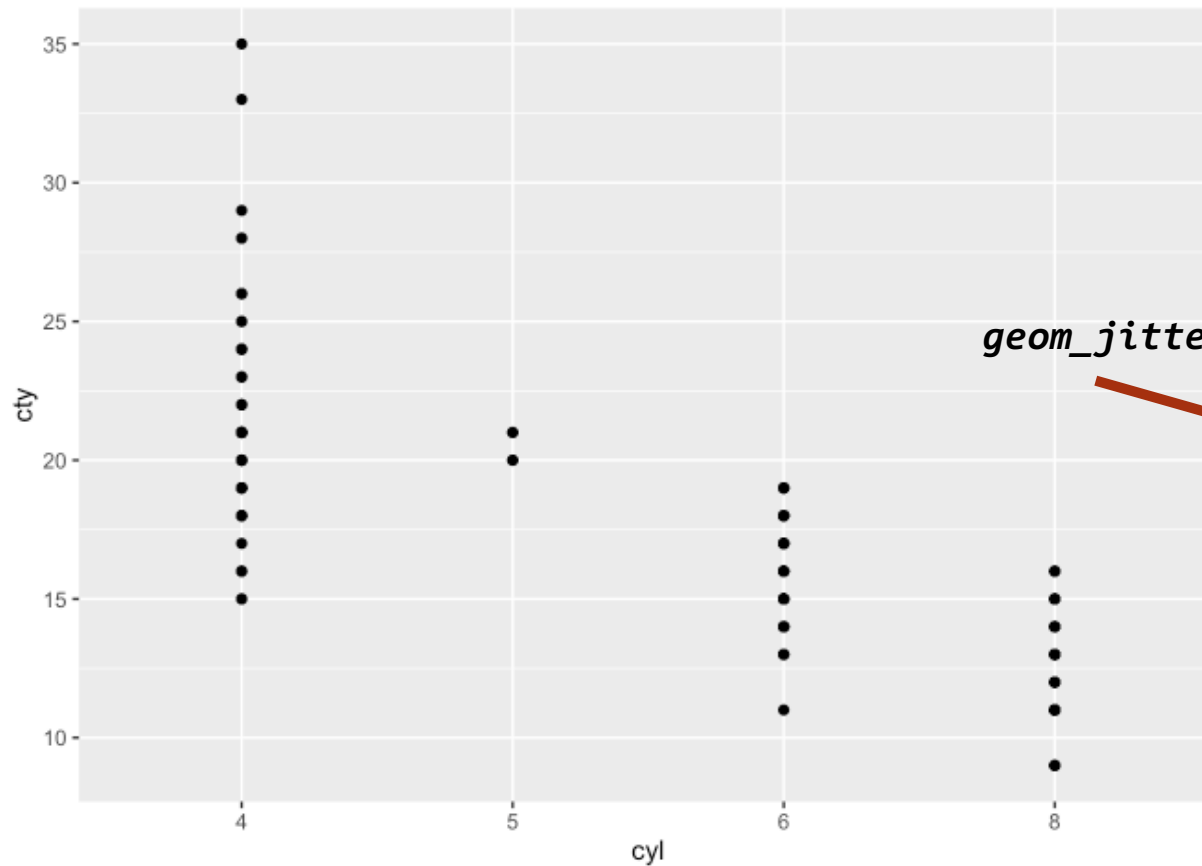


g + geom_violin(scale = "area")

x, y, alpha, color, fill, linetype, size, weight

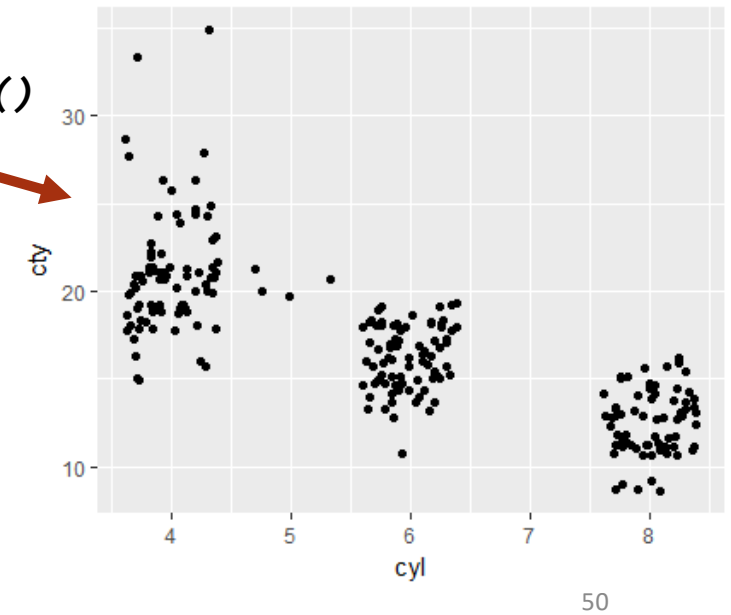
Construcción por capas: GEOMS.

```
ggplot(mpg, aes(cyl, cty)) + geom_point()
```



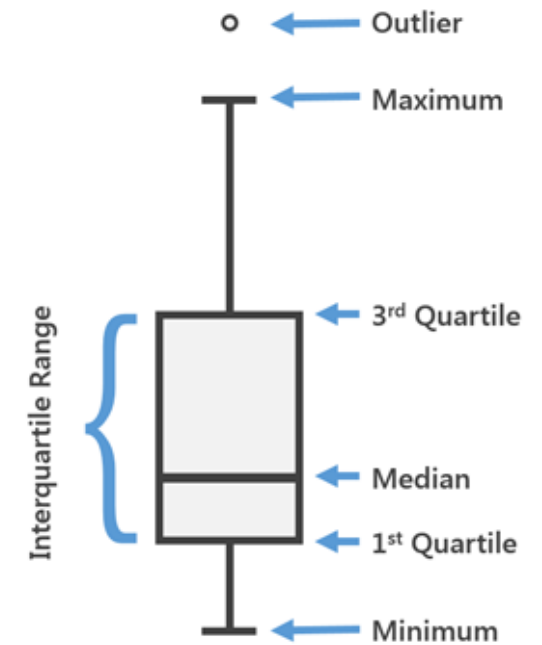
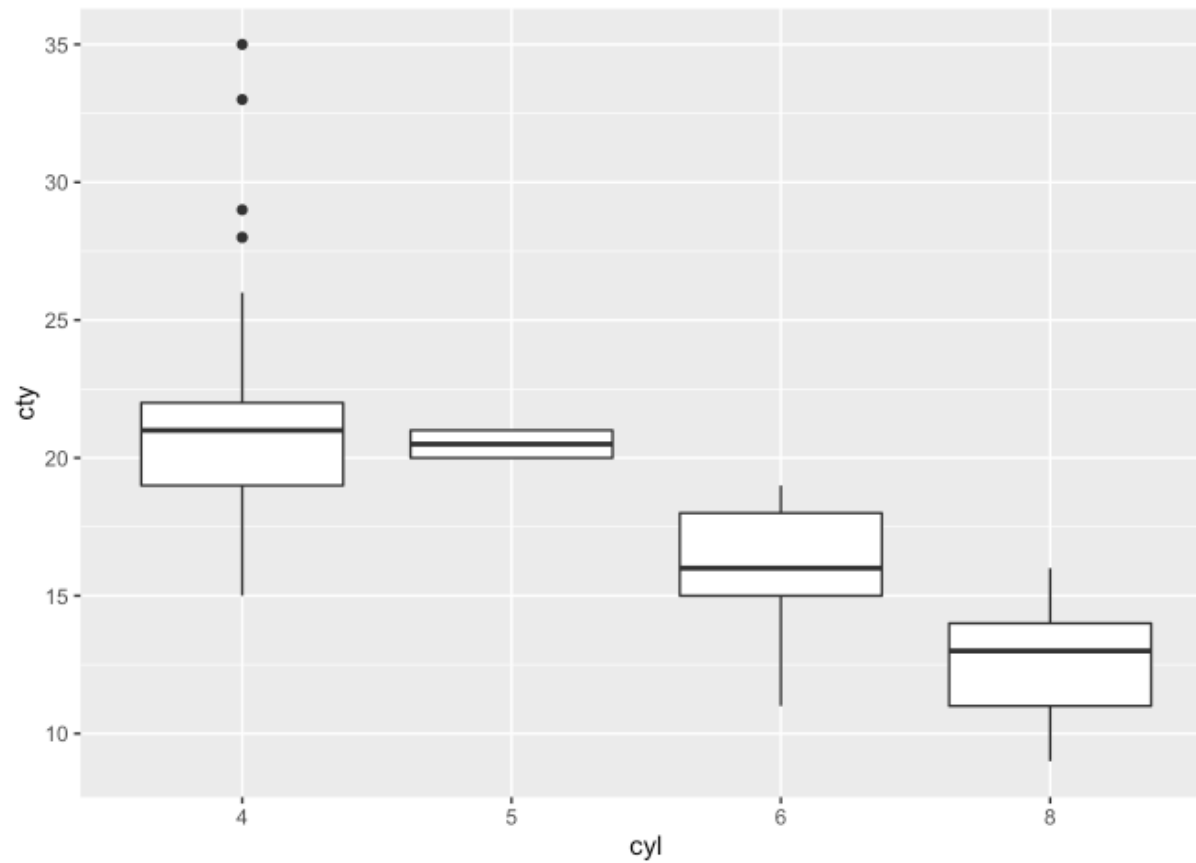
Comportamiento de **variables continuas** con respecto a **variables discretas**.

geom_jitter()



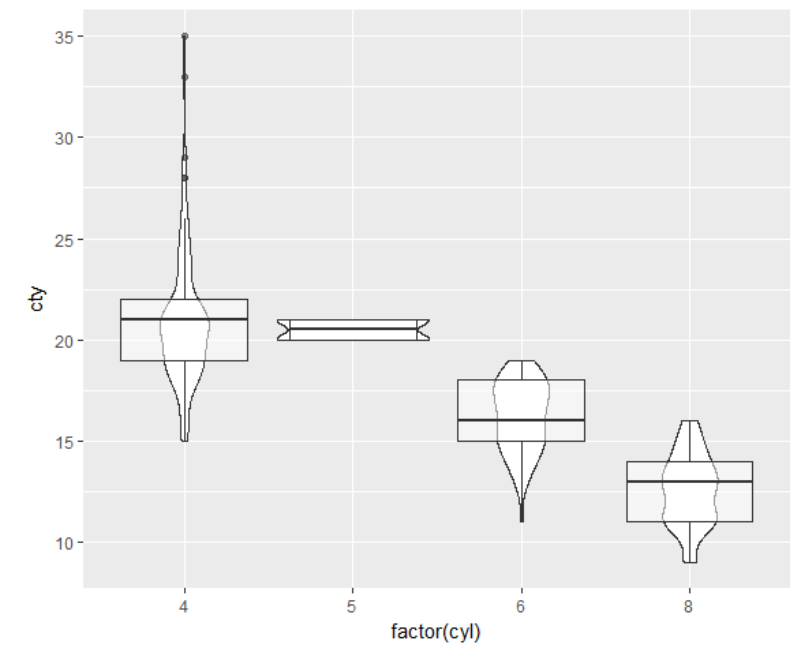
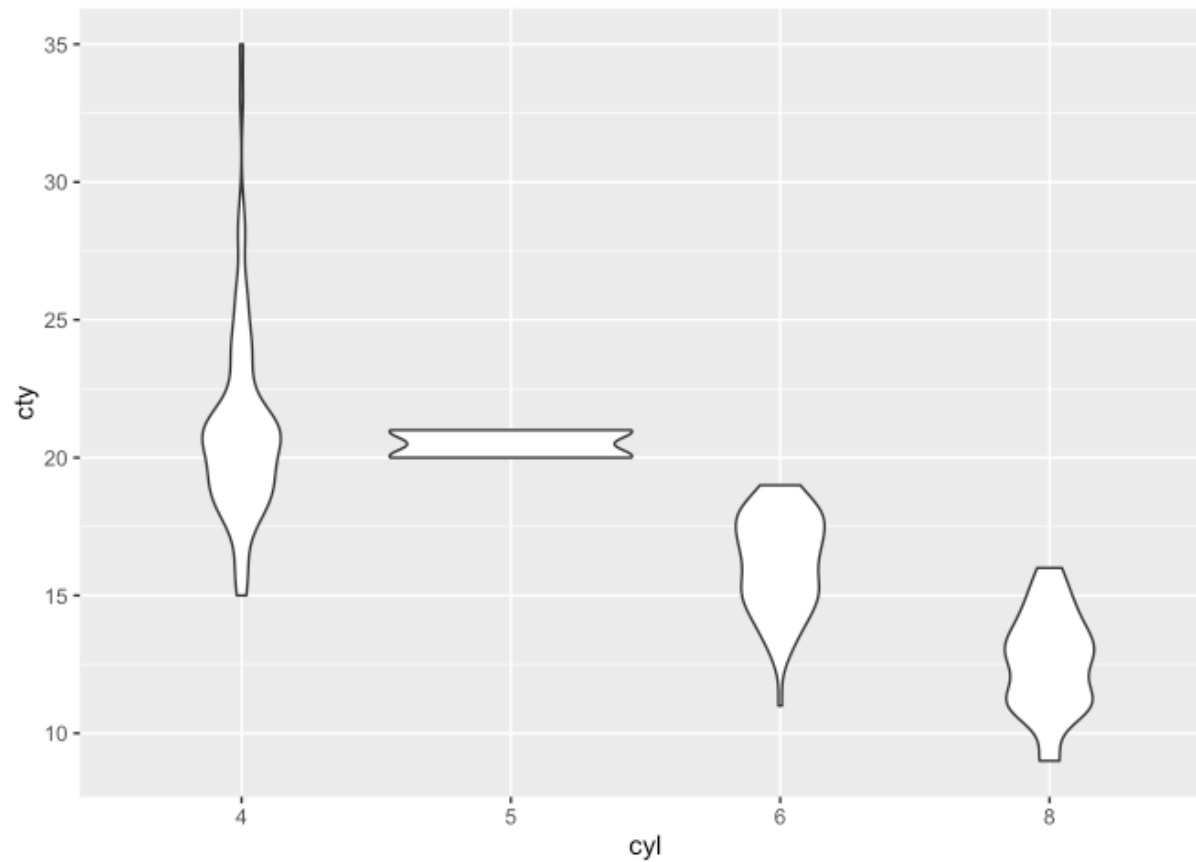
Construcción por capas: GEOMS.

```
ggplot(mpg, aes(cyl, cty)) + geom_boxplot()
```



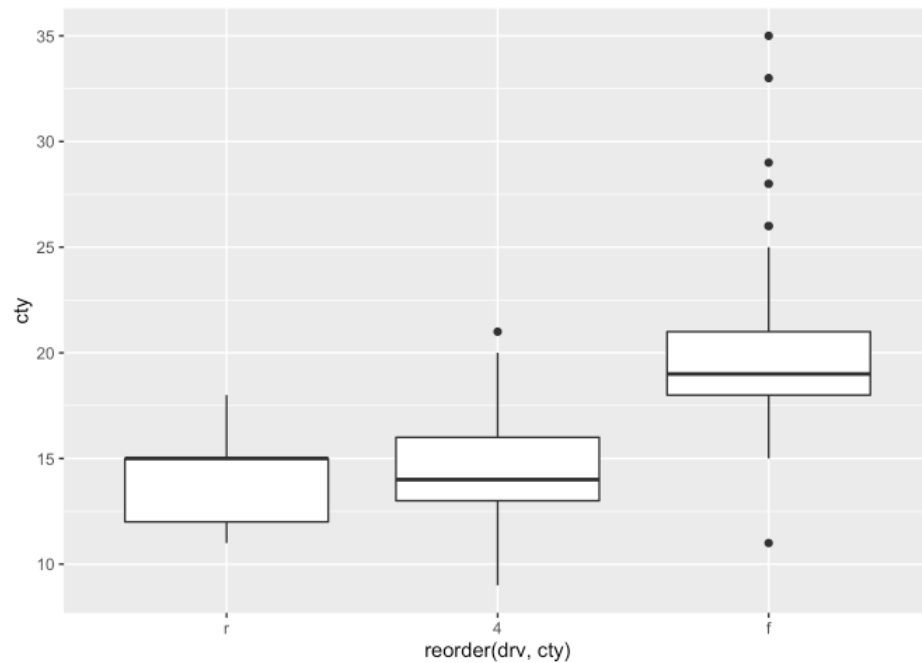
Construcción por capas: GEOMS.

```
ggplot(mpg, aes(cyl, cty)) + geom_violin()
```

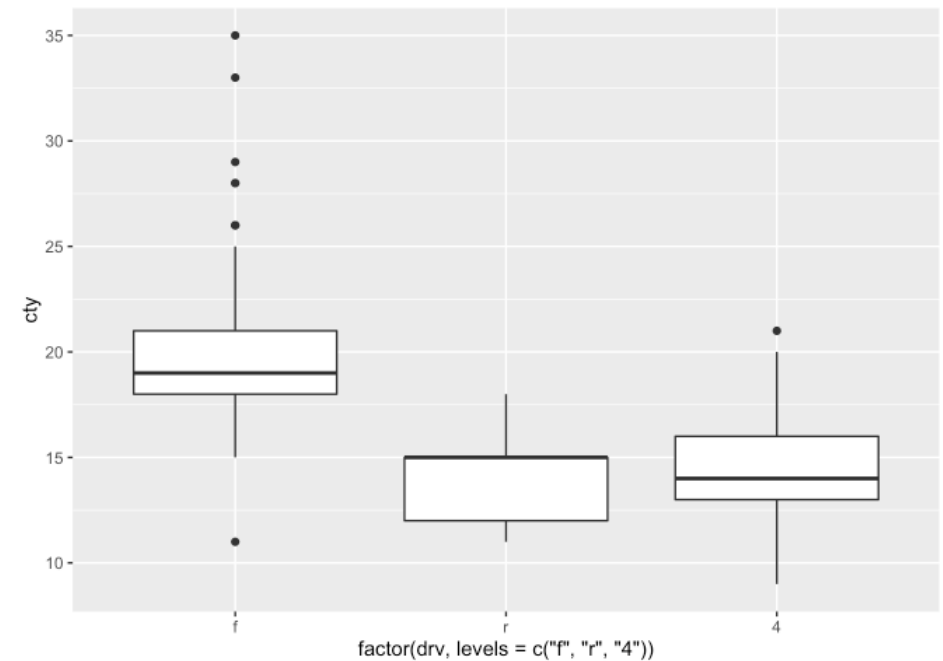


Construcción por capas: GEOMS.

```
ggplot(mpg, aes(reorder(drv, cty))) + geom_boxplot()
```

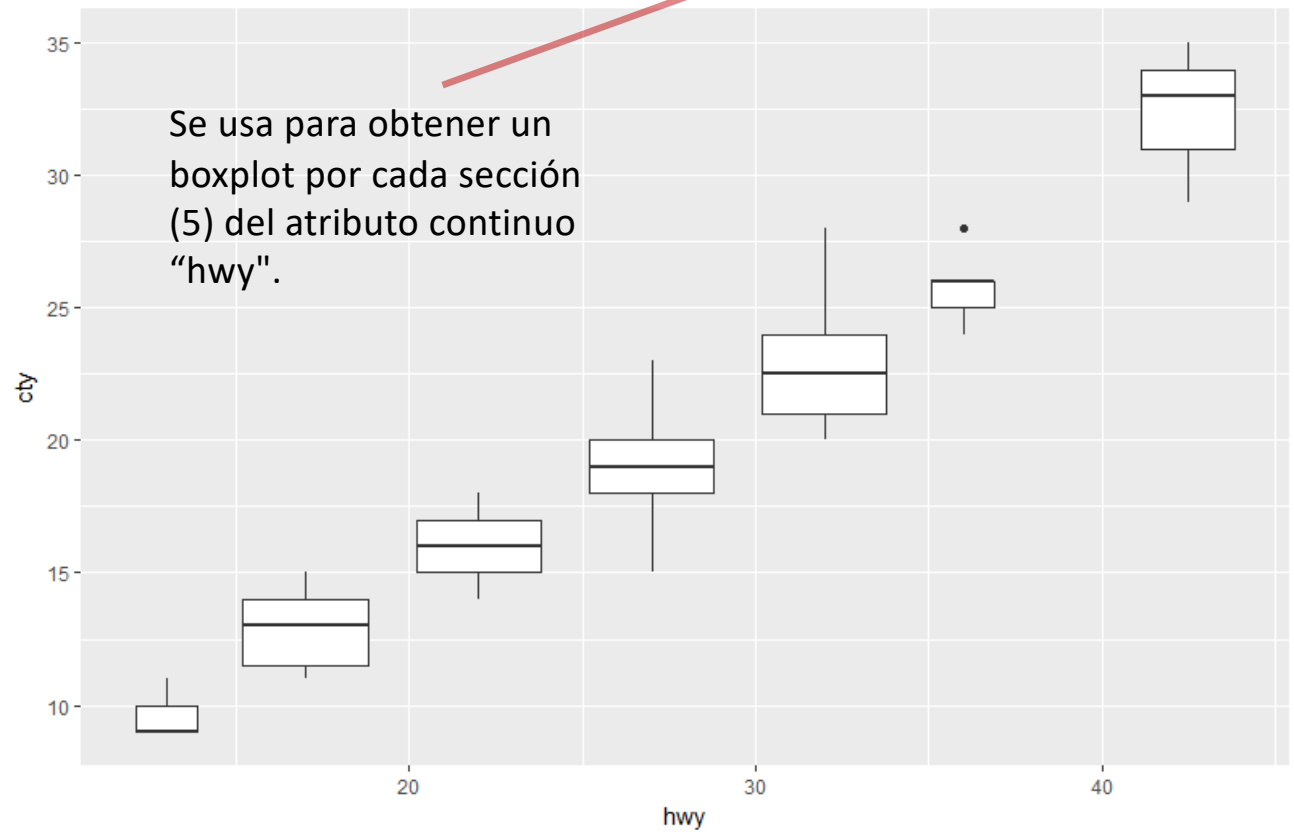
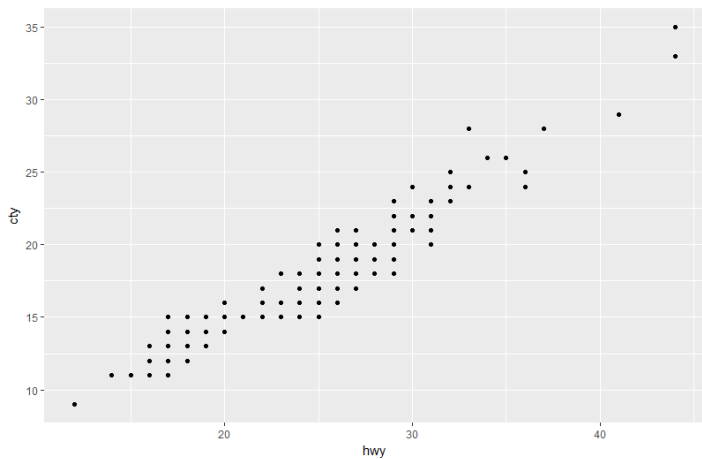


```
ggplot(mpg, aes(factor(drv, levels=c('f', 'r', '4')),  
              cty)) +  
  geom_boxplot()
```



Construcción por capas: GEOMS.

```
ggplot(mpg, aes(hwy, cty, group=round_any(hwy, 5, floor))) +  
  geom_boxplot()
```

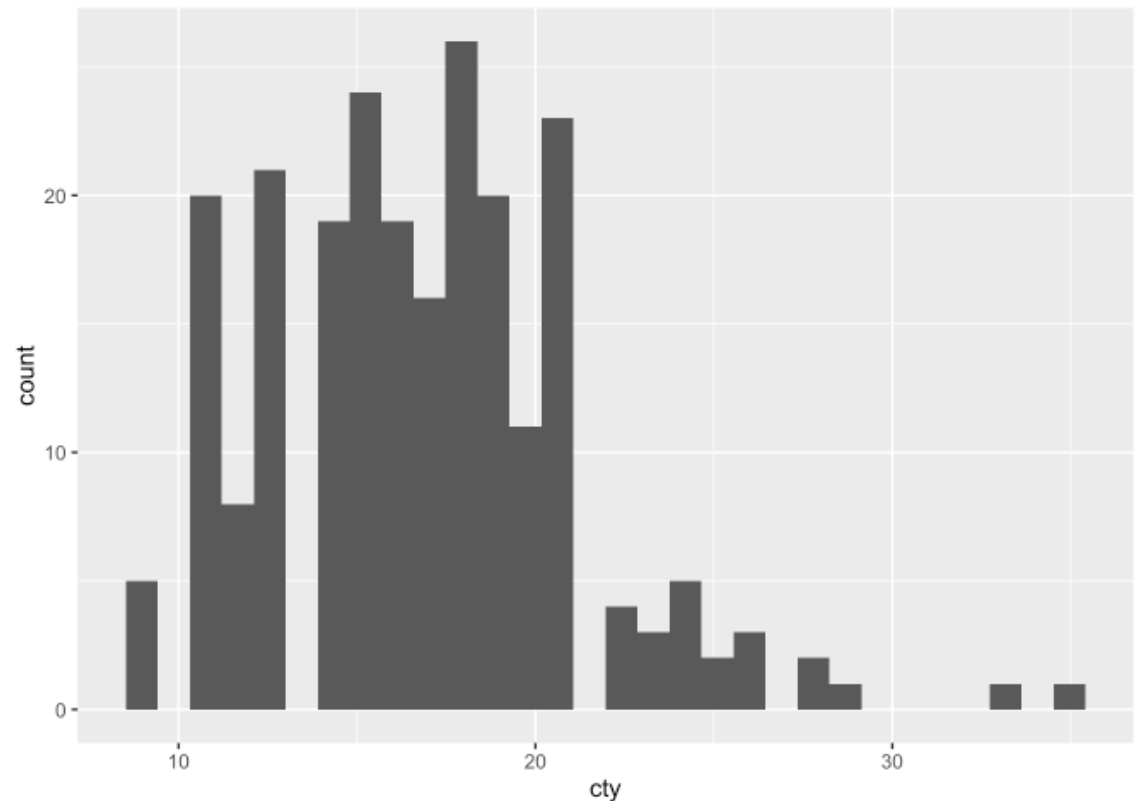


Construcción por capas: GEOMS.

- Para **distribuciones continuas 1D** se tiene la geometría de tipo histogram:
`geom_histogram()`
- Entiende las estéticas:
 - **binwidth** : para el tamaño de los contenedores.
 - **bin** : número de contenedores.
 - **position**: para indicar la disposición de las “barras”.

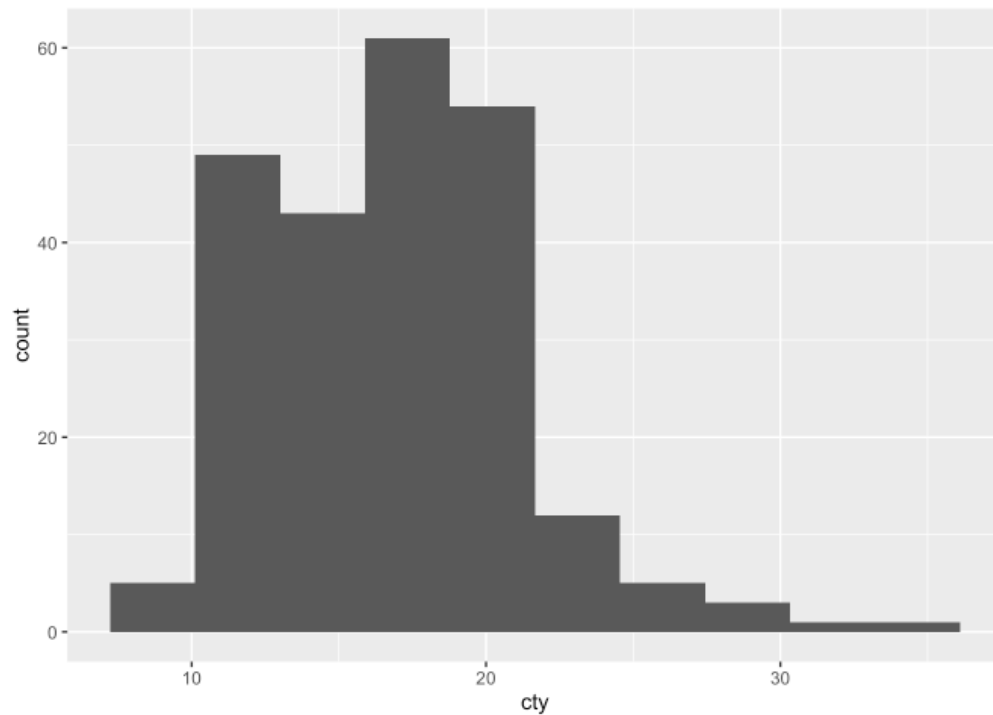
```
ggplot(mpg, aes(cty)) + geom_histogram()
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

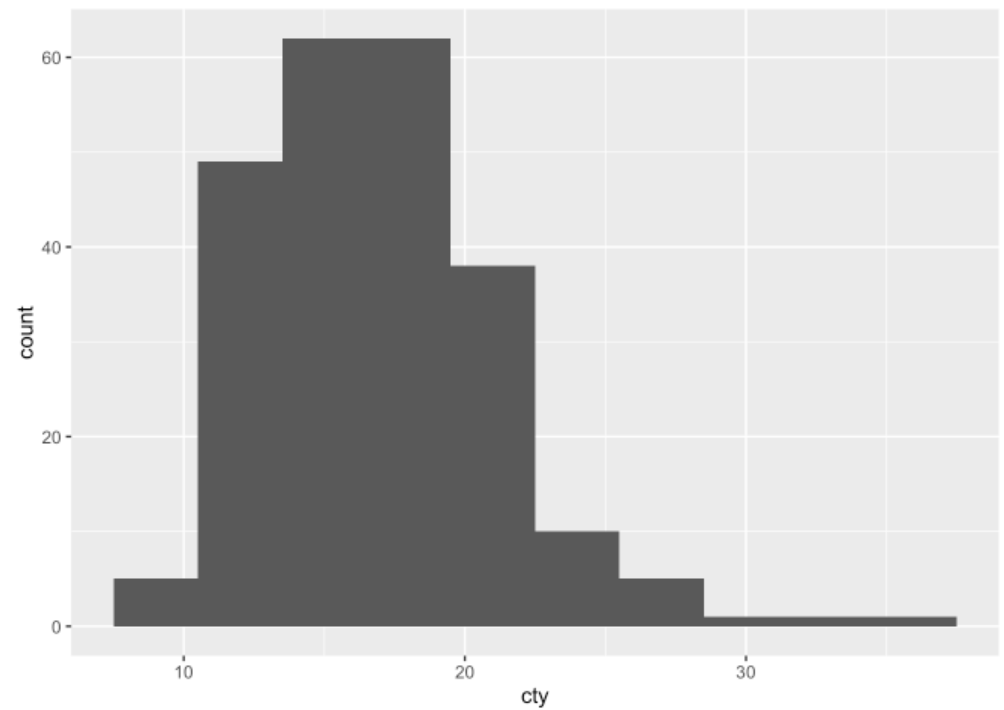


Construcción por capas: GEOMS.

```
ggplot(mpg, aes(cty)) + geom_histogram(bins=10)
```

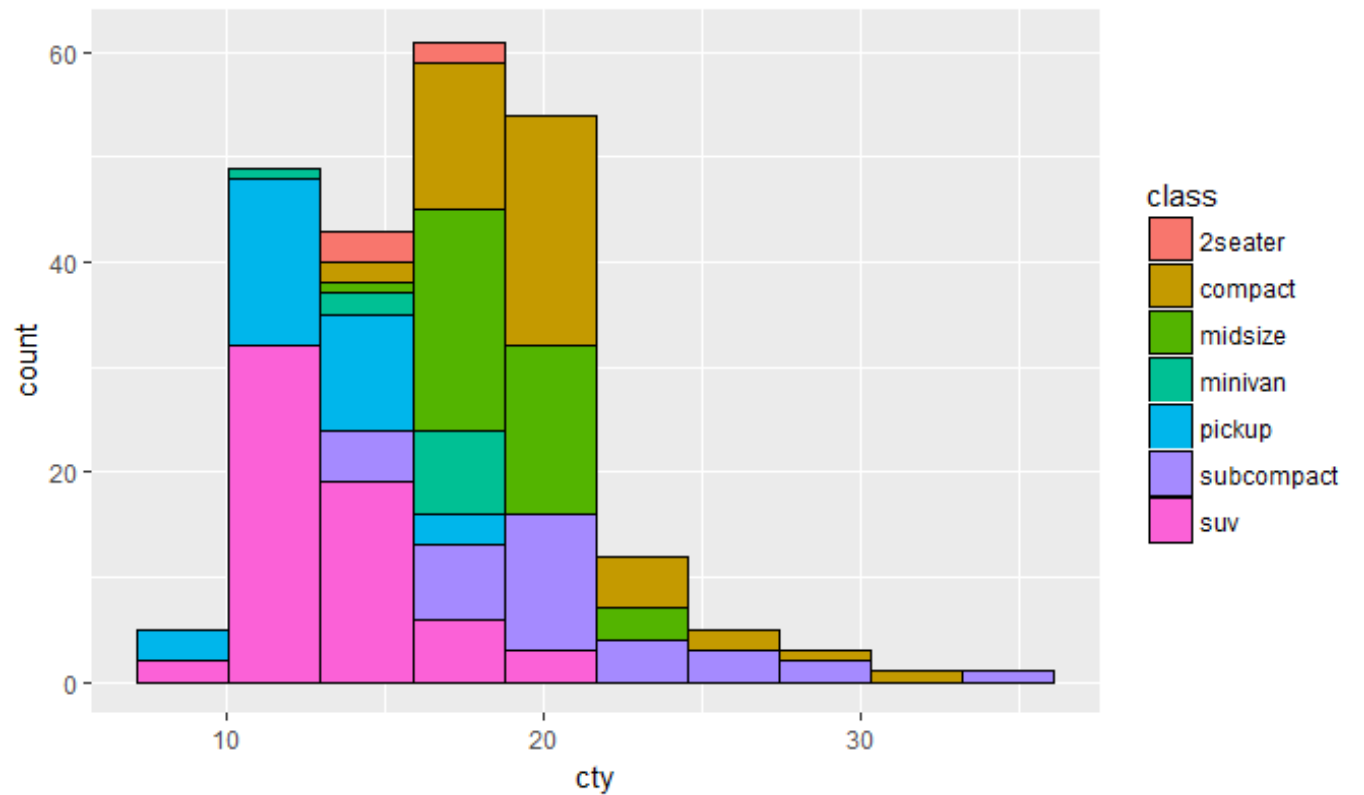
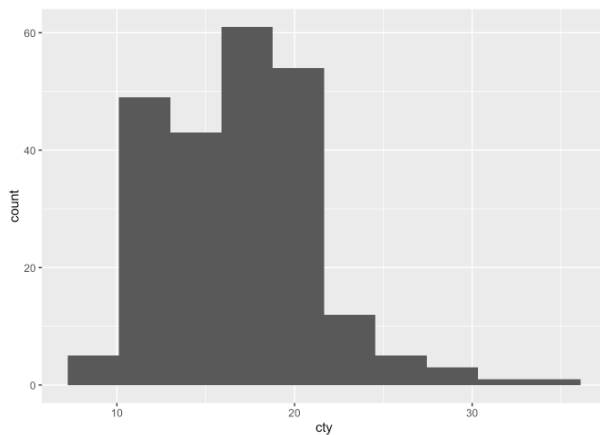


```
ggplot(mpg, aes(cty)) + geom_histogram(binwidth=3)
```



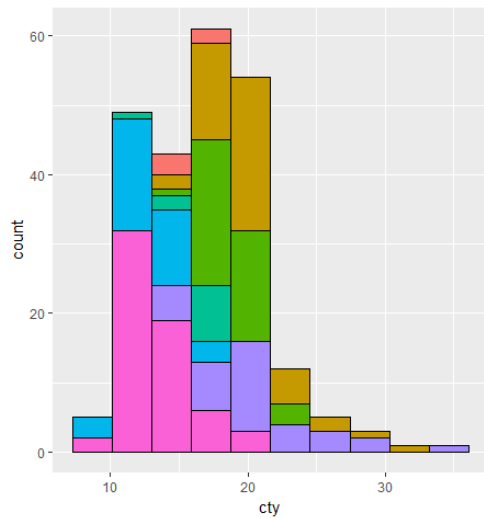
Construcción por capas: GEOMS.

```
ggplot(mpg, aes(cty, fill = class)) + geom_histogram(bins = 10, colour = "black")
```



Construcción por capas: GEOM

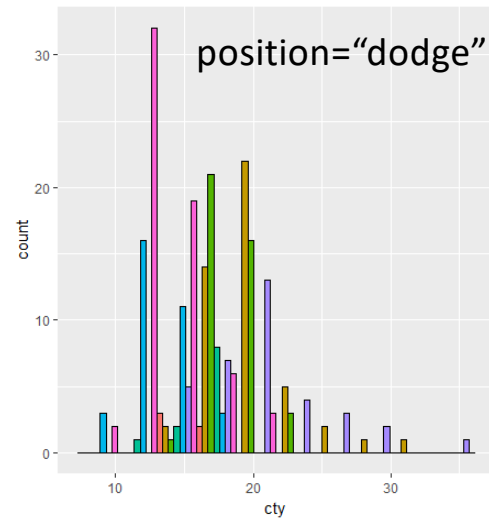
```
ggplot(mpg, aes(cty, fill = class)) + geom_histogram(bins = 10, position = "X")
```



position="stack"

class

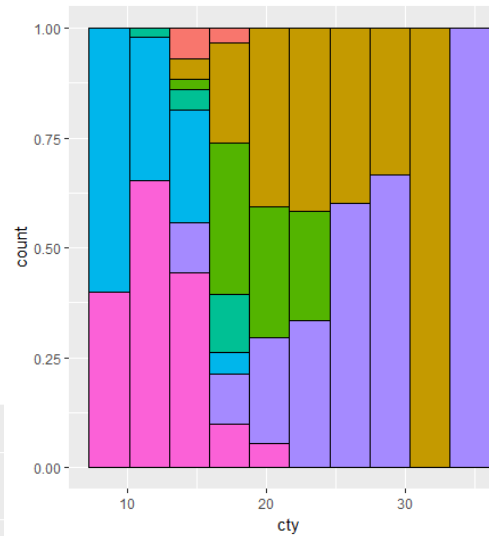
- 2seater
- compact
- midsize
- minivan
- pickup
- subcompact
- suv



position="dodge"

class

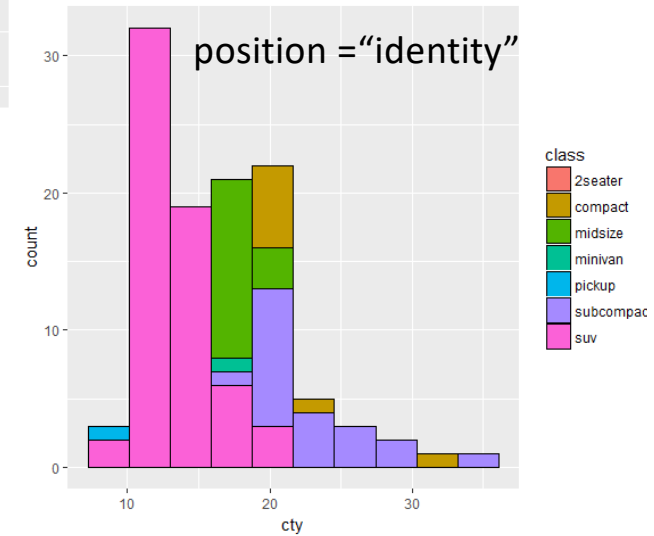
- 2seater
- compact
- midsize
- minivan
- pickup
- subcompact
- suv



position="fill"

class

- 2seater
- compact
- midsize
- minivan
- pickup
- subcompact
- suv



position="identity"

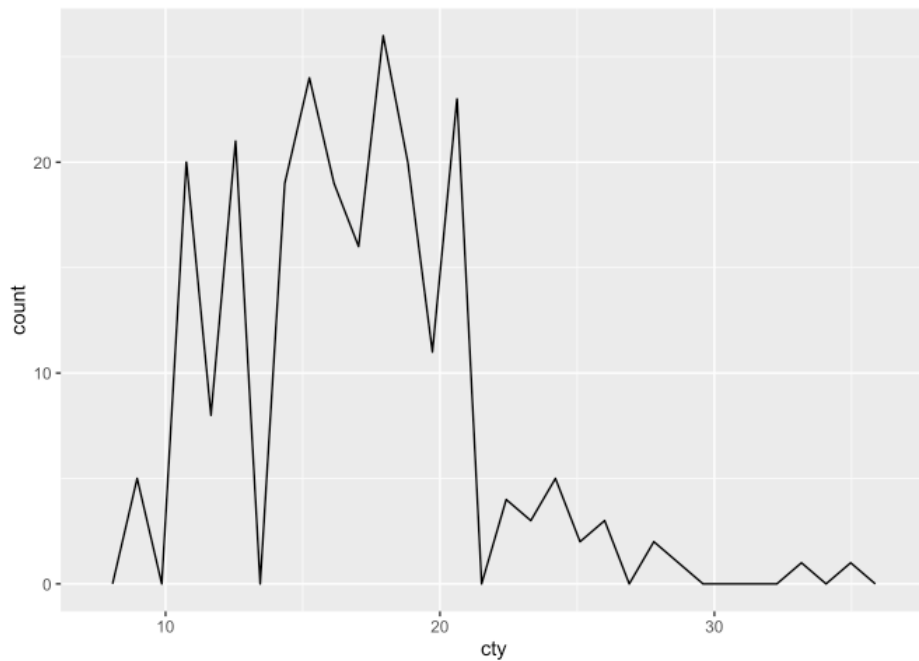
class

- 2seater
- compact
- midsize
- minivan
- pickup
- subcompact
- suv

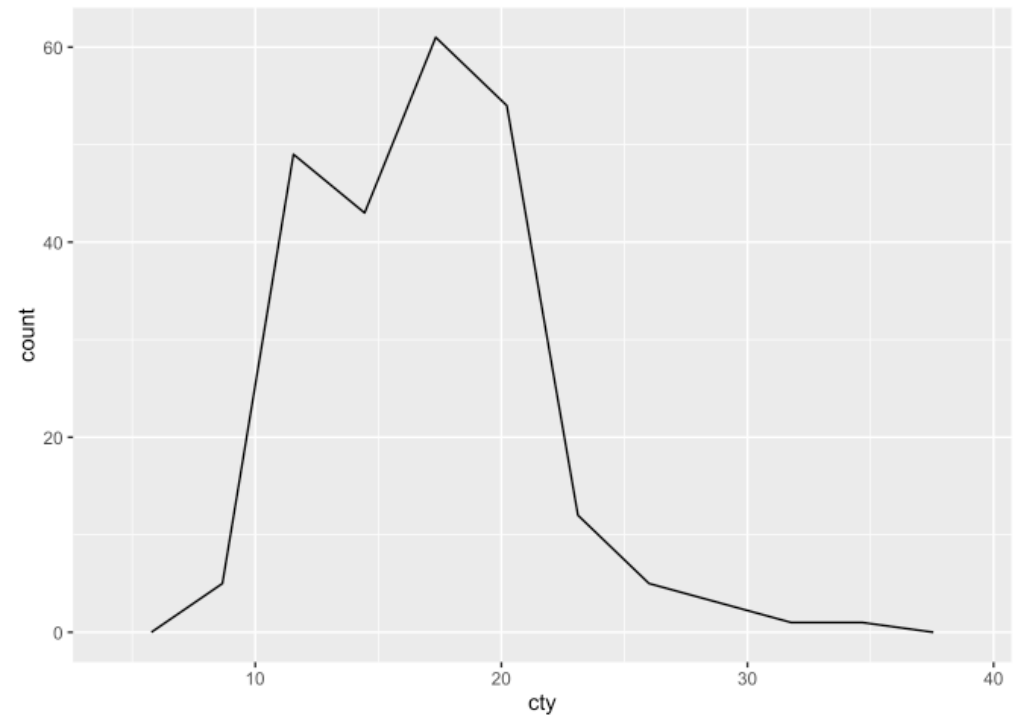
Construcción por capas: GEOMS.

```
ggplot(mpg, aes(cty)) + geom_freqpoly()
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

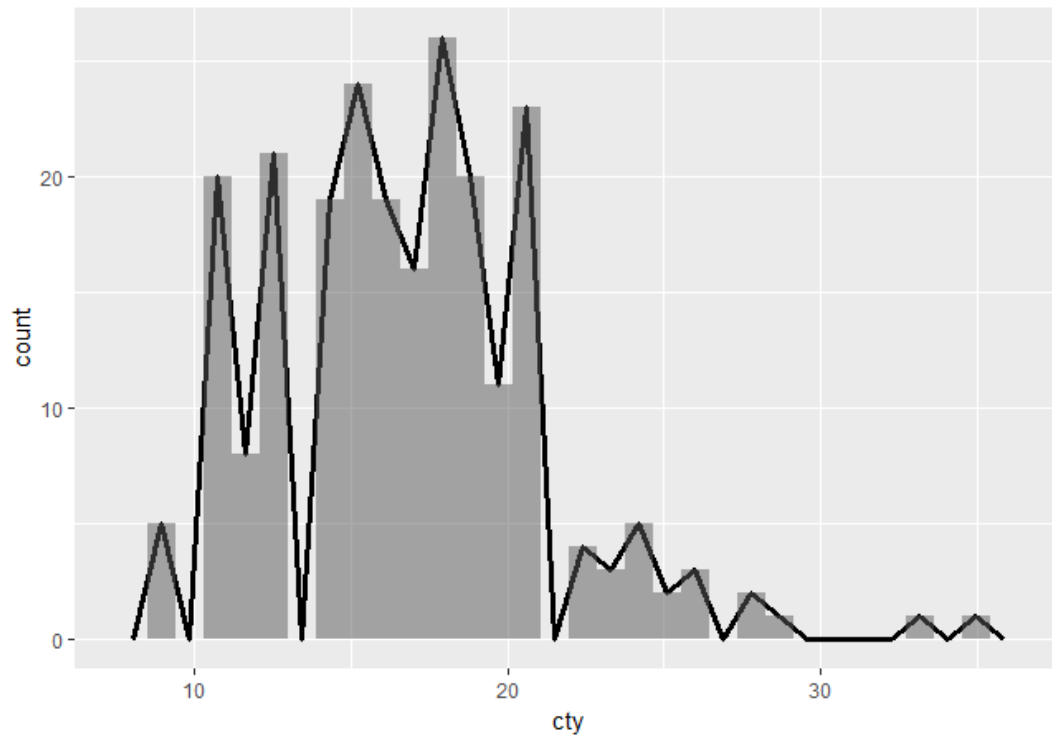


```
ggplot(mpg, aes(cty)) +  
  geom_freqpoly(bins = 10)
```

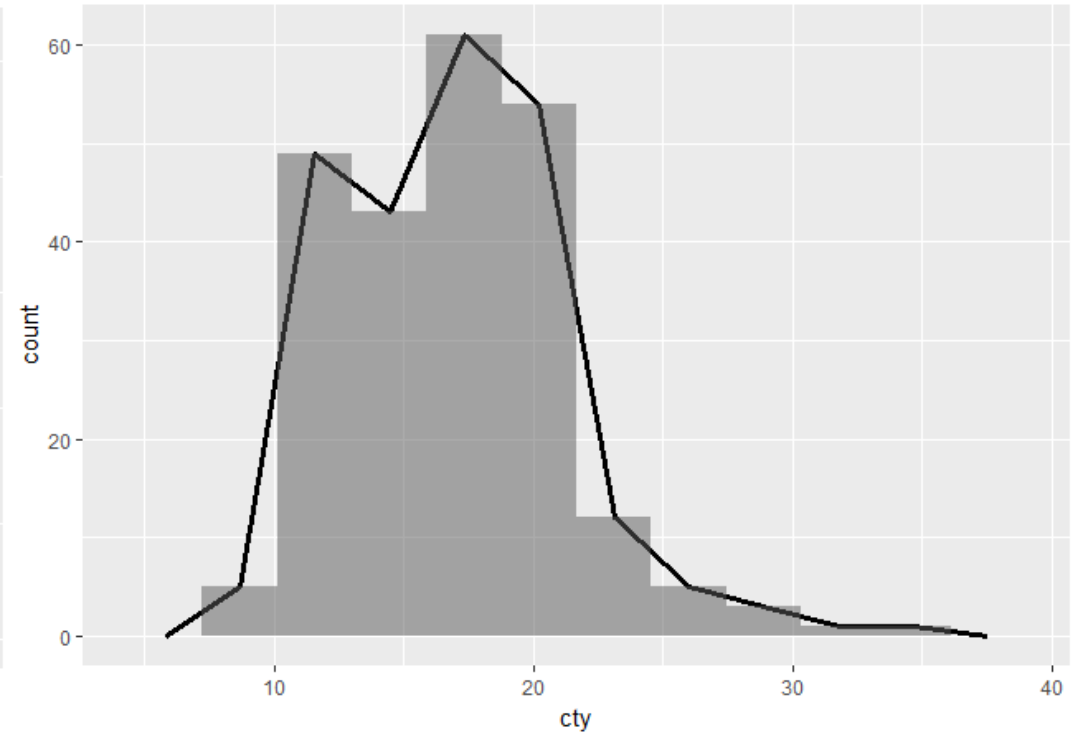


Construcción por capas: GEOMS.

```
ggplot(mpg, aes(cty)) + geom_freqpoly()
```

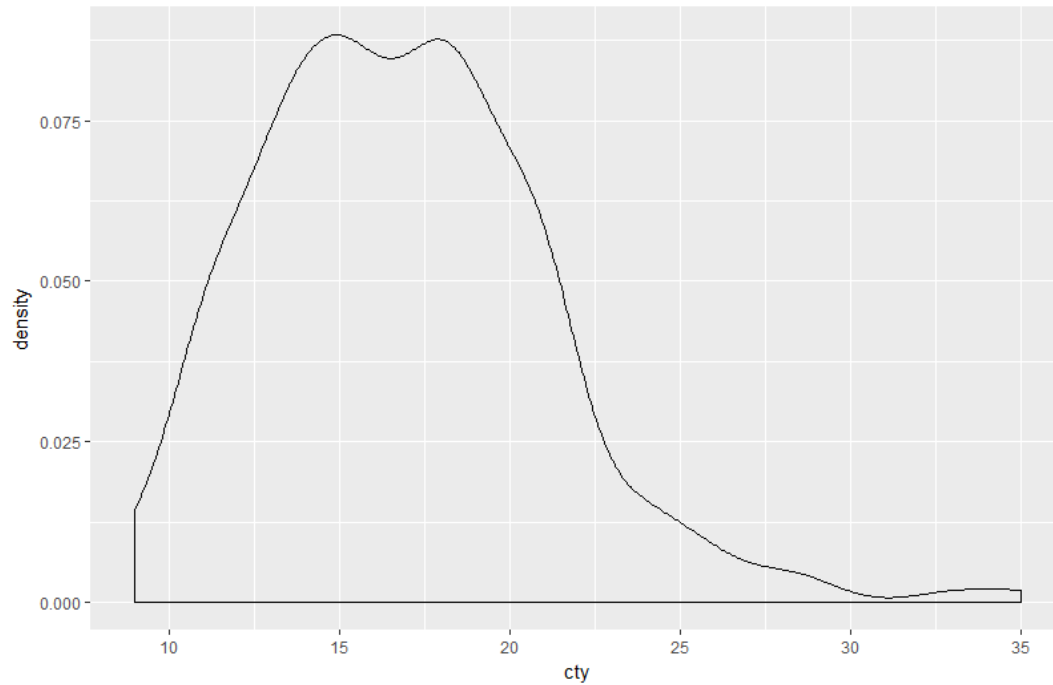


```
ggplot(mpg, aes(cty)) +  
  geom_freqpoly(bins = 10)
```

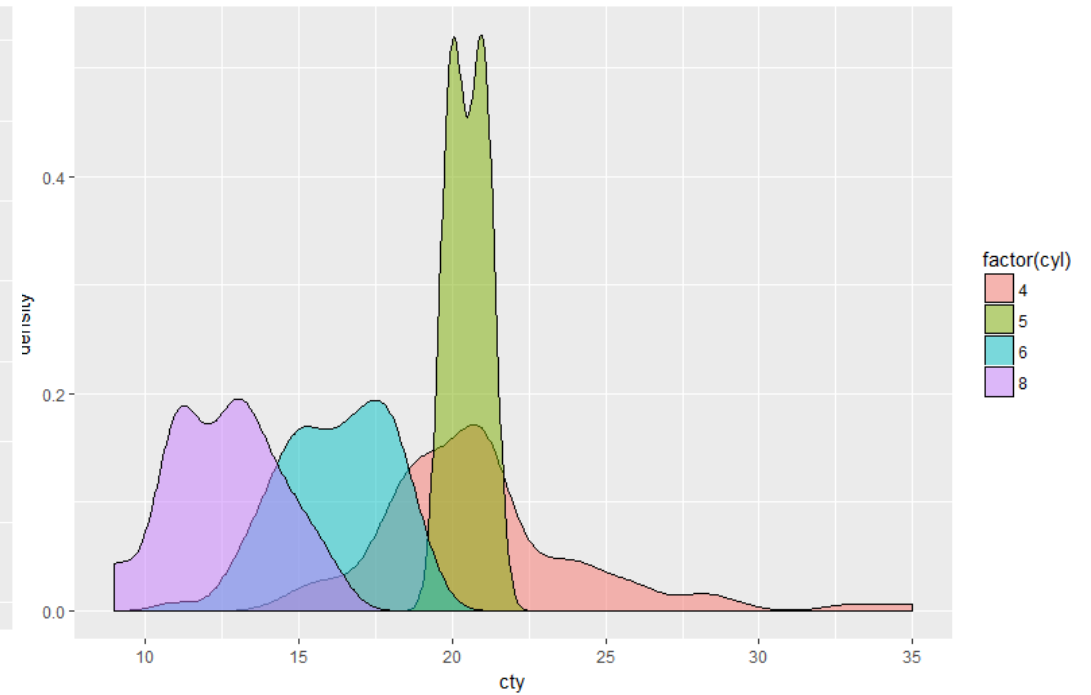


Detalles de ggplot: Distribuciones.

```
ggplot(mpg, aes(cty)) + geom_density()
```

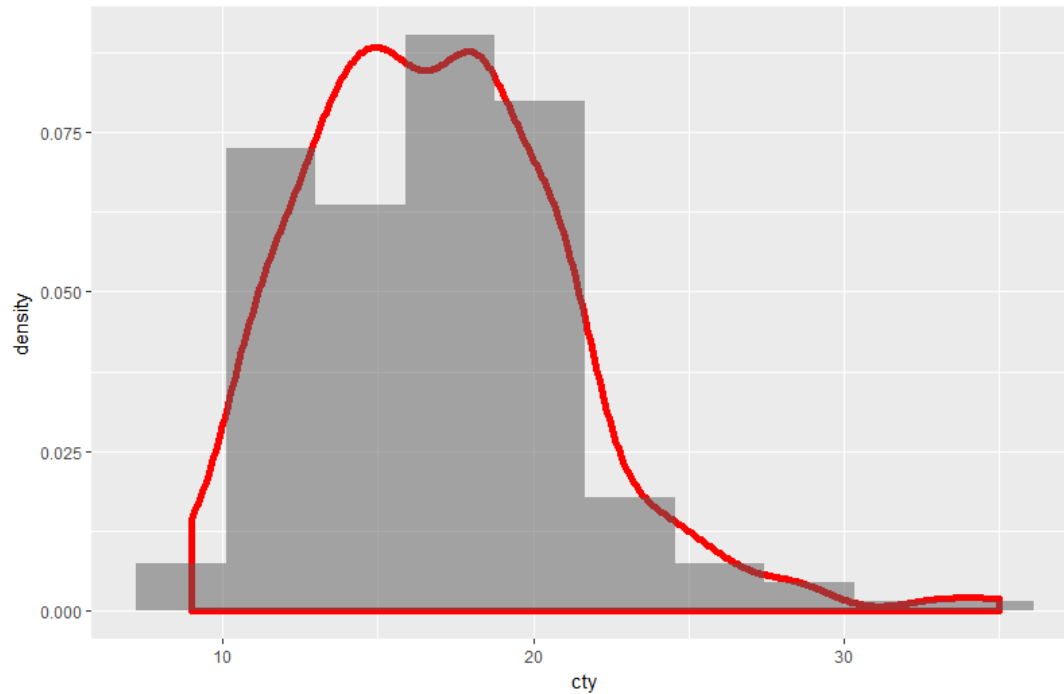


```
ggplot(mpg, aes(cty, fill = cyl)) +  
  geom_density(alpha = 1/2)
```

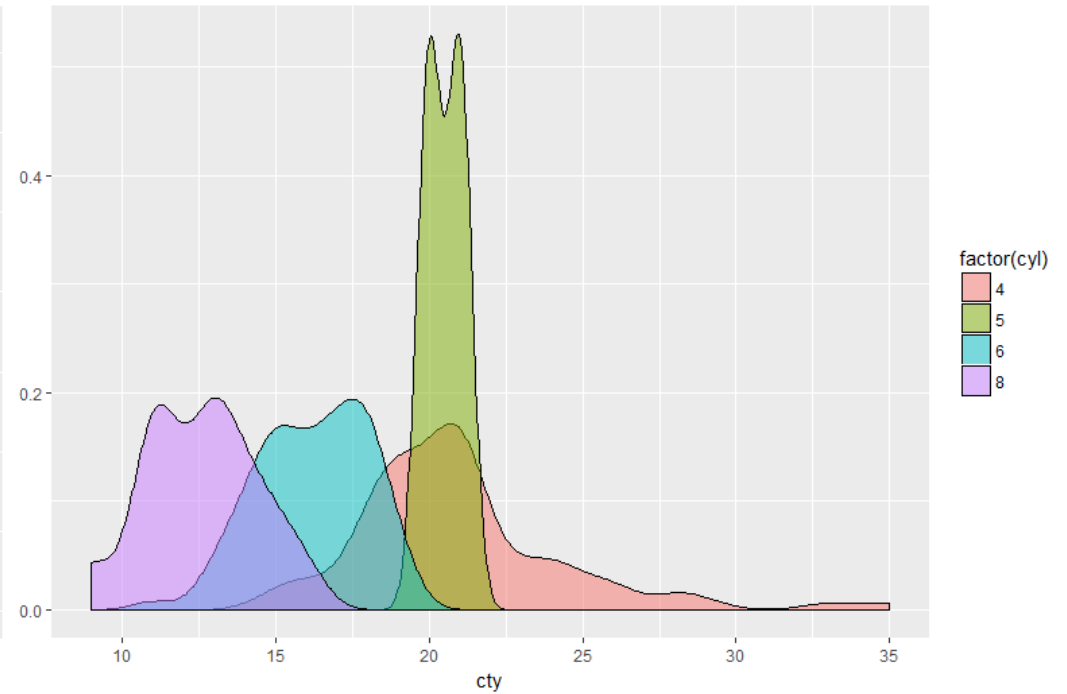


Detalles de ggplot: Distribuciones.

```
ggplot(mpg, aes(cty)) +  
  geom_density(colour = "red", size = 2) +  
  geom_histogram(aes(y=..density..),  
    bins = 10, alpha = 1/2)
```



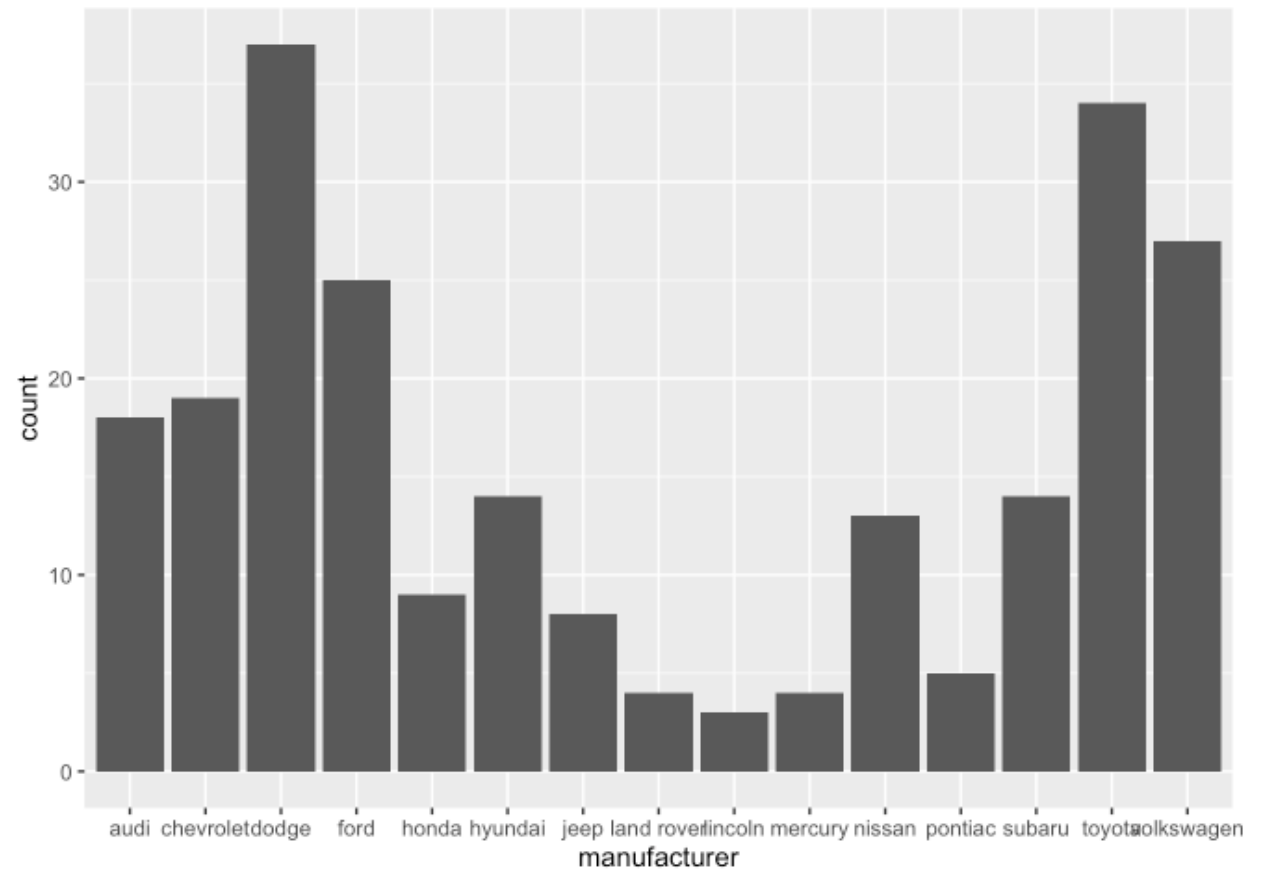
```
ggplot(mpg, aes(cty, fill = cyl)) +  
  geom_density(alpha = 1/2)
```



Construcción por capas: GEOMS.

- Para **distribuciones discretas 1D** se utiliza la geometría de tipo barra (`geom_bar()`).

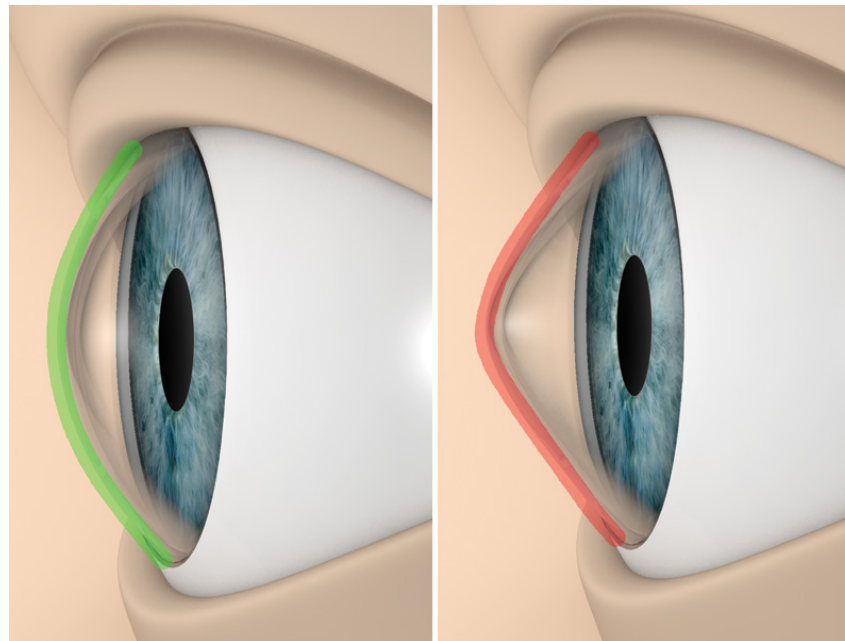
```
ggplot(mpg, aes(manufacturer)) + geom_bar()
```



ggplot: Ejercicio práctico.

Ej1. ggplot() y la base de datos de queratocono

Análisis de información mediante ggplot().

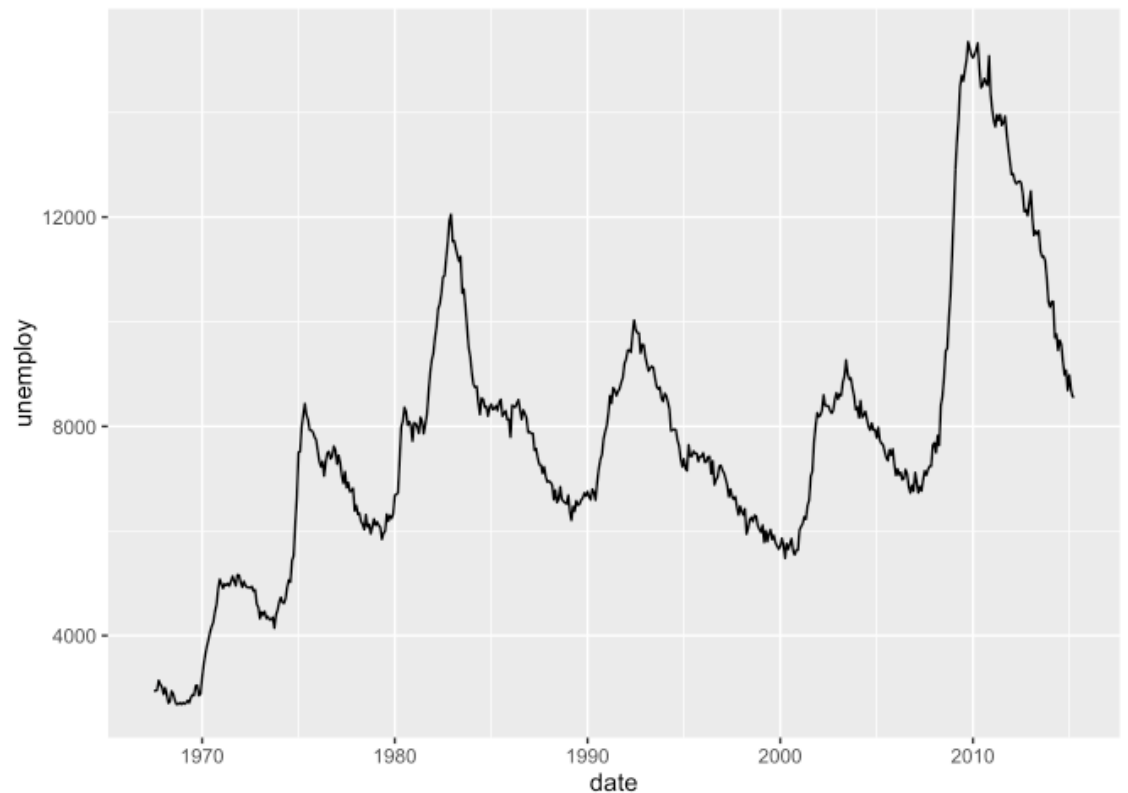


Construcción por capas: GEOMS.

- Series temporales:
 - Se trazan usando gráficos de líneas para unir las observaciones consecutivas.
 - Usualmente, graficamos el tiempo en el eje X.

	date	pce	pop	psavert	uempmed	unemploy
1	1967-07-01	507.4	198712	12.5	4.5	2944
2	1967-08-01	510.5	198911	12.5	4.7	2945
3	1967-09-01	516.3	199113	11.7	4.6	2958
4	1967-10-01	512.9	199311	12.5	4.9	3143
5	1967-11-01	518.1	199498	12.5	4.7	3066
6	1967-12-01	525.8	199657	12.1	4.8	3018
7	1968-01-01	531.5	199808	11.7	5.1	2878
8	1968-02-01	534.2	199920	12.2	4.5	3001
9	1968-03-01	544.9	200056	11.6	4.1	2877
10	1968-04-01	544.6	200208	12.2	4.6	2709
11	1968-05-01	550.4	200361	12.0	4.4	2740
12	1968-06-01	556.8	200536	11.6	4.4	2938
13	1968-07-01	563.8	200706	10.6	4.5	2883

```
ggplot(economics, aes(date, unemploy)) + geom_line()
```

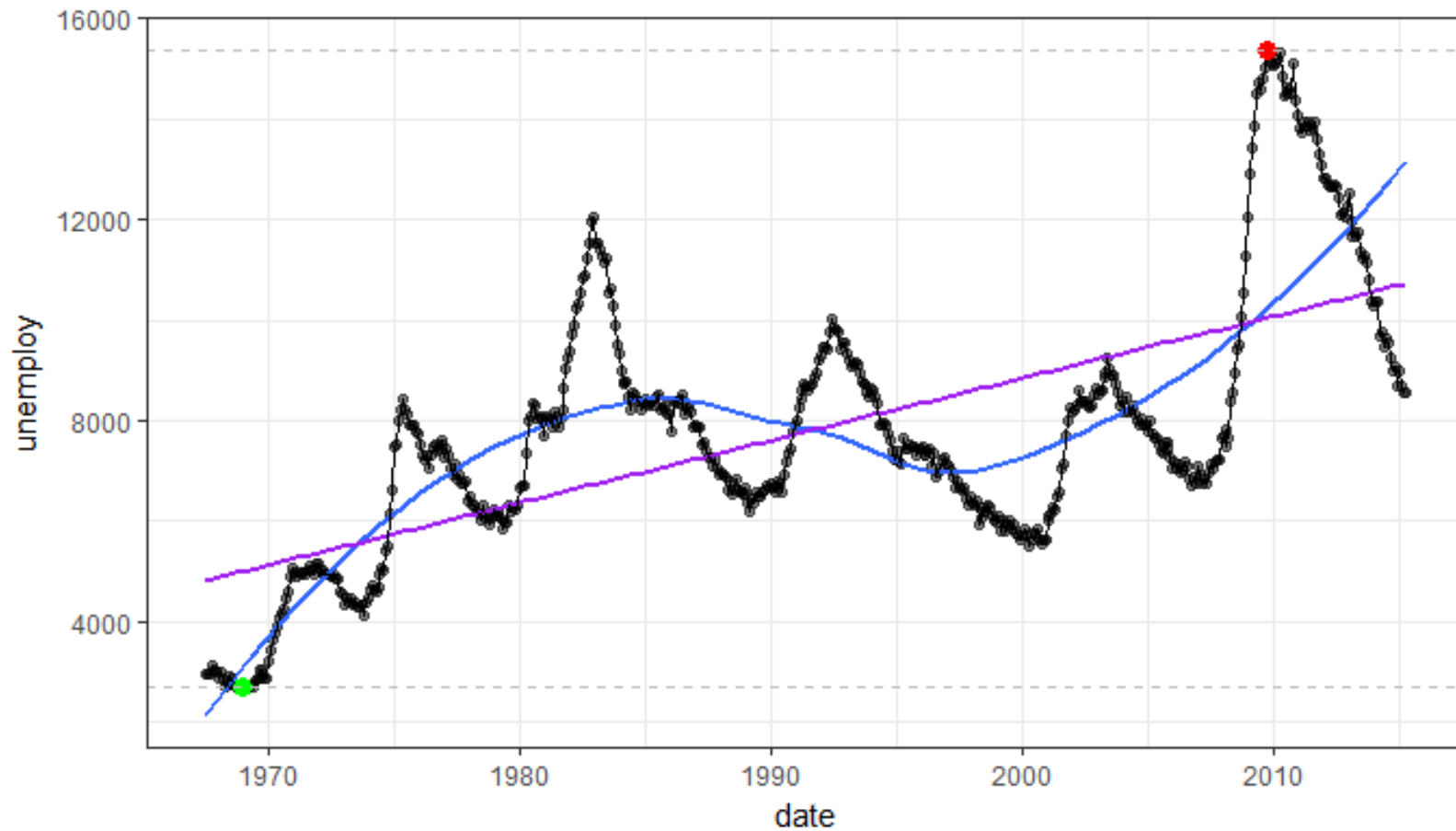


Construcción por capas: GEOMS

- Para anotar gráficos se pueden usar los siguientes **geoms**:
 - **geom_text** : para añadir descripciones de tipo texto o etiquetar puntos.
 - **geom_vline** : para añadir líneas verticales.
 - **geom_hline** : para añadir líneas horizontales.
 - **geom_abline** : para añadir líneas de cualquier inclinación.
 - **geom_rect** : para resaltar zonas rectangulares.
 - **geom_line**, **geom_path** y **geom_segment** : para añadir líneas

Construcción por capas: GEOMS.

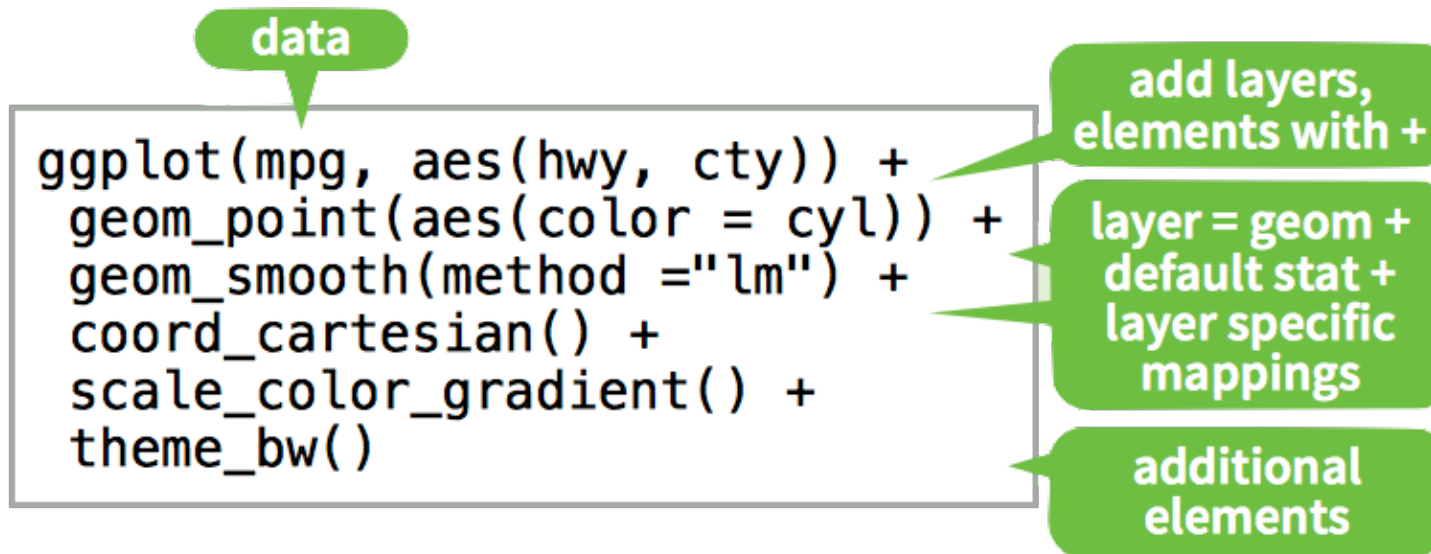
TEST AULA 3



Construcción por capas: GEOMS.

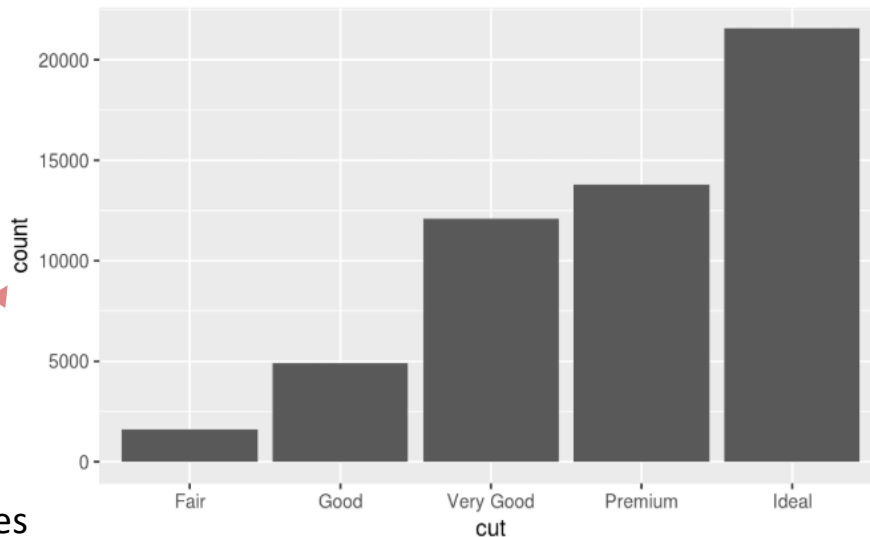
- Los gráficos básicos 2D requieren, como mínimo, de las estéticas `x`, `y`.
- **Todos los gráficos 2D entienden:** color (`colour`), tamaño (`size`) y relleno (`fill`).
- La geometría tipo **punto** (`point`) entiende la forma (`shape`).
- La geometría tipo **línea** (`line` o `path`) entiende el tipo de línea (`linetype`).

Construcción por capas: GEOMS.



Construcción por capas: STATS

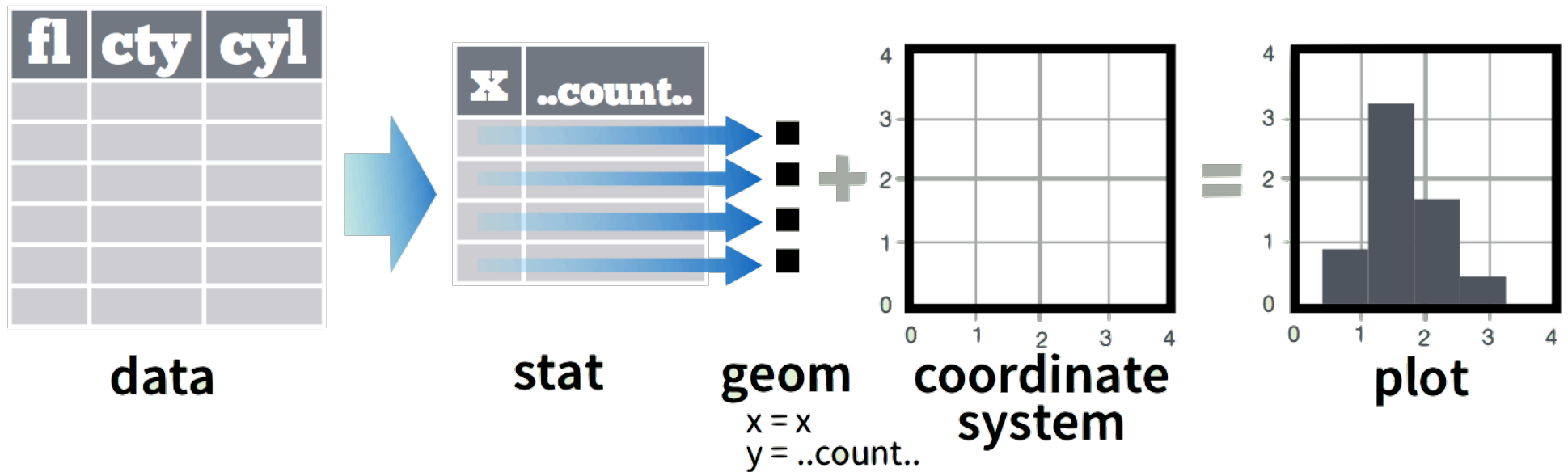
```
ggplot(data = diamonds) +  
  geom_bar(mapping = aes(x = cut))
```



count no es
una variable de
diamonds!

- Muchos gráficos (e.g., diagramas de dispersión) muestran los valores brutos de un conjunto de datos.
- Sin embargo:
 - Los gráficos de barras, histogramas y polígonos de frecuencia almacenan sus datos y luego **grafican los conteos** (por contenedor).
 - Los modelos de ajuste generan un modelo y pintan las predicciones del mismo.
 - Los diagramas de cajas calculan un resumen de la distribución y lo muestran con formato particular.
- Los algoritmos utilizados para calcular estos nuevos valores para un gráfico son los llamados **stat**, abreviatura de transformación estadística.

Construcción por capas: STATS



- Cada estadística crea variables adicionales para mapear la estética. Estas variables usan una sintaxis común de **..name..**.
- Las funciones **stat** y las funciones **geom** combinan una estadística con una geometría para crear una capa,
 - E.g., **stat_bin(geom = "bar")** hace lo mismo que **geom_bar(stat = "bin")**

Construcción por capas: STATS

1. `geom_bar()` begins with the **diamonds** data set

```
ggplot(data = diamonds) +  
  geom_bar(mapping = aes(x = cut))
```

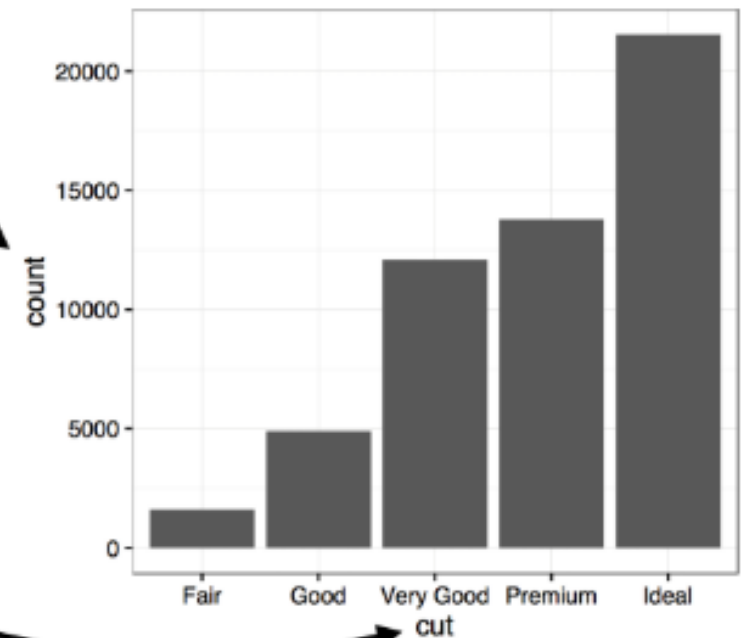
carat	cut	color	clarity	depth	table	price	x	y	z
0.23	Ideal	E	SI2	61.5	55	326	3.95	3.98	2.43
0.21	Premium	E	SI1	59.8	61	326	3.89	3.84	2.31
0.23	Good	E	VS1	56.9	65	327	4.05	4.07	2.31
0.29	Premium	I	VS2	62.4	58	334	4.20	4.23	2.63
0.31	Good	J	SI2	63.3	58	335	4.34	4.35	2.75
...	...	...	...	...	...	...	...	...	...

2. `geom_bar()` transforms the data with the "count" stat, which returns a data set of cut values and counts.

`stat_count()`

cut	count	prop
Fair	1610	1
Good	4906	1
Very Good	12082	1
Premium	13791	1
Ideal	21551	1

3. `geom_bar()` uses the transformed data to build the plot. cut is mapped to the x axis, count is mapped to the y axis.



Construcción por capas: STATS

- P.e., **stat_bin** añade las siguientes variables:
 - **..count..**: número de observaciones de cada “bin”.
 - **..density..**: densidad de las observaciones (porcentaje/ancho_del_bin).
 - **..x..**: centro del compartimiento.

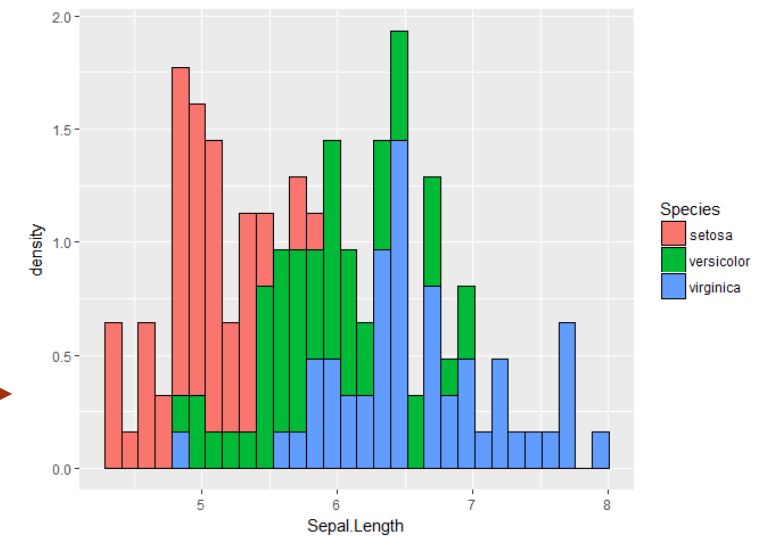
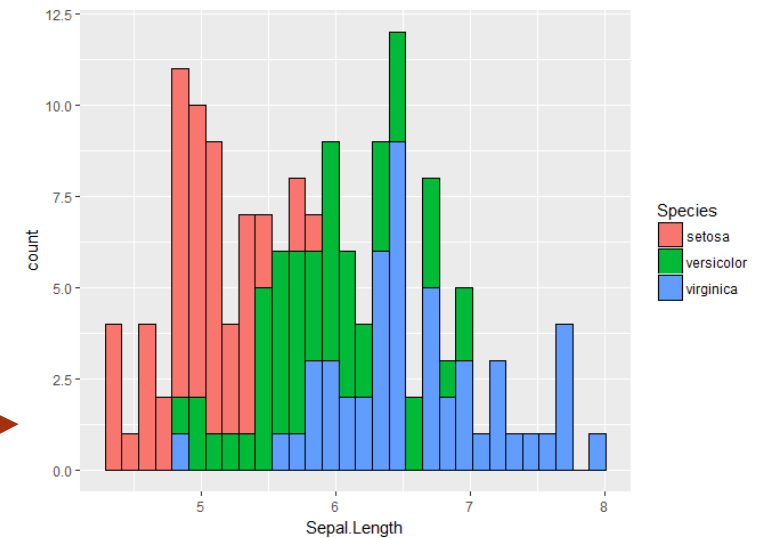
	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa

Stat_bin

- [4.4, 4.5) = 4
- [4.5, 4.6) = 1
- [4.6, 4.7) = 4
- ...

```
ggplot(iris) +  
geom_histogram(aes(Sepal.Length,  
y = ..count.., fill = Species),  
colour = "black")
```

```
ggplot(iris) +  
geom_histogram(aes(Sepal.Length,  
y = ..density.., fill = Species),  
colour = "black")
```



Construcción por capas: STATS.

Name	Description
bin	Bin data
boxplot	Calculate components of box-and-whisker plot
contour	Contours of 3d data
density	Density estimation, 1d
density_2d	Density estimation, 2d
function	Superimpose a function
identity	Don't transform data
qq	Calculation for quantile-quantile plot
quantile	Continuous quantiles
smooth	Add a smoother
spoke	Convert angle and radius to xend and yend
step	Create stair steps
sum	Sum unique values. Useful for overplotting on scatter-plots
summary	Summarise y values at every unique x
unique	Remove duplicates

Construcción por capas: STATS

Name	Default stat	Aesthetics
abline	abline	colour, linetype, size
area	identity	colour, fill, linetype, size, x, y
bar	bin	colour, fill, linetype, size, weight, x
bin2d	bin2d	colour, fill, linetype, size, weight, xmax, xmin, ymax, ymin
blank	identity	
boxplot	boxplot	colour, fill, lower, middle, size, upper, weight, x, ymax, ymin
contour	contour	colour, linetype, size, weight, x, y
crossbar	identity	colour, fill, linetype, size, x, y, ymax, ymin
density	density	colour, fill, linetype, size, weight, x, y
density2d	density2d	colour, linetype, size, weight, x, y
errorbar	identity	colour, linetype, size, width, x, ymax, ymin
freqpoly	bin	colour, linetype, size
hex	binhex	colour, fill, size, x, y
histogram	bin	colour, fill, linetype, size, weight, x
hline	hline	colour, linetype, size
jitter	identity	colour, fill, shape, size, x, y
line	identity	colour, linetype, size, x, y

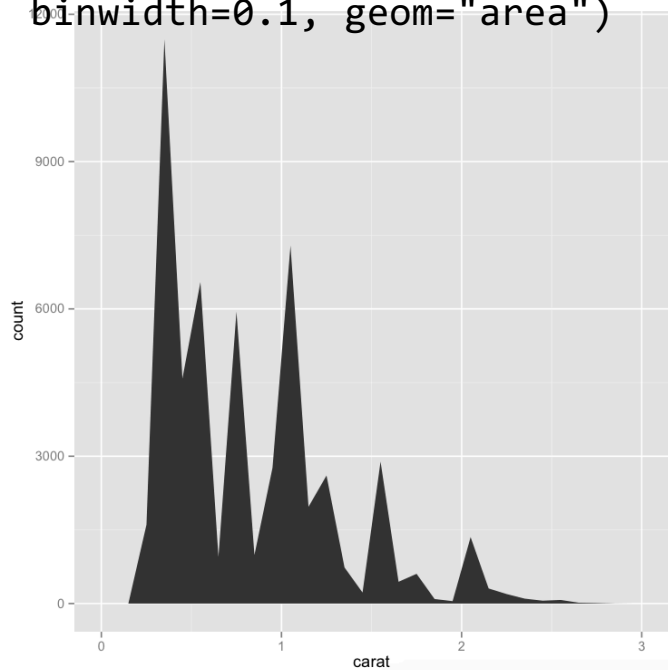
linerrange	identity	colour, linetype, size, x, ymax, ymin
path	identity	colour, linetype, size, x, y
point	identity	colour, fill, shape, size, x, y
pointrange	identity	colour, fill, linetype, shape, size, x, y, ymax, ymin
polygon	identity	colour, fill, linetype, size, x, y
quantile	quantile	colour, linetype, size, weight, x, y
rect	identity	colour, fill, linetype, size, xmax, xmin, ymax, ymin
ribbon	identity	colour, fill, linetype, size, x, ymax, ymin
rug	identity	colour, linetype, size
segment	identity	colour, linetype, size, x, xend, y, yend
smooth	smooth	alpha, colour, fill, linetype, size, weight, x, y
step	identity	colour, linetype, size, x, y
text	identity	angle, colour, hjust, label, size, vjust, x, y
tile	identity	colour, fill, linetype, size, x, y
vline	vline	colour, linetype, size

Construcción por capas: STATS

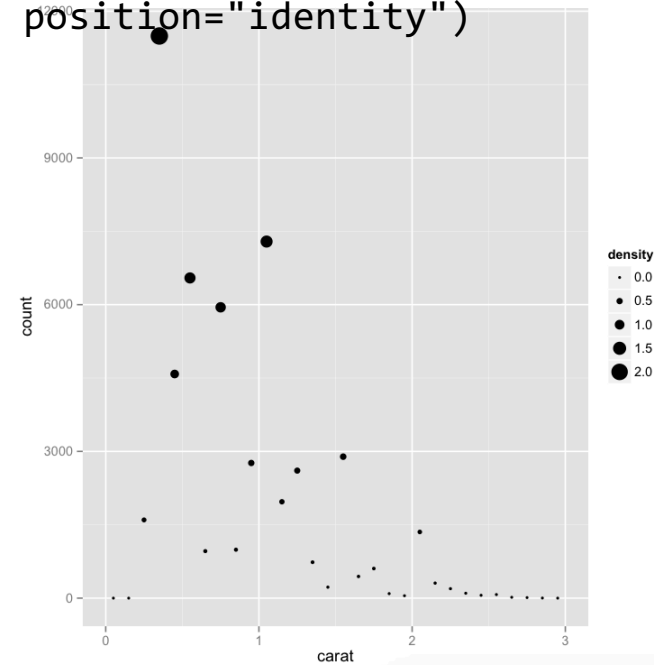
- Mediante la conexión de **geoms** y diferentes **stats** se pueden crear nuevos gráficos:

```
d <- ggplot(diamonds, aes(carat))+xlim(0,3)
```

```
d+stat_bin(aes(ymax=..count..),  
binwidth=0.1, geom="area")
```



```
d+stat_bin(aes(size=..density..),  
binwidth=0.1, geom="point",  
position="identity")
```



Construcción por capas: Ejercicio práctico.

Ej2. ggplot() y la construcción de gráficos por capas

- Dataset: outcome-of-care-measures.csv
 - Información sobre la mortalidad y el número de readmisiones en 30 días por enfermedades relacionadas con los **ataques al corazón, fallos cardíacos y neumonías**.
- Vamos a construir 3 gráficos con múltiple información añadida capa a capa:
 - (HISTOGRAMA) Gráfico de los ratios de mortalidad por ataque cardíaco en periodos de 30 días.
 - (DENSITY) Gráfico comparativo de los ratios de mortalidad en 30 días por ataque al corazón, fallo cardíaco y neumonía.
 - (BOXPLOT) Gráfico del ratio de muertes cada 30 días por estado de EEUU.

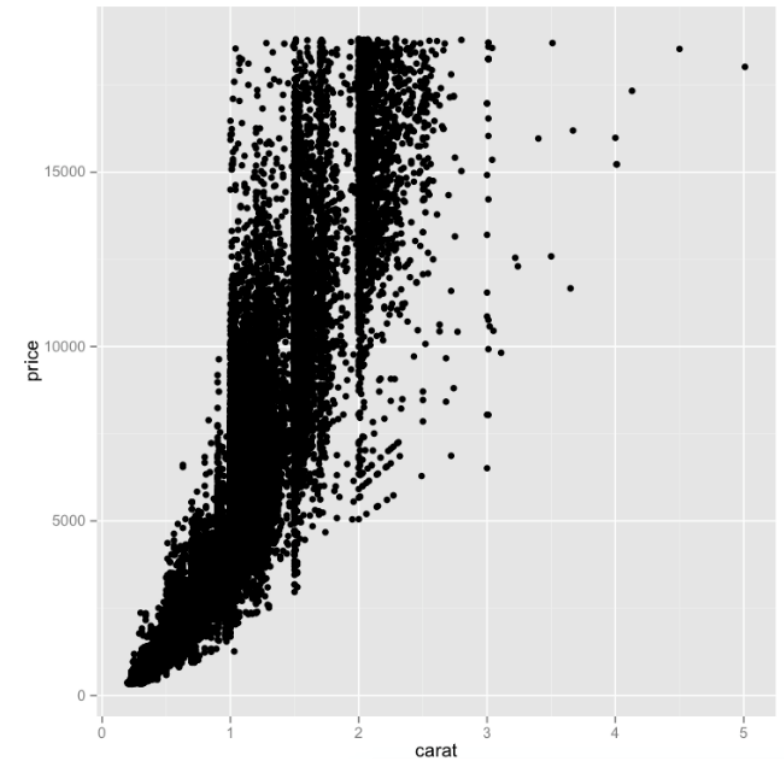
ÍNDICE

3. Detalles de ggplot2.

- Sobredibujado (overplotting).
- Dibujado de mapas.

Detalles de ggplot: Overplotting.

- Los gráficos de dispersión permiten ver la relación entre dos variables continuas.
- **PROBLEMA:** cuando hay gran cantidad de información el **sobredibujado puede ocultar relaciones**.
- Este problema es conocido como “overplotting”:
 1. Hacer los puntos más pequeños.
 2. Utilizar transparencias.
 3. Aplicar “jittering” en caso de distribuciones discretas.



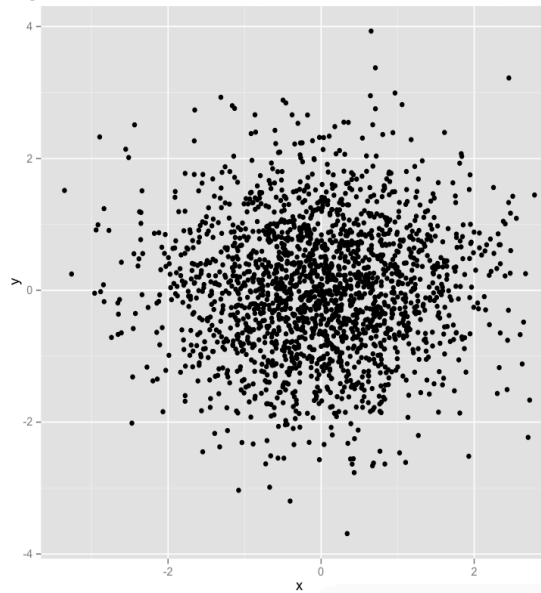
Detalles de ggplot: Overplotting.

- Haciendo los puntos más pequeños:

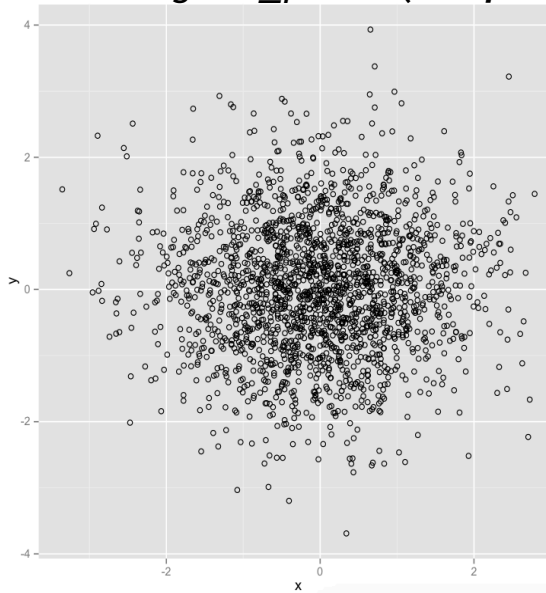
```
df <- data.frame(x=rnorm(2000), y=rnorm(2000))  
norm <- ggplot(df, aes(x,y))
```

□ 0	× 4	⊕ 10	■ 15	■ 22
○ 1	▽ 6	⊗ 11	● 16	● 21
△ 2	⊠ 7	⊞ 12	▲ 17	▲ 24
◇ 5	⋆ 8	⊠ 13	◆ 18	◆ 23
+ 3	◊ 9	◻ 14	● 19	● 20

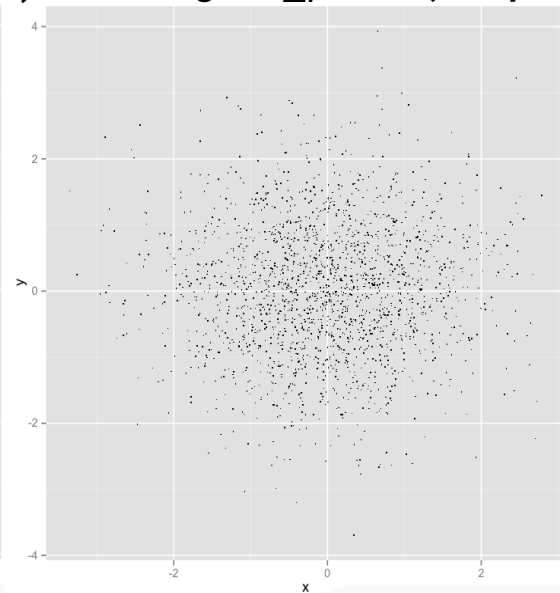
`norm + geom_point(size = 0.5)`



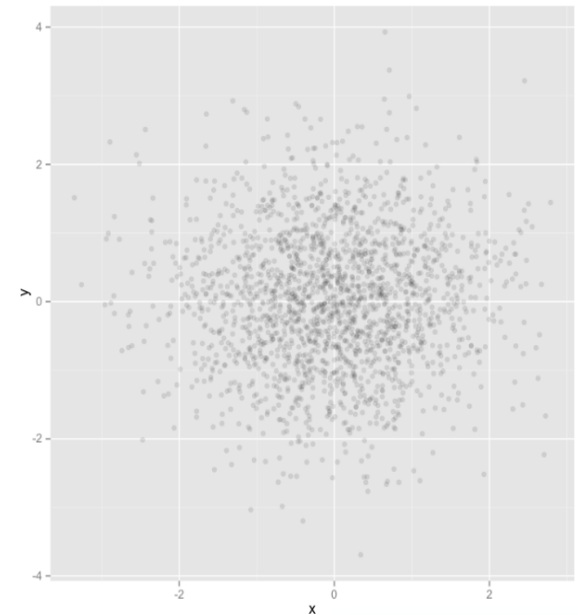
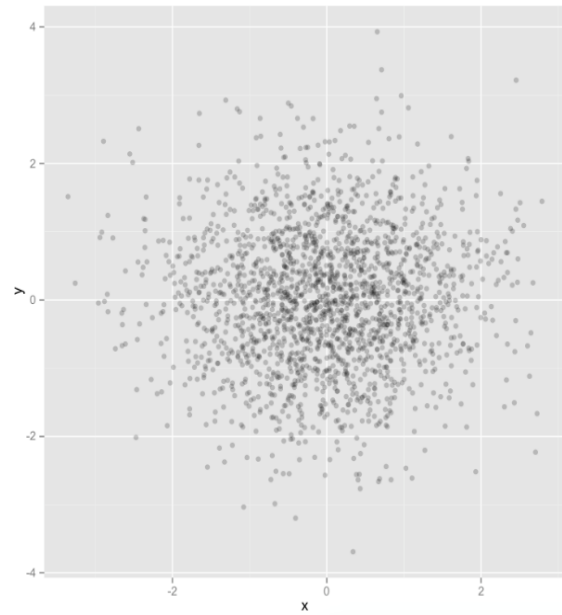
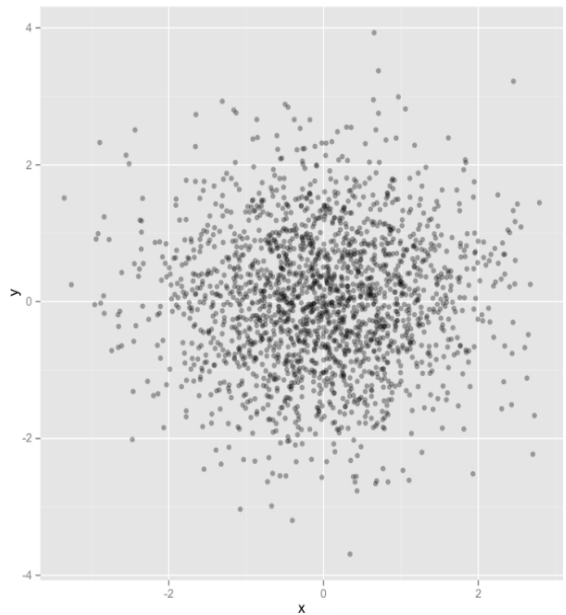
`norm + geom_point(shape=1)`



`norm+geom_point(shape=".")`



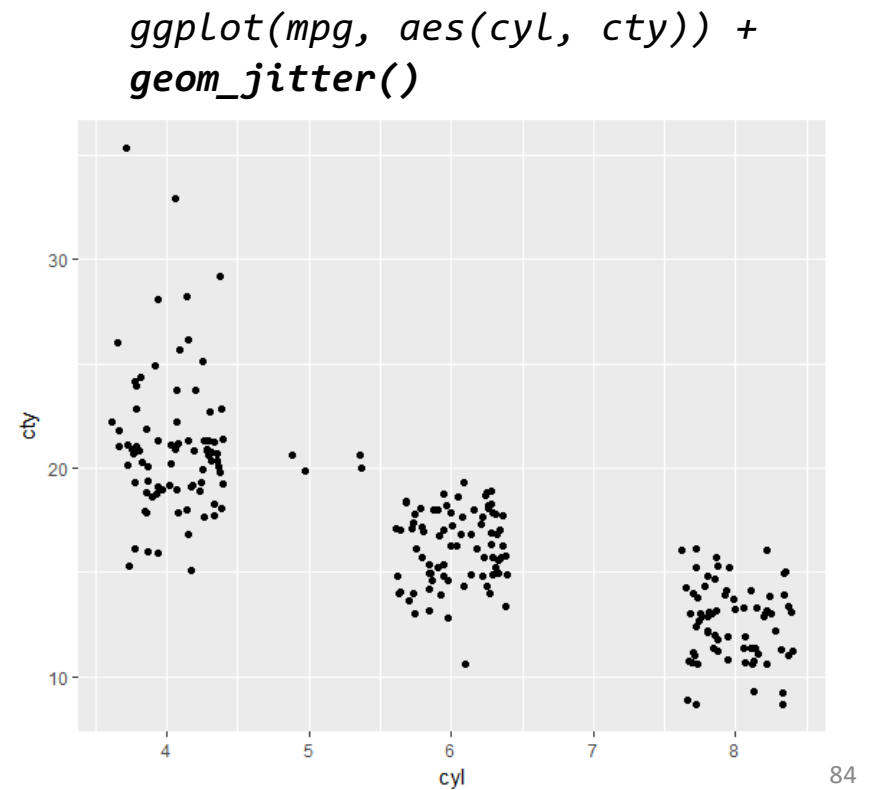
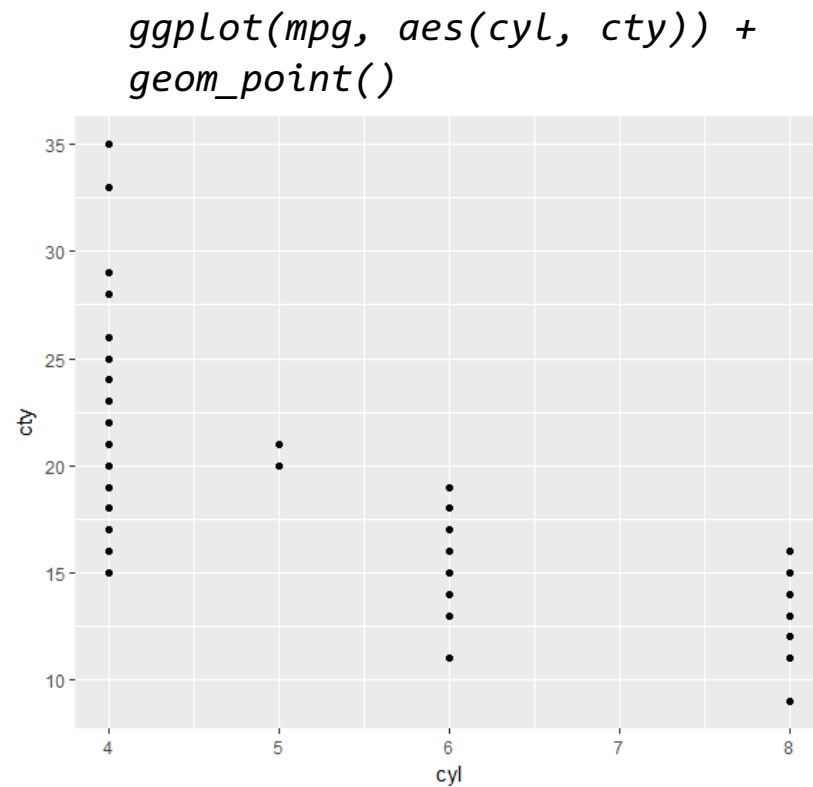
Detalles de ggplot: Overplotting.



```
norm + geom_point(alpha=1/3)  
norm + geom_point(alpha=1/5)  
norm + geom_point(alpha=1/10)
```

Detalles de ggplot: Overplotting.

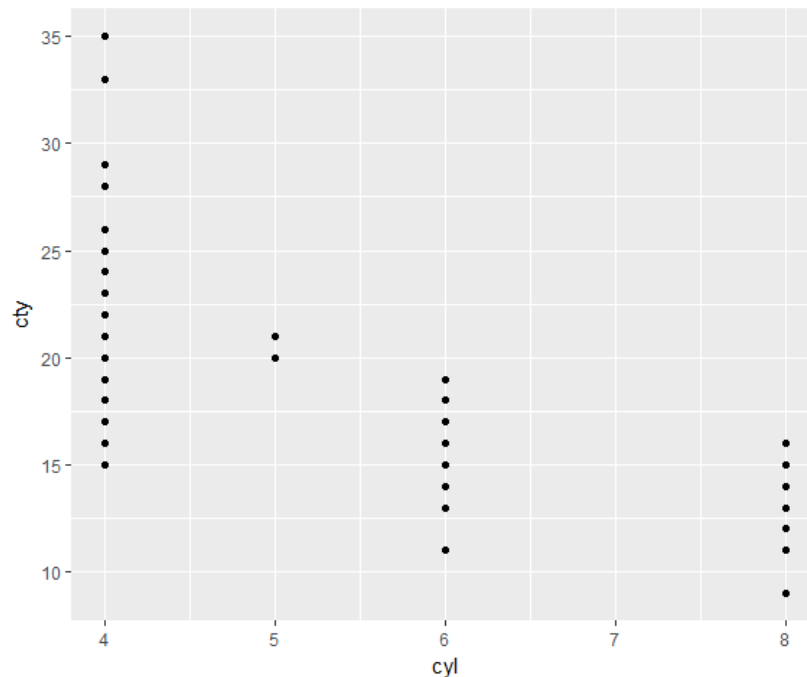
- Aplicar “jittering” en el caso de distribuciones discretas:



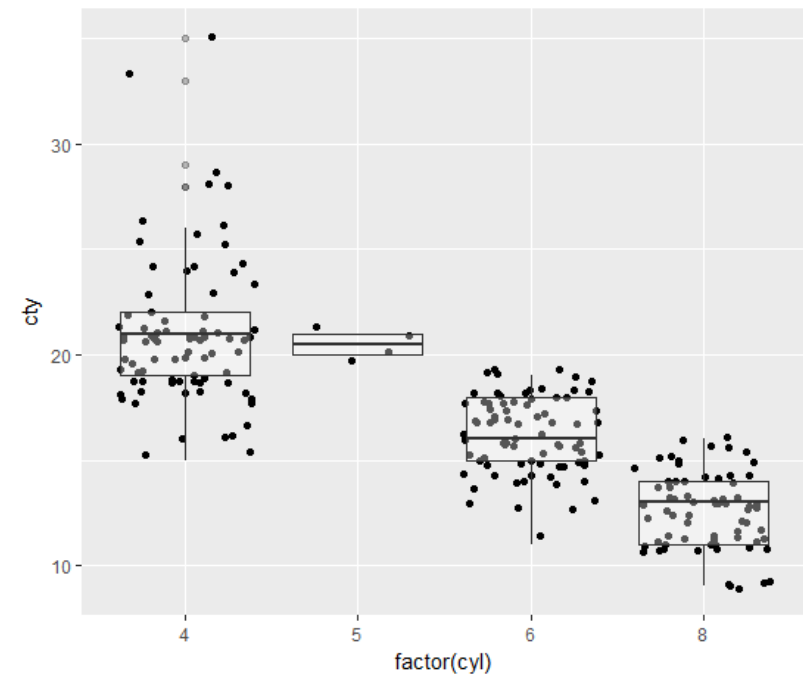
Detalles de ggplot: Overplotting.

- Aplicar “jittering” en el caso de distribuciones discretas:

```
ggplot(mpg, aes(cyl, cty)) +  
  geom_point()
```



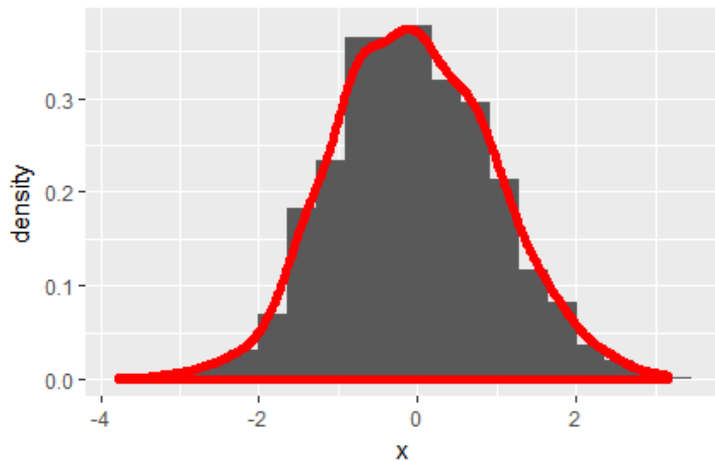
```
ggplot(mpg, aes(cyl, cty)) +  
  geom_jitter() + geom_boxplot(alpha = 1/3)
```



Detalles de ggplot: Overplotting.

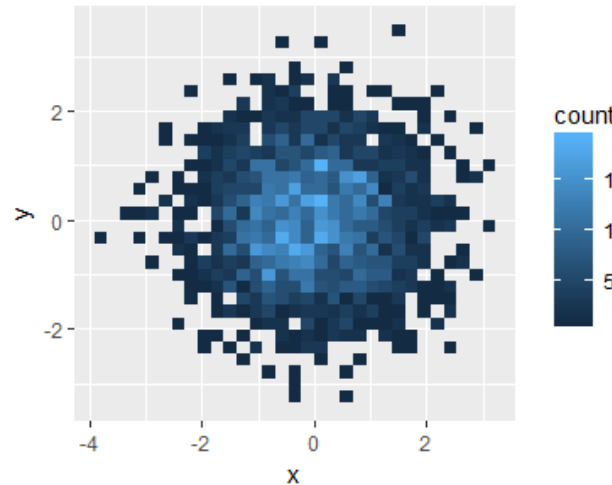
- El overplotting también se puede afrontar como **un problema de estimación de densidad 2D**. Dos variantes:
 - Particionar el espacio y conteo de puntos por partición (*stat_bin*, *stat_bin2d*)
 - Superponer contornos de esta distribución en el gráfico de dispersión (*density*, *density2d*)

1D:

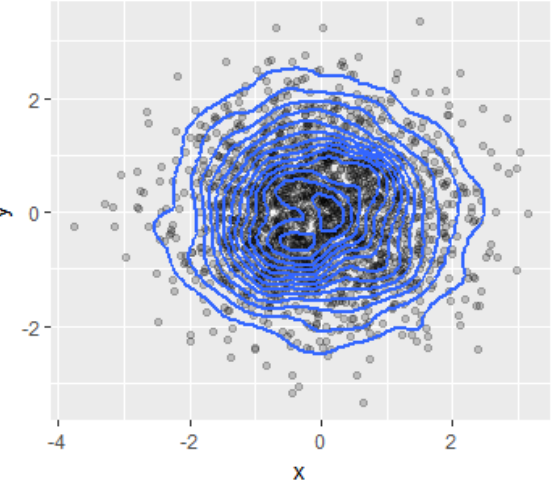


```
ggplot(df, aes(x)) +  
stat_bin(aes(y = ..density..), bins = 20) +  
geom_density(colour = "red", size = 2)
```

2D:



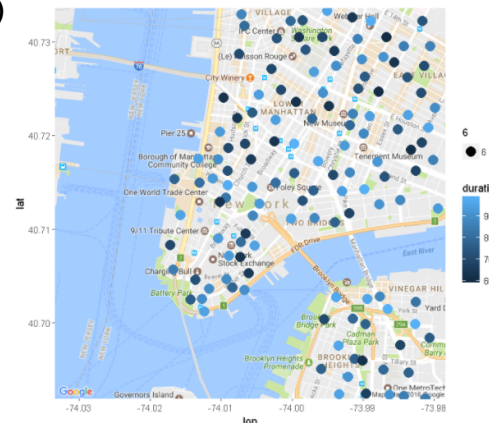
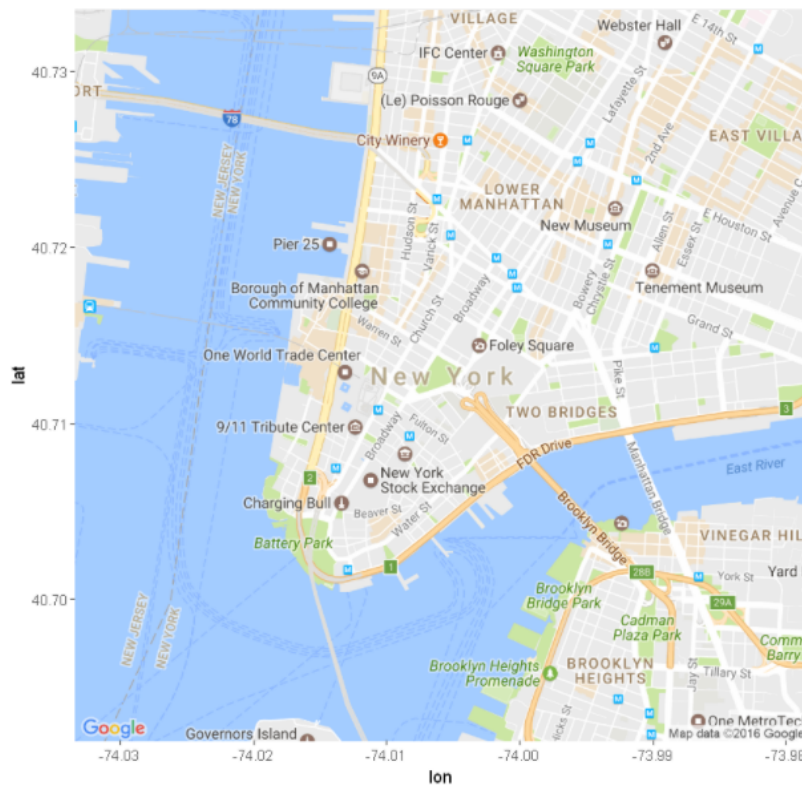
```
ggplot(df, aes(x,y)) + stat_bin2d()
```



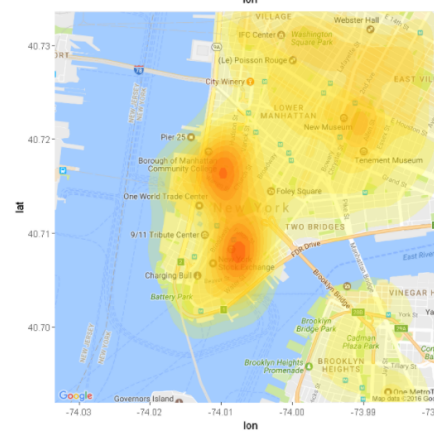
```
ggplot(df, aes(x,y)) +  
geom_point(alpha = 1/5) +  
geom_density2d(size = 1)
```


Detalles de ggplot: Dibujando mapas.

```
library(ggmap)
ggmap(get_map("New York City",
zoom=14, maptype="roadmap", source="google"))
```



```
+ geom_point(aes(x=x,y=y,
color= duration, size = 6),
data=df)
```



```
+ stat_density2d(aes(x = x,
y = y, fill=..level..), data=df,
geom="polygon", alpha=0.2) +
scale_fill_gradient(
low = "yellow", high = "red")
```

Detalles de ggplot: Dibujando mapas.

- ggplot permite trabajar con diferentes formatos de mapas (*geom_polygon*). El más interesante *shp*:
 - Vectores de información geoespacial (polígonos) junto con información no espacial (NO es un dataframe).
 - <http://www.statsilk.com/maps/download-free-shapefile-maps>
- Los ficheros *shp* son tratados como *dataframes*.

Pasos a seguir:

1. Cargar la librería sf:

```
library(sf)  
mapa <- st_read(nombre_fichero)
```



NOTA: la variable *mapa* contiene información espacial (polígonos) e información relacionada con las localizaciones.

Detalles de ggplot: Dibujando mapas.

2. Visualizar el mapa con ggplot:

- shp contienen: geometría de polígonos (coordenadas XY), proyección e información de atributos.
- Para visualizar el mapa:

```
ggplot(mapa) + geom_sf()
```

3. Visualización:

```
ggplot(completo, aes(fill=atributo_de_relleno)) +  
geom_sf()
```

Detalles de ggplot: Dibujando mapas.

```
#librerias requeridas: sf, ggplot2, dplyr
```

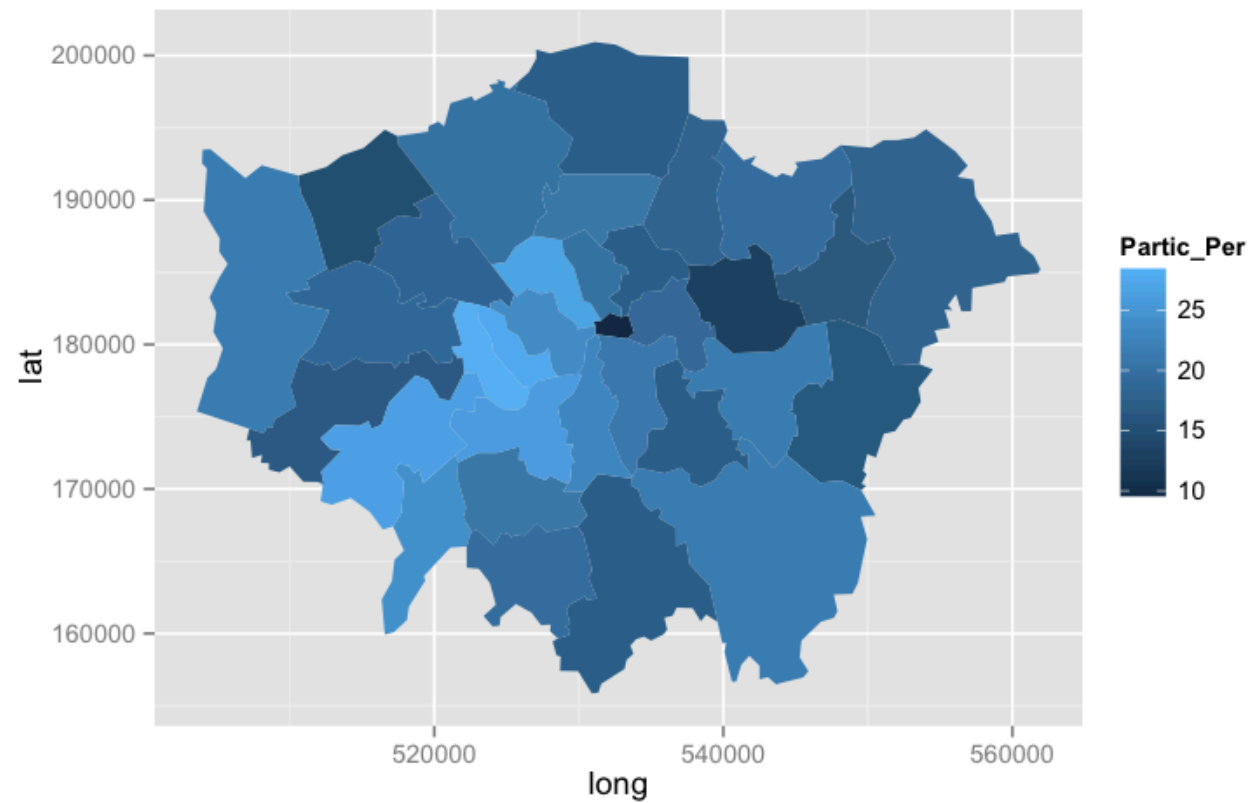
```
londres <- st_read("london_sport.shp")
```

```
#londres.f <- tidy(londres, region="ons_label")
```

```
ggplot(londres, aes(fill = Partic_Per)) + geom_sf()
```

Detalles de ggplot: Dibujando mapas.

- Resultado:



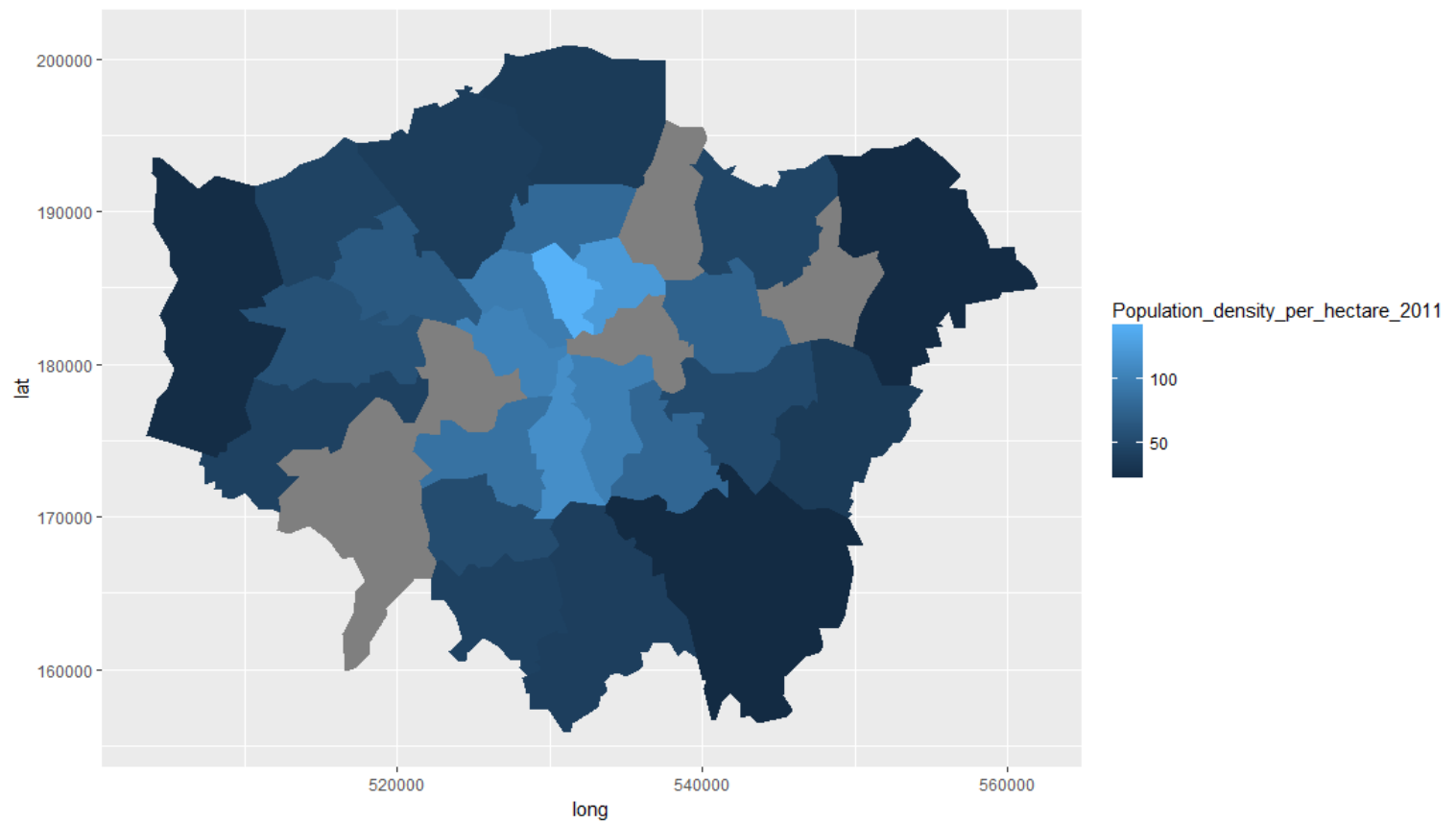
Detalles de ggplot: Dibujando mapas.

- Visualizar información “externa” al mapa.
- Pasos a seguir:
 1. Cargar la información (p.e. de fichero en formato csv):
`datos <- read.csv("nom_fich_con_extensión")`
 2. Concatenar variable mapa y variable con datos externos:
`completo <- XXX_join(mapa, datos)`
 3. Continuar con los pasos 2 y 3 para visualización de mapas.

Detalles de ggplot: Dibujando mapas.

```
library(sf)
library(ggplot2)
library(dplyr)
datos <- read.csv("bor_profiles.csv")
completo <- left_join(londres, datos)
glondres <- ggplot(completo, aes(fill = Population_density_per_hectare_2011)) + geom_sf()
glondres
```

Detalles de ggplot: Dibujando mapas.



Detalles de ggplot: Ejercicio práctico.

Ej3. Análisis de información geográfica.

- Análisis de información en base a su distribución geográfica.
- Ficheros:
 - **ne_110m_admin_0_countries.shp** : shp con información geográfica de todo el mundo.
 - **datosmundo.csv** el cual es un fichero con información sobre la esperanza de vida en diferentes países del mundo.
- En este ejercicio analizaremos comparativamente la esperanza de vida a nivel mundial en el año 1990 y el año 2013.

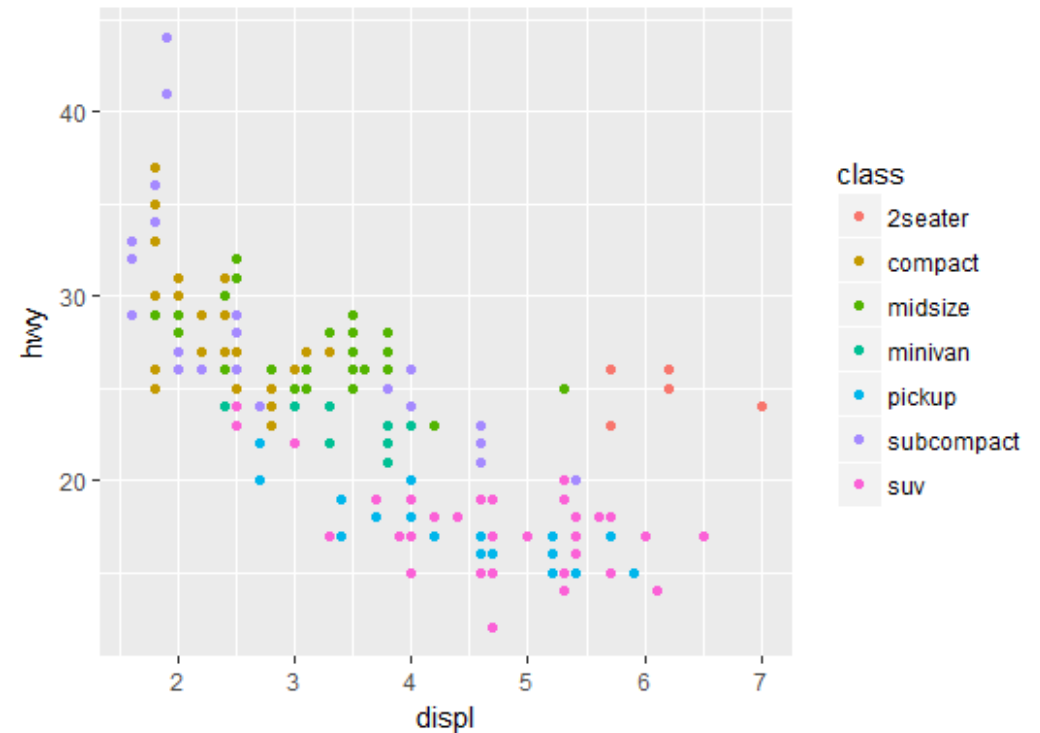
ÍNDICE

4. Posicionamiento.

- Faceteado.
- Sistemas de coordenadas.

Posicionamiento: Faceteado.

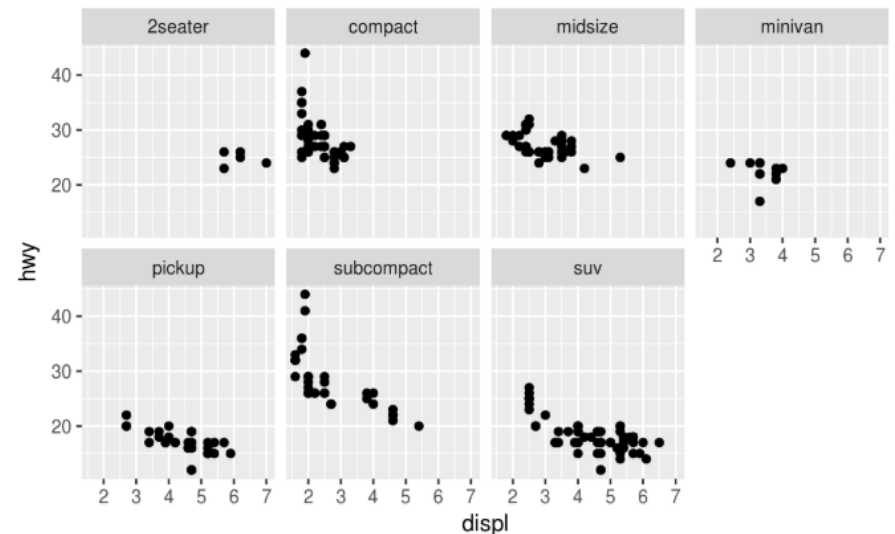
- Una forma de agregar variables adicionales en un gráfico es por medio de **estéticas** (**size**, **colour**, **shape**...).
- Otra forma, particularmente útil para variables categóricas, es dividir el gráfico en **facets**.
 - Subparcelas que muestran un subconjunto de los datos.



Posicionamiento: Faceteado.

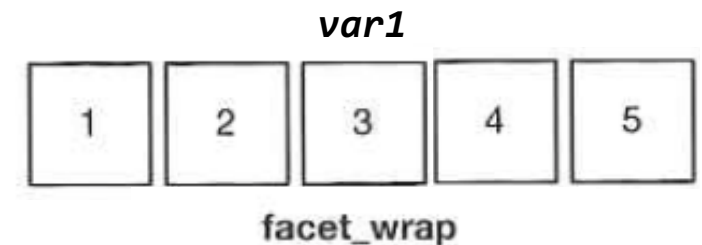
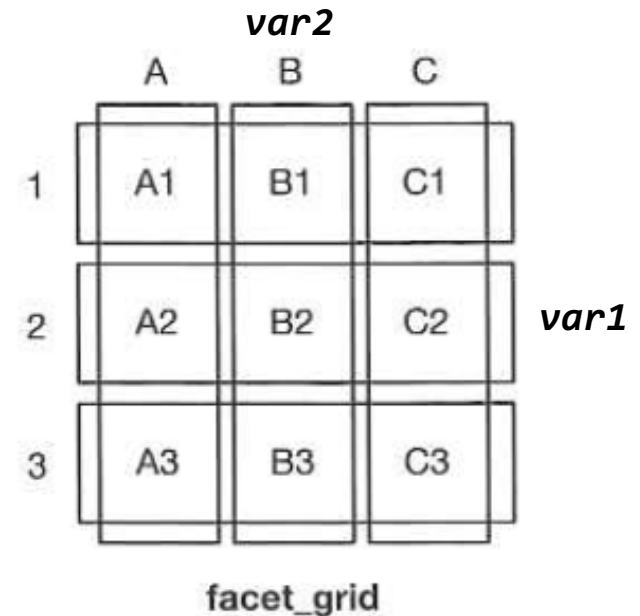
- Una forma de agregar variables adicionales en un gráfico es por medio de **estéticas** (*size*, *colour*, *shape*...).
- Otra forma, particularmente útil para variables categóricas, es dividir el gráfico en **facets**.
 - Subparcelas que muestran un subconjunto de los datos.

```
ggplot(data = mpg) +  
  geom_point(mapping = aes(x = displ, y = hwy)) +  
  facet_wrap(~ class, nrow = 2)
```



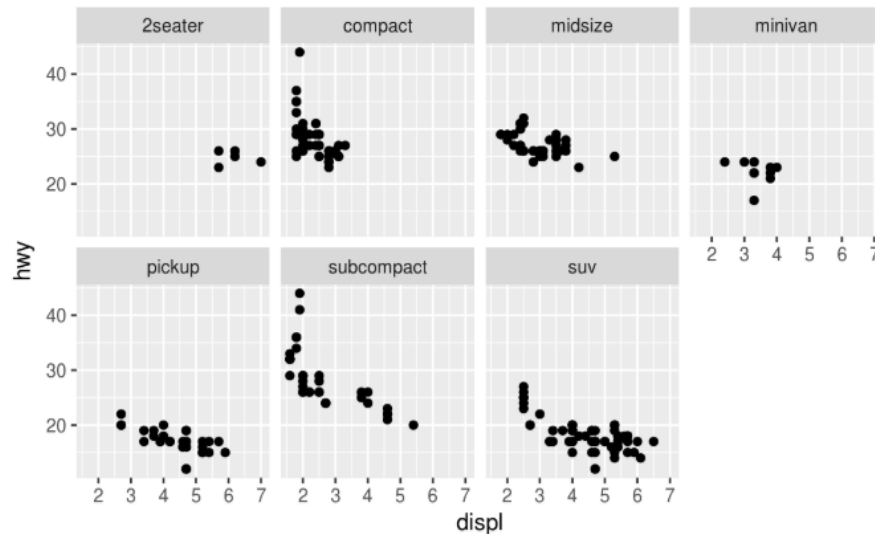
Posicionamiento: Faceteado.

- El faceteado produce pequeños gráficos correspondientes a diferentes subconjuntos de datos.
- Es ideal para análisis exploratorio:
 - Como varían **x**, **y** con respecto a **z** (discreta)
- Hay dos tipos:
 - [Grid] ... + *facet_grid* (*var1* ~ *var2*)
 - [Rectángulo] ... + *facet_wrap*(~ *var1*)
- Atributos comunes:
 - **facets**: establece los subconjuntos.
 - **scales**: fijas o libres.

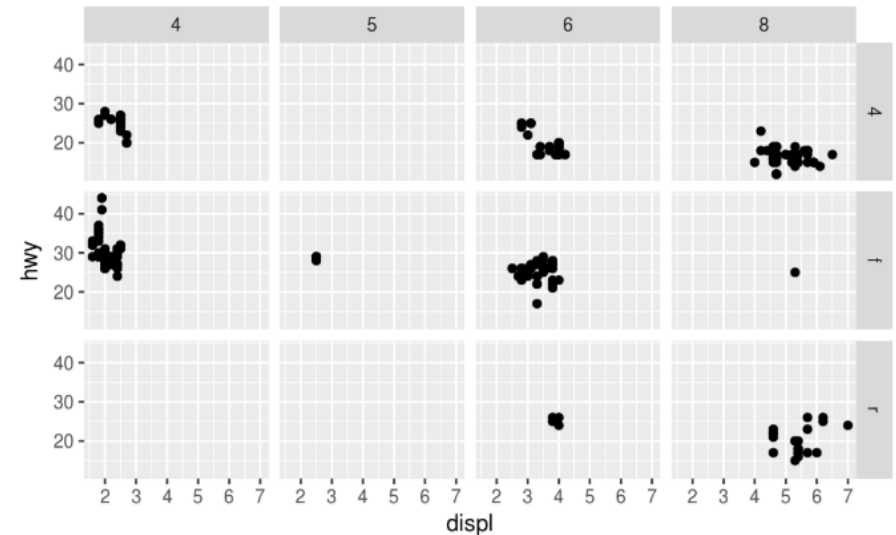


Posicionamiento: Faceteado.

```
ggplot(data = mpg) +  
  geom_point(mapping = aes(x = displ, y = hwy)) +  
  facet_wrap(~ class, nrow = 2)
```



```
ggplot(data = mpg) +  
  geom_point(mapping = aes(x = displ, y = hwy)) +  
  facet_grid(drv ~ cyl)
```

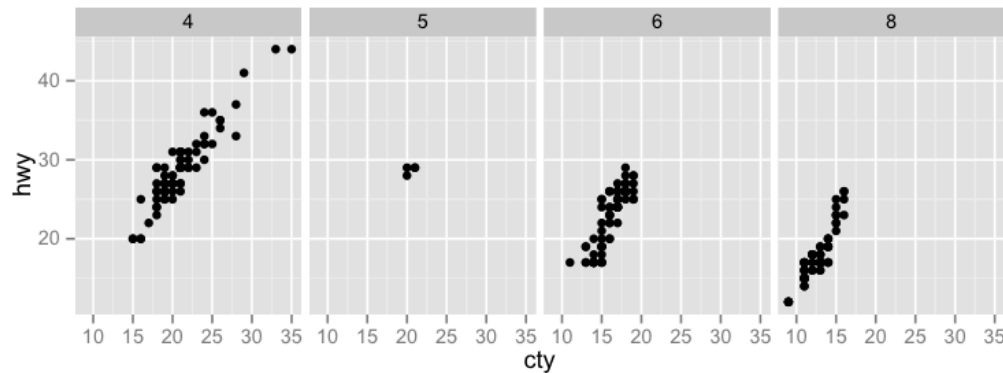


Genera “secuencias” de gráficos. Es útil cuando se quiere analizar una **variable con múltiples niveles**

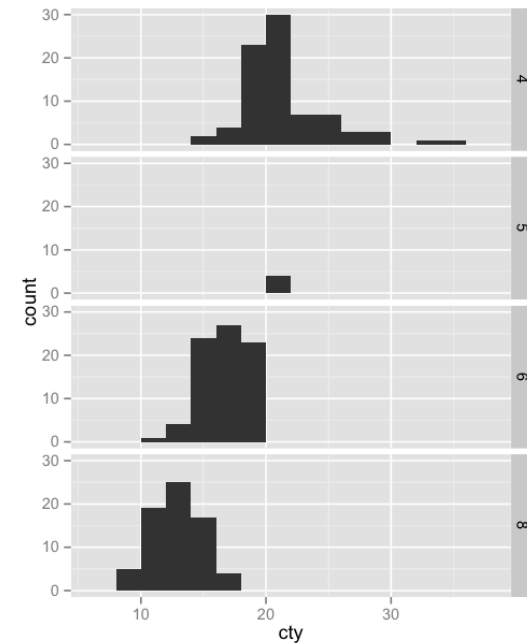
Posicionamiento: Faceteado.

- `facet_grid(facets, margins=..., scales=...)`

- “`. ~ a`” en una fila (`.~cyl`)



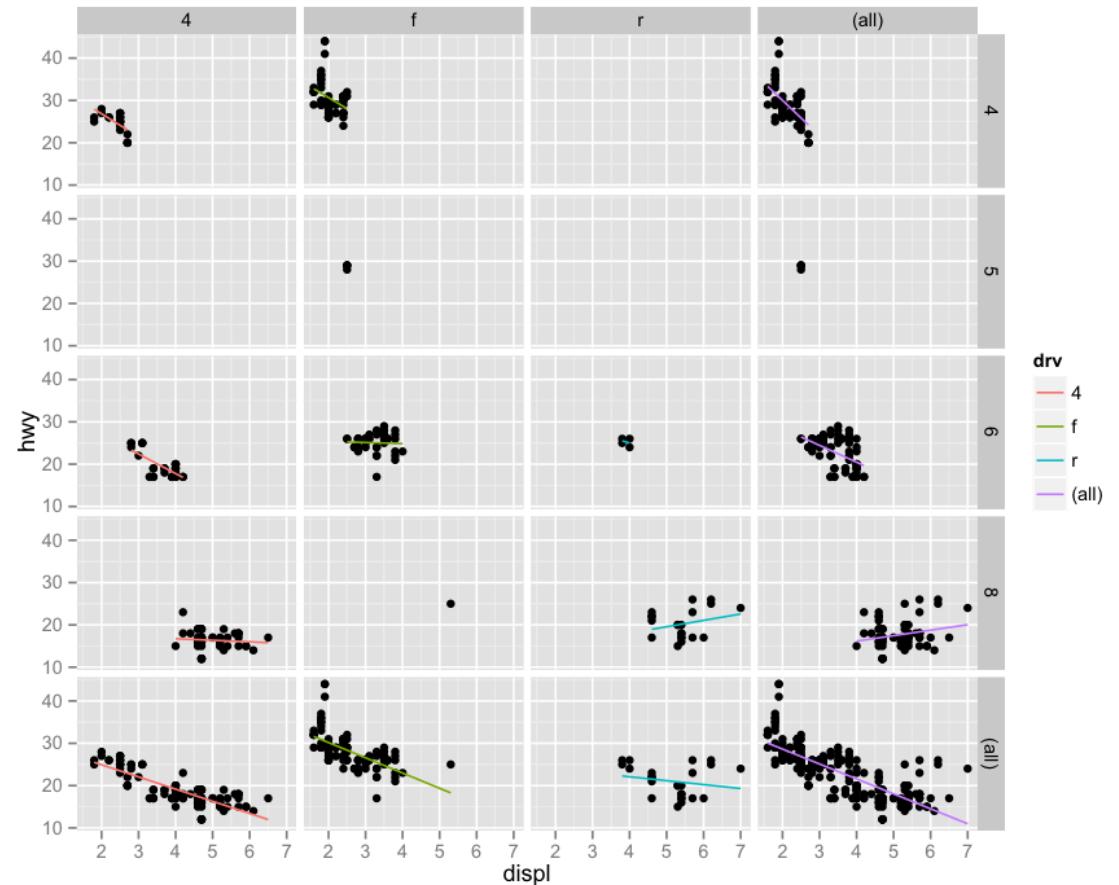
- “`b ~ .`” en una columna (`cyl~.`)



Posicionamiento: Faceteado.

- En *margins* se especifica si se deben mostrar las **distribuciones marginales** o no (valor por defecto).

```
p <- ggplot(aes(displ, hwy),  
data=mpg) + geom_point()  
  
p + geom_smooth(aes(colour=drv),  
method="lm", se=F) +  
facet_grid(cyl~drv, margins=T)
```

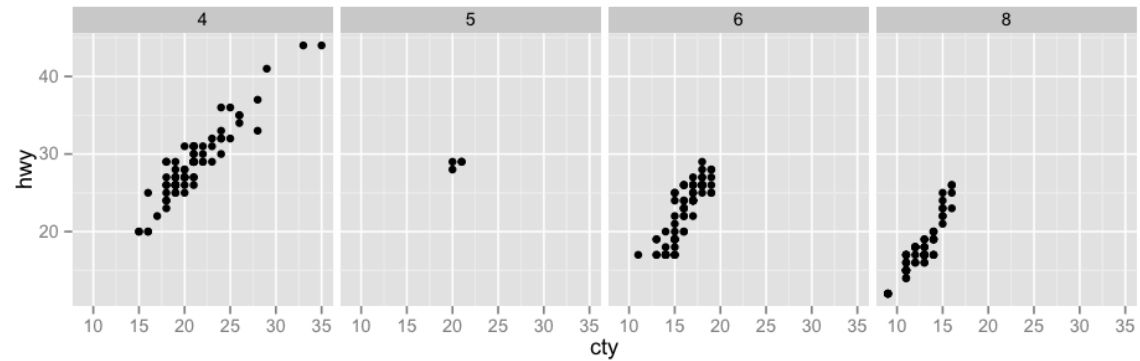


Posicionamiento: Faceteado.

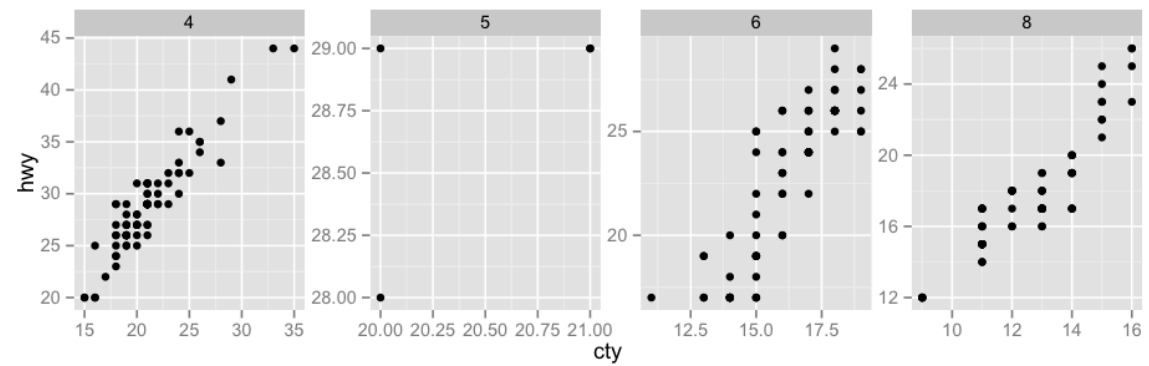
- Las escalas se controlan mediante el parámetro ***scales***:
 - ***scales="fixed"*** : las escalas en ***x*** e ***y*** son comunes en todas las gráficas.
 - ***scales="free"*** : las escalas en ***x*** e ***y*** varían en cada panel.
 - ***scales="free_x"*** : la escala en ***x*** es libre, en ***y*** es fija.
 - ***scales="free_y"*** : la escala en ***x*** es fija, en ***y*** es libre.
- Las escalas **fijas** nos permiten comparar conjuntos de **manera absoluta**.
- Las escalas **libres** nos permiten comparar conjuntos de **manera relativa**.

Posicionamiento: Faceteado.

- *scales="fixed"*



- *scales="free"*

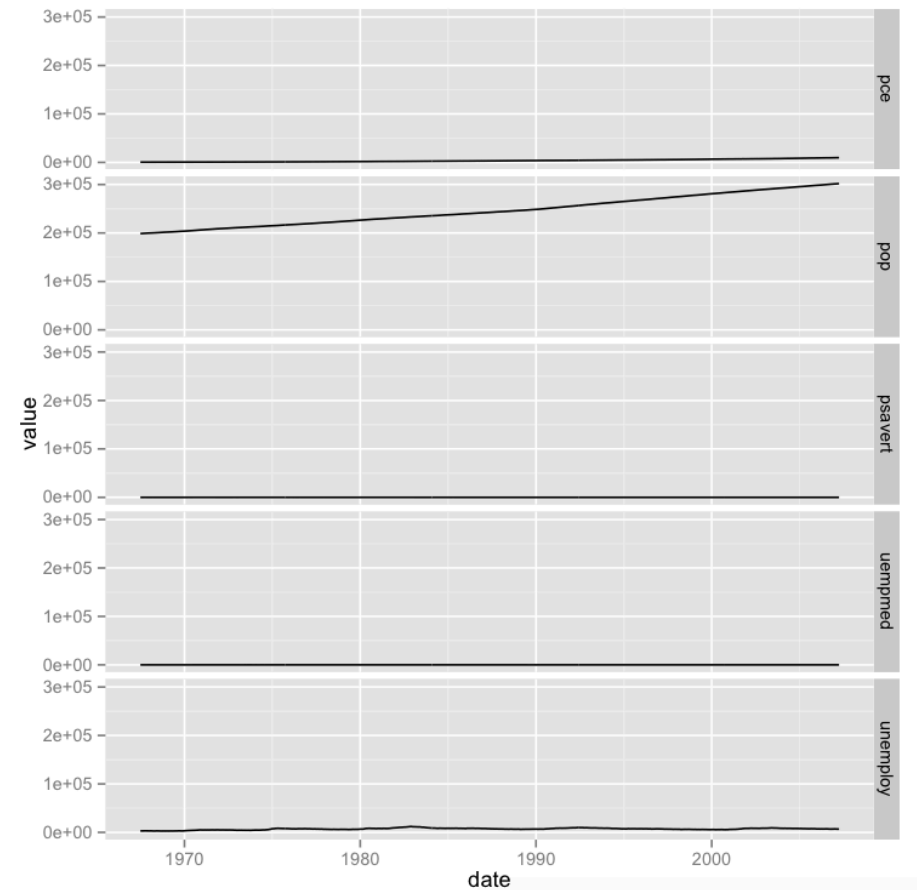


Posicionamiento: Faceteado.

- Las escalas libres son útiles cuando queremos mostrar **múltiples series temporales** que han sido medidas utilizando **diferentes escalas**.

```
library(reshape2)
head(economics)
em<-melt(economics, id="date")

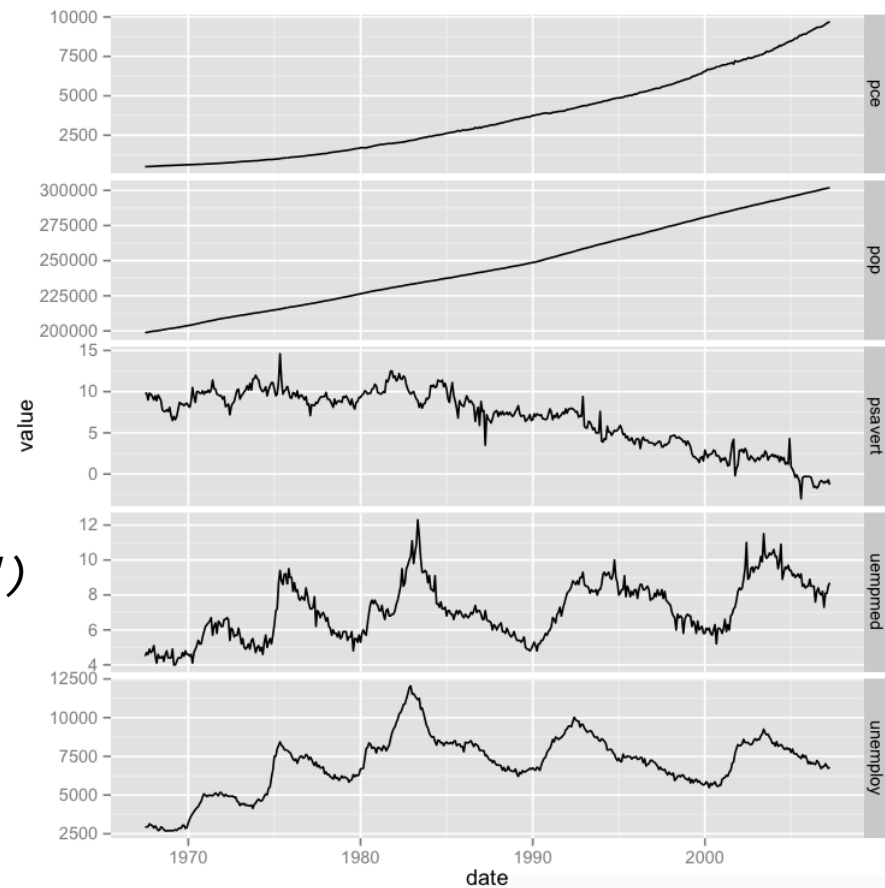
ggplot(em, aes(date, value)) +
  geom_line() +
  facet_grid(variable ~., scales="fixed")
```



Posicionamiento: Faceteado.

- `scales="free_y"`

```
ggplot(em, aes(date, value)) +  
  geom_line() +  
  facet_grid(variable ~., scales="free_y")
```



Posicionamiento: Faceteado.

- Se puede realizar faceteado con variables **continuas** previa **conversión a categóricas** (dicretización):
 - Divide la información en **n contenedores del mismo tamaño**
 - *cut_interval(x, n=...)*.
 - Divide la información en **contenedores del tamaño fijado**
 - *cut_interval(x, Length=...)*.
 - Divide la información en **n contenedores**, cada uno de los cuales tiene el **mismo número de puntos aproximadamente**
 - *cut_number(x, n=...)*.
- NOTA: el faceteado con variable continua únicamente trabaja con variables en el “dataset” (no con funciones). Hay que crear nuevas variables.

Posicionamiento: Faceteado.

```
# n contenedores del mismo tamaño  
(intervalo)  
mpg$disp_wn <- cut_interval(mpg$displ, n=6)
```

```
# en contenedores del tamaño fijado  
mpg$disp_ww <- cut_interval(mpg$displ,  
length=1)
```

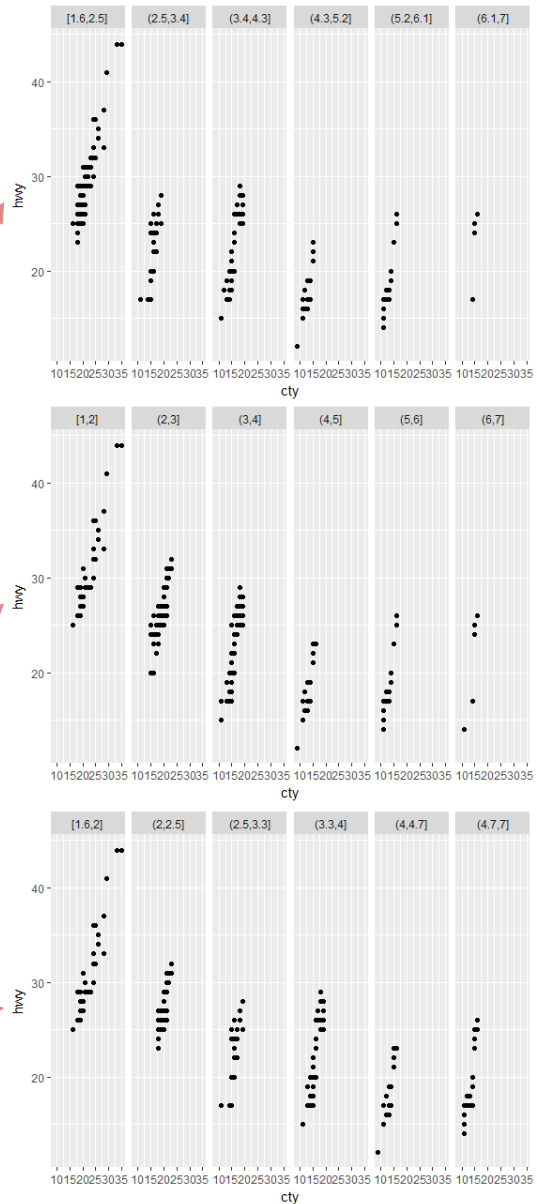
```
# n contenedores, cada uno de los cuales  
tiene el mismo número de puntos aprox.  
mpg$disp_nn <- cut_number(mpg$displ, n=6)
```

```
plot <- ggplot(mpg, aes(cty, hwy)) +  
geom_point()
```

```
plot + facet_wrap(~disp_wn, nrow=1)
```

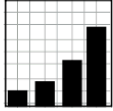
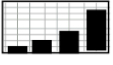
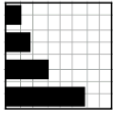
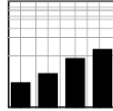
```
plot + facet_wrap(~disp_ww, nrow=1)
```

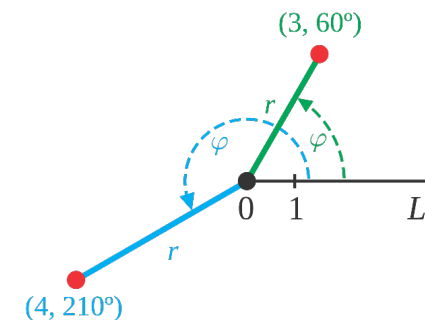
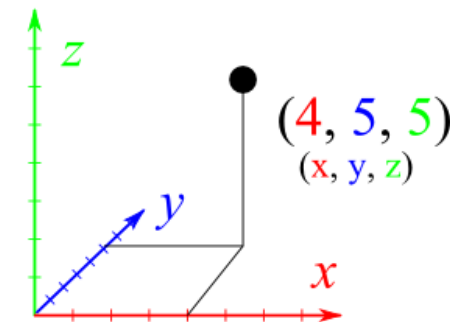
```
plot + facet_wrap(~disp_nn, nrow=1)
```



Posicionamiento: Sistema de coordenadas.

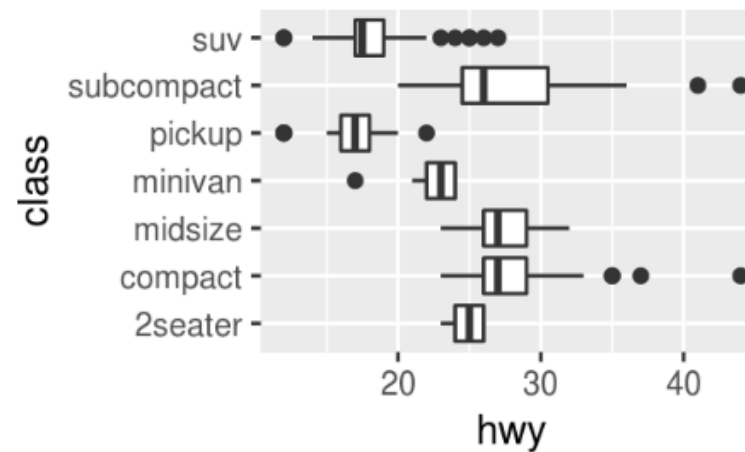
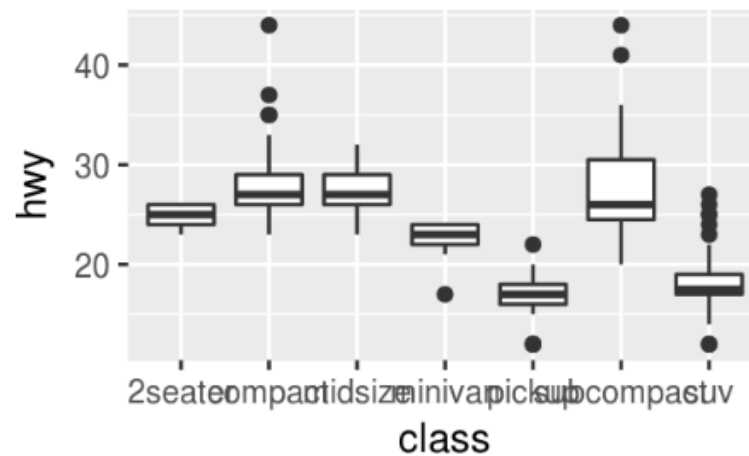
- **ggplot2** posee distintos sistemas de coordenadas que afectan a los EJES una vez se han calculado las estadísticas.

	<pre>r <- b + geom_bar()</pre>
	<pre>r + coord_cartesian(xlim = c(0, 5))</pre> xlim, ylim The default cartesian coordinate system
	<pre>r + coord_fixed(ratio = 1/2)</pre> ratio, xlim, ylim Cartesian coordinates with fixed aspect ratio between x and y units
	<pre>r + coord_flip()</pre> xlim, ylim Flipped Cartesian coordinates
	<pre>r + coord_polar(theta = "x", direction=1)</pre> theta, start, direction Polar coordinates
	<pre>r + coord_trans(ytrans = "sqrt")</pre> xtrans, ytrans, limx, limy Transformed cartesian coordinates. Set extras and strains to the name of a window function.



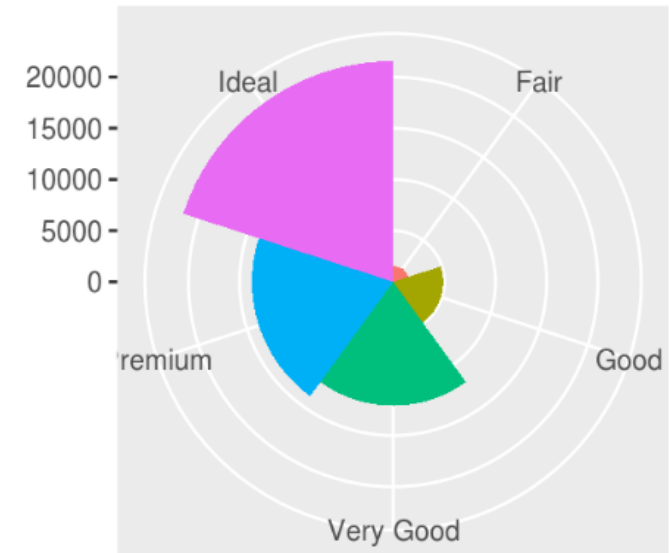
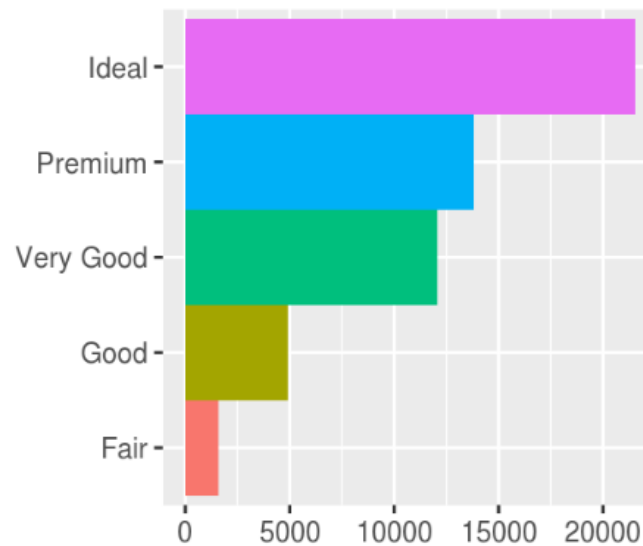
Posicionamiento: Sistema de coordenadas.

```
ggplot(data = mpg, mapping = aes(x = class, y = hwy)) +  
  geom_boxplot()  
  
ggplot(data = mpg, mapping = aes(x = class, y = hwy)) +  
  geom_boxplot() +  
  coord_flip()
```



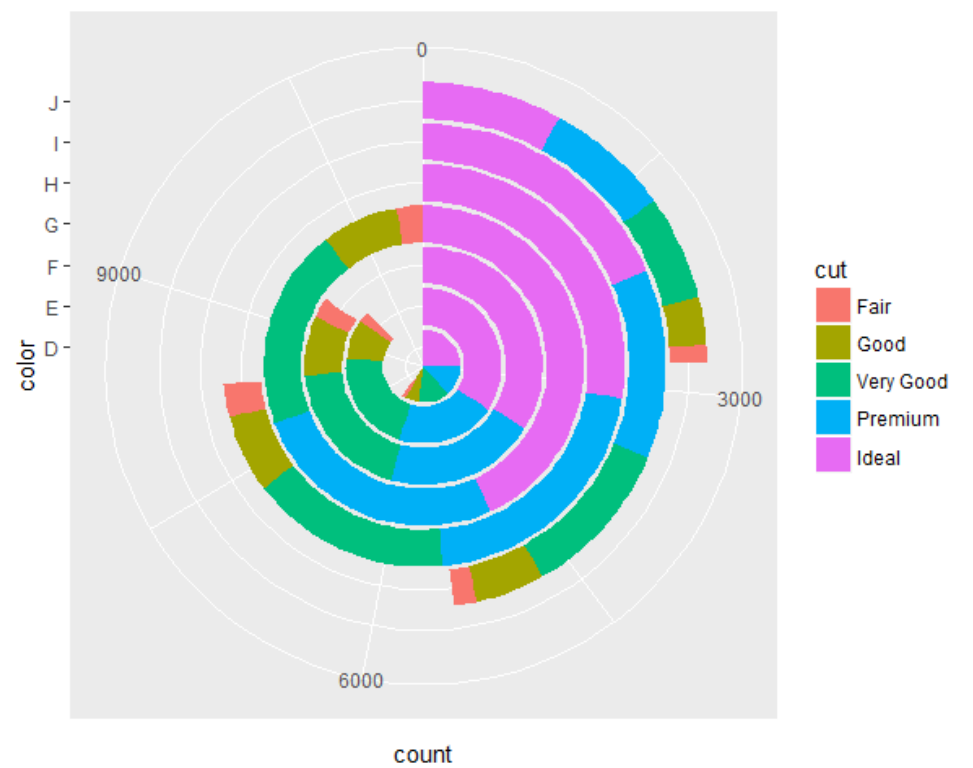
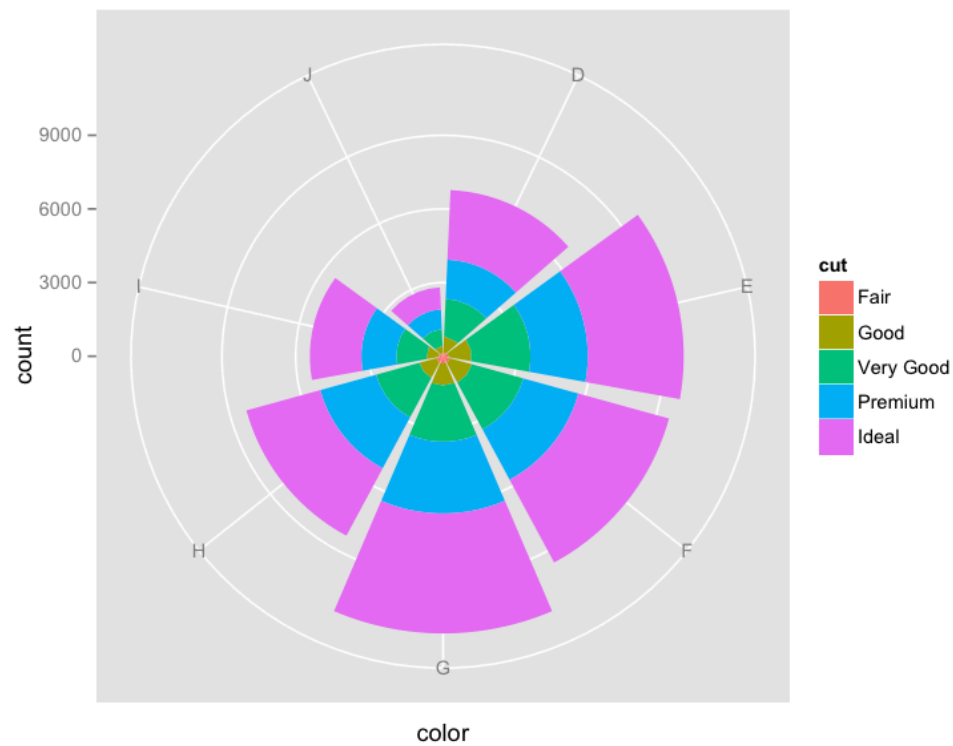
Posicionamiento: Sistema de coordenadas.

```
bar <- ggplot(data = diamonds) +  
  geom_bar(  
    mapping = aes(x = cut, fill = cut),  
    show.legend = FALSE,  
    width = 1  
  ) +  
  theme(aspect.ratio = 1) +  
  labs(x = NULL, y = NULL)  
  
bar + coord_flip()  
bar + coord_polar()
```



Posicionamiento: Sistema de coordenadas.

TEST AULA 4



Posicionamiento: Ejercicio.

Ej4. Posicionamiento: Faceteado.

- Ficheros:
 - **outcome-of-care-measures.csv** que contiene información sobre la mortalidad y el número de readmisiones en 30 días por enfermedades relacionadas con los ataques al corazón, fallos cardíacos y neumonías.
 - **hospital-data.csv** que contiene información sobre cada hospital de los 400 de los que dispone la base de datos.
- Vamos a construir 2 gráficos con faceteado en base a un factor:
 - faceteado tipo rejilla (grid)
 - faceteado tipo cinta (wrap).

Índice

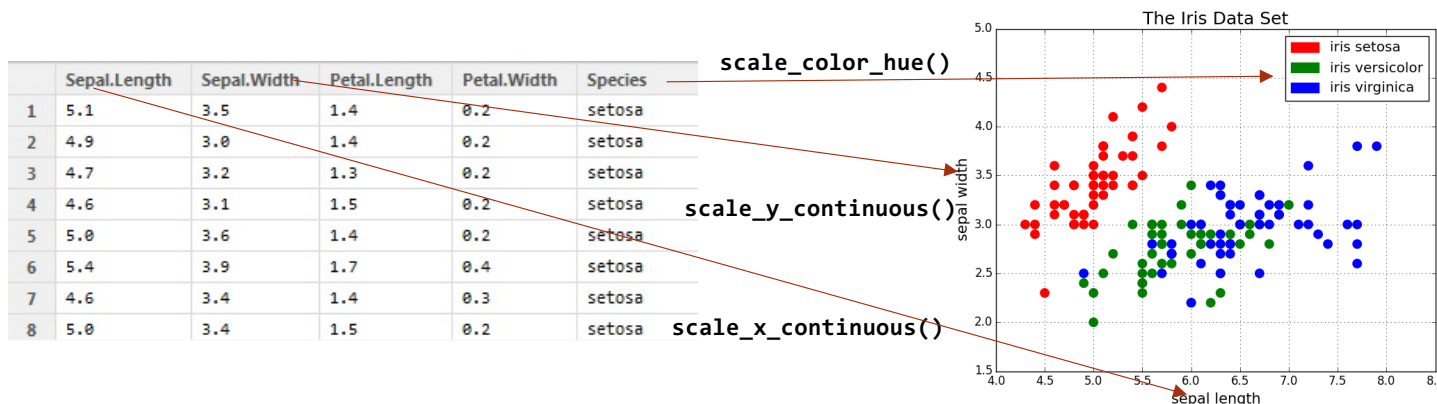
5. Escalas

- Introducción.
- Escalas de posición.
- Escalas de color.

Escalas: Introducción

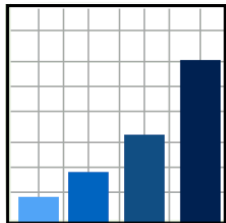
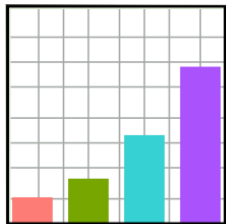
- Las **escalas** controlan el mapeado de **datos** a **estéticas**.

```
ggplot(iris, aes(X = Sepal.Length, y = Sepal.Width, color = Species)) + geom_point()
```



- Transforman los datos en algo visible: **tamaño, color, posición o forma**.
- El proceso de escalado atraviesa tres fases:
 - Transformación**: transforma los datos antes del mapeado (logaritmo, cuadrado, raíz cuadrada, ...).
 - Entrenamiento**: se aprende el dominio de escala (mín, máx,...)
 - Mapeado**: fase final de transformación de los datos en estéticas.

Escalas: Introducción



```
n <- b + geom_bar(aes(fill = fl))
```

n

scale_

aesthetic
to adjust

prepackaged
scale to use

scale specific
arguments

```
n + scale_fill_manual(
```

```
  values = c("skyblue", "royalblue", "blue", "navy"),  
  limits = c("d", "e", "p", "r"), breaks = c("d", "e", "p", "r"),  
  name = "fuel", labels = c("D", "E", "P", "R"))
```

range of values to
include in mapping

title to use in
legend/axis

labels to use in
legend/axis

breaks to use in
legend/axis

Fija el dominio de la escala.
Cualquier valor fuera de escala es
descartado (OJO: no es zoom).

Permite asignar
etiquetas a la leyenda

Controlan que elementos aparecerán en la
leyenda y en los ejes (**breaks**) y qué
etiquetas tendrán (**Labels**).

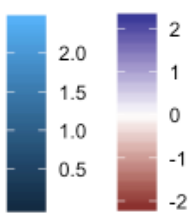
Escalas: de posición.

- Hay dos escalas de posición:
 - Para el eje horizontal (`scale_x_continuous()`, `scale_x_discrete()`)
 - Para el eje vertical (`scale_y_continuous()`, `scale_y_discrete()`)
- Su objetivo fundamental es **cambiar los límites de los ejes**.
 - E.g., `scale_x_continuous(limits = c(10, 20))`
- Shortcuts:
 - `... + xlim(10, 20)`
 - `xlim(20, 10)`
 - `xlim("a", "b", "c")`
 - `xlim(as.Date(c("2008-05-01", "2008-08-01")))`
- No es un zoom sobre los datos de la gráfica.

Escalas: de color.

- Hay bastantes formas de mapear valores en colores:

- Tres métodos basados en gradientes para valores **continuos**.



- `scale_colour_gradient(low, high)`, `scale_fill_gradient(low, high)`
- `scale_colour_gradient2(low, med, high)`, `scale_fill_gradient2(low, med, high)`
- `scale_colour_gradientn(colours)`, `scale_fill_gradientn(colours)`

- Dos métodos para el mapeo de valores **discretos**.

- `scale_colour_brewer()` y `scale_fill_brewer()` :

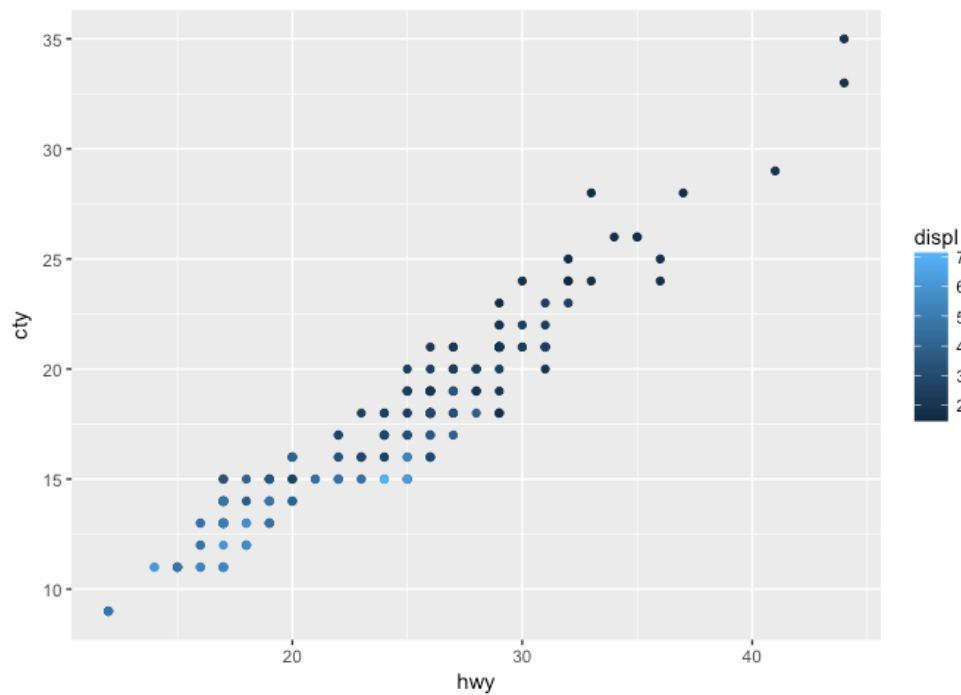
- La paleta de colores se elige de forma manual (parámetro `palette`).
- Posibles valores a asignar a `palette`: “Set1”, “Dark2”, “Set2”, “Pastel1”, “Pastel2”, “Accent”.

- `scale_colour_manual()` :

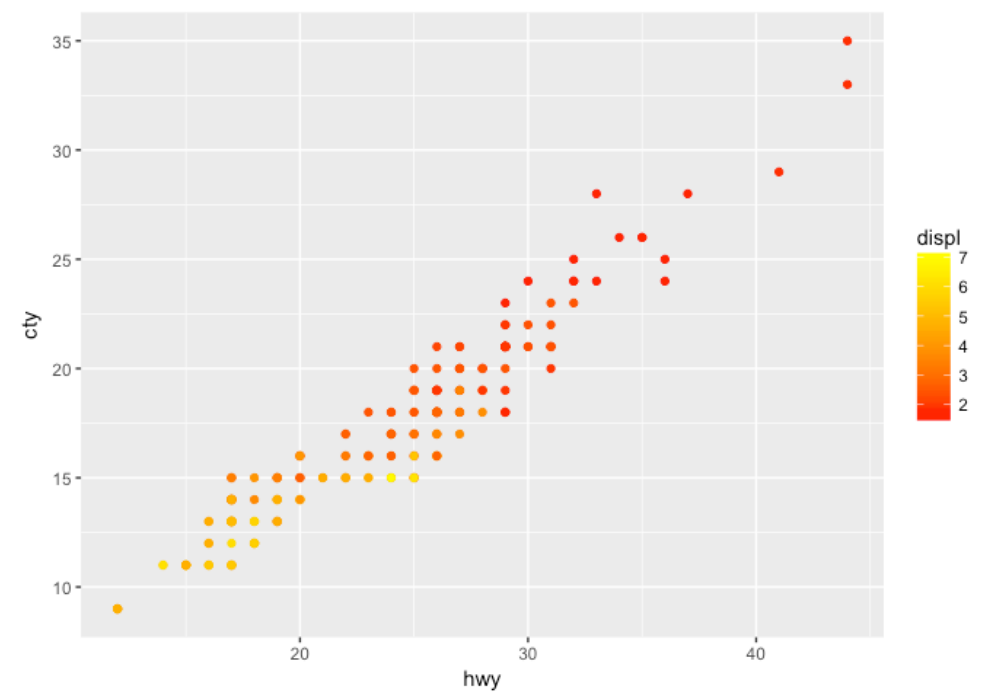
- Permite mapear los colores de forma manual (generar la leyenda).

Escalas: Color (Variables continuas)

```
a <- ggplot(mpg, aes(x=hwy,y=cty,col=displ)) +  
  geom_point(); a
```

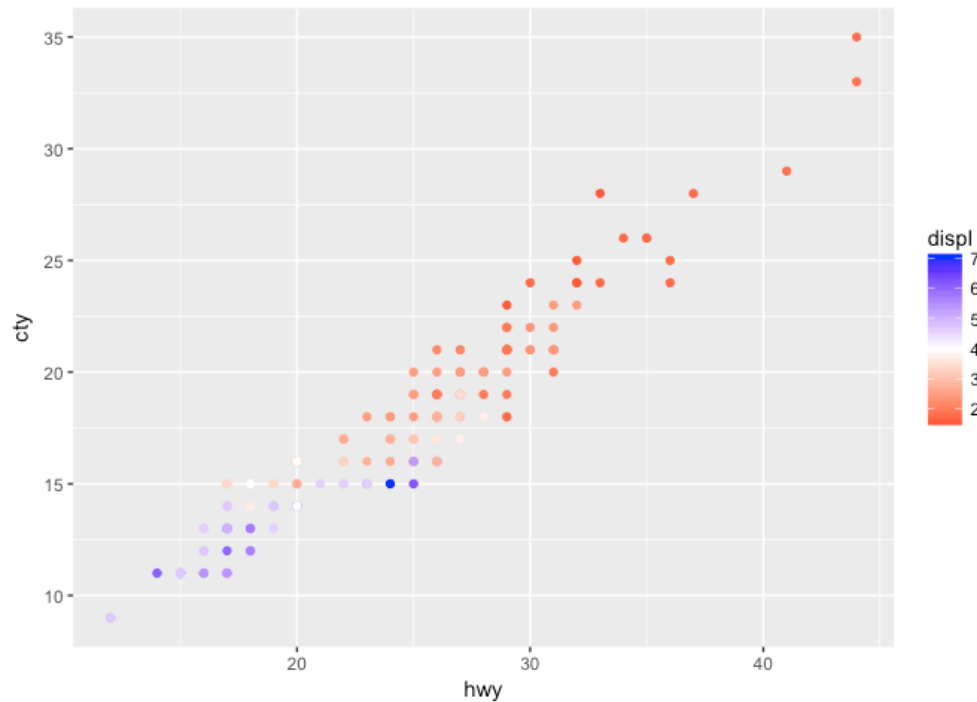


```
a + scale_color_gradient( low = "red",  
  high = "yellow")
```

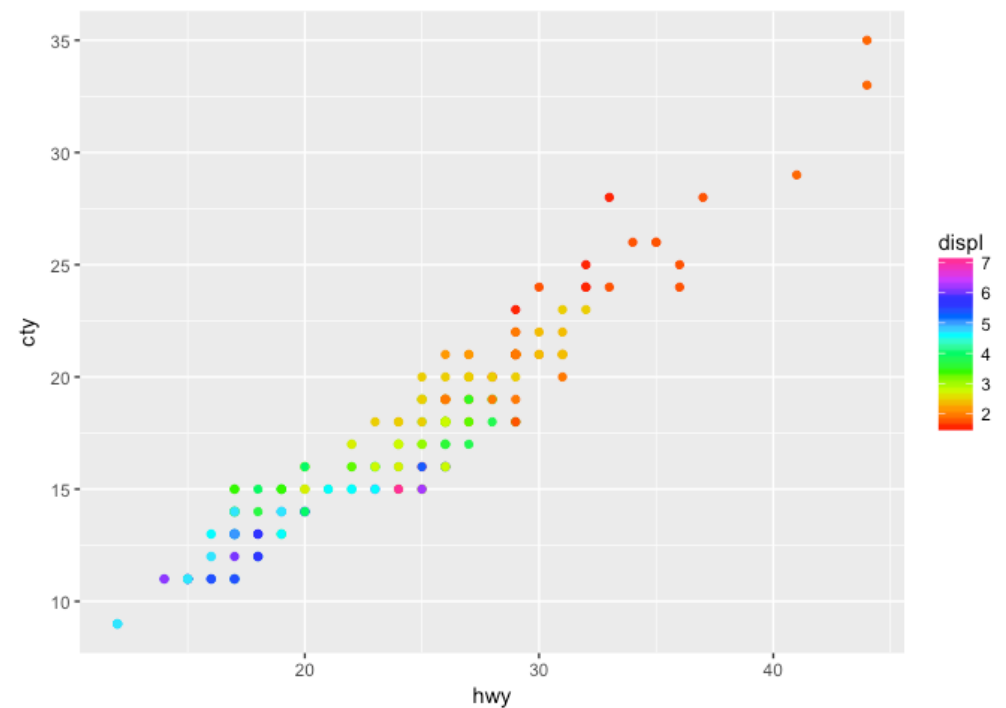


Escalas: Color (Variables continuas)

```
a + scale_color_gradient2(low = "red", high = "blue",  
  mid = "white", midpoint = 4)
```

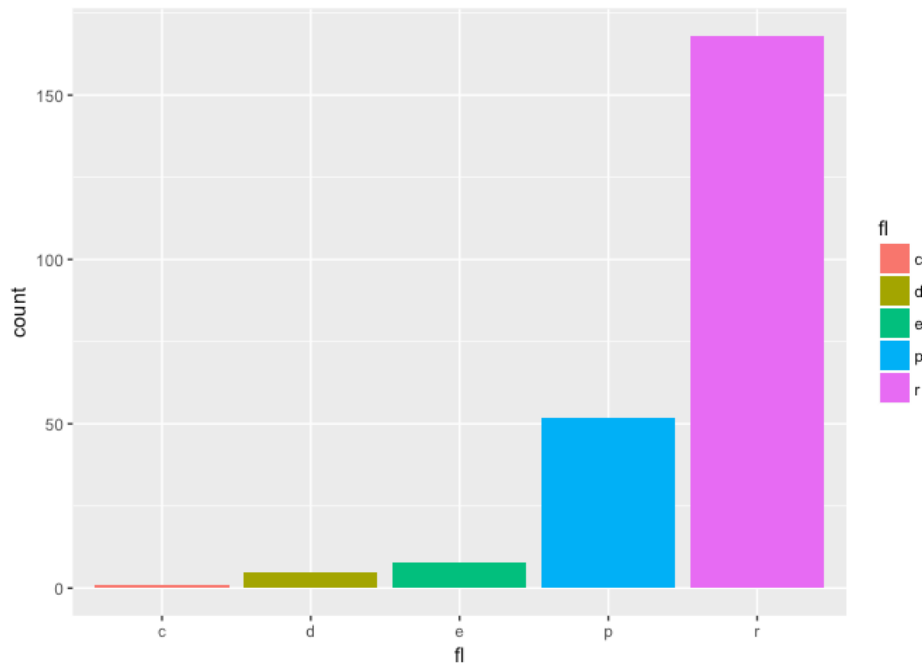


```
a + scale_color_gradientn(  
  colours = rainbow(10))
```

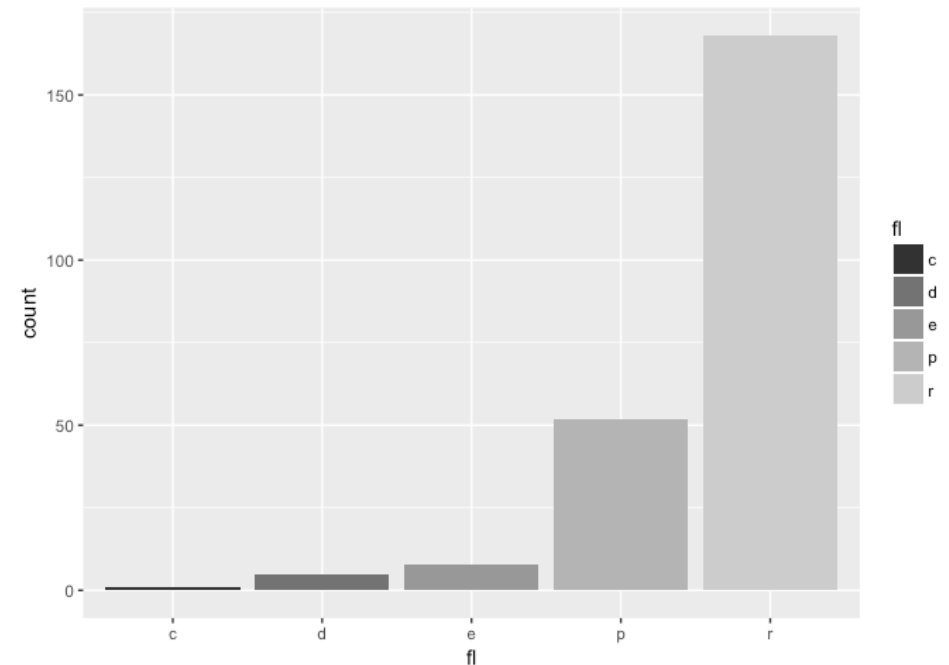


Escalas: Color (Variables discretas)

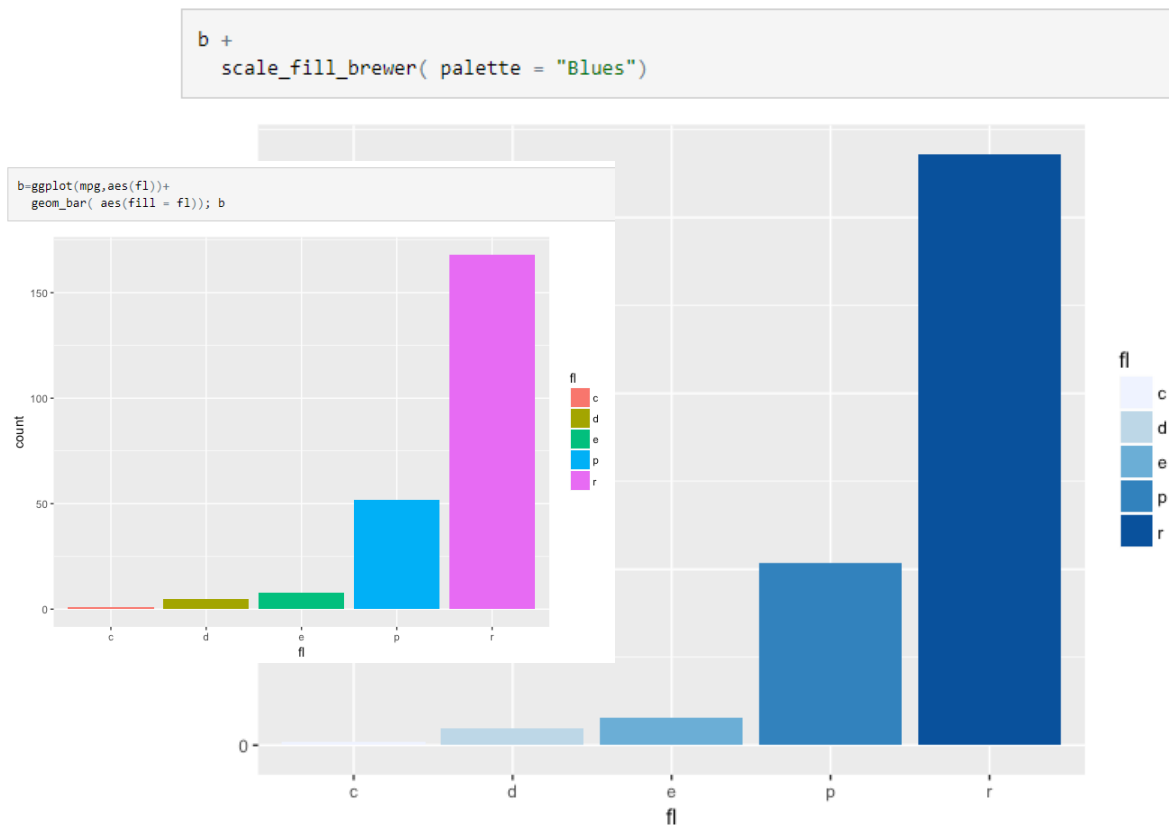
```
b=ggplot(mpg,aes(fl))+  
  geom_bar( aes(fill = fl)); b
```



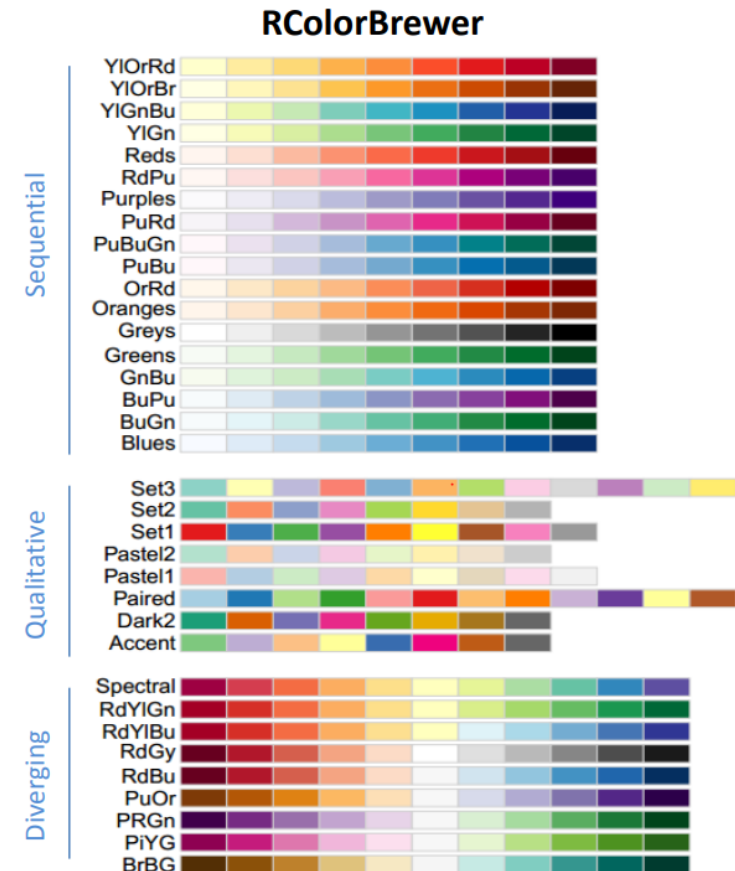
```
b + scale_fill_grey( start = 0.2, end = 0.8,  
  na.value = "red")
```



Escalas: Color (Variables discretas)



<https://www.nceas.ucsb.edu/~frazier/RSpatialGuides/colorPaletteCheatsheet.pdf>



To display RColorBrewer palette: `display.brewer.all()`

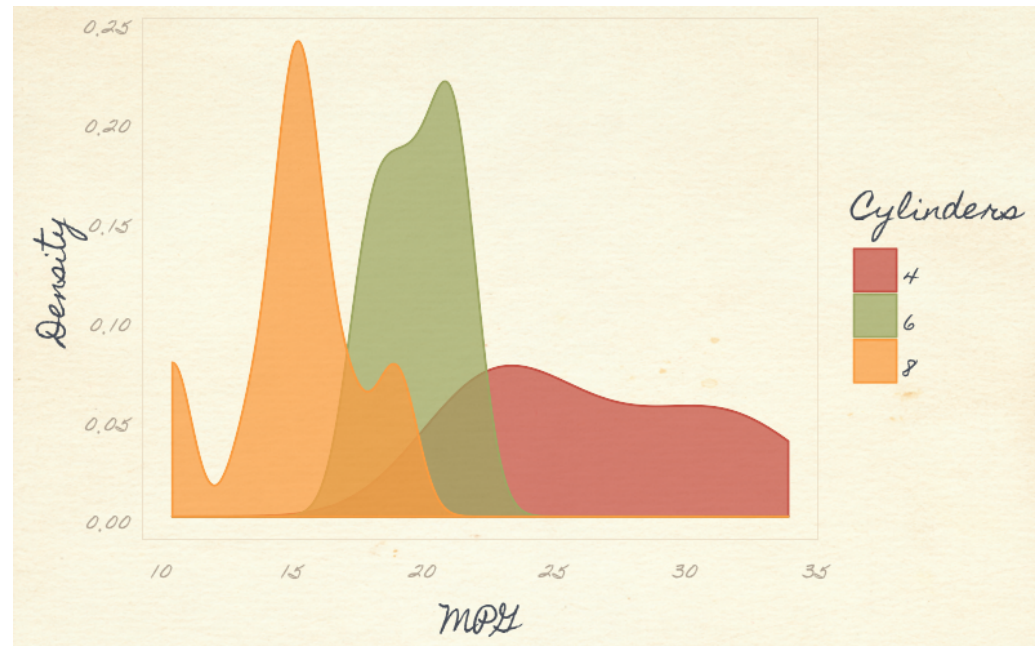
For interactive color selector: <http://colorbrewer2.org/>

6. Puliendo los gráficos.

- Los “temas”
- Guardar los resultados.
- Múltiples gráficos.

Puliendo los gráficos: Los temas.

- Los “temas” permiten controlar los elementos de los gráficos que **no están directamente relacionados con los datos**.
- Los “temas” dan control sobre elementos como:
 - Tipo de letra.
 - Título del gráfico.
 - Título de los ejes.
 - Etiquetas de los ejes.
 - Tipos de líneas de rejilla.
 - Color de líneas de rejilla.
 - Fondo.
 - Posición de la leyenda.
 - ...



Puliendo los gráficos: Los temas.

- La instanciación de los “temas” se realiza mediante la función ***theme()***:
 - Type es el tipo de función `element_xxx` que se precisa para instanciar dicho parámetro:
 - xxx = {line|text|rect|...}

E.g.,

```
...+ theme(plot.title =  
  element_text(size=20))
```

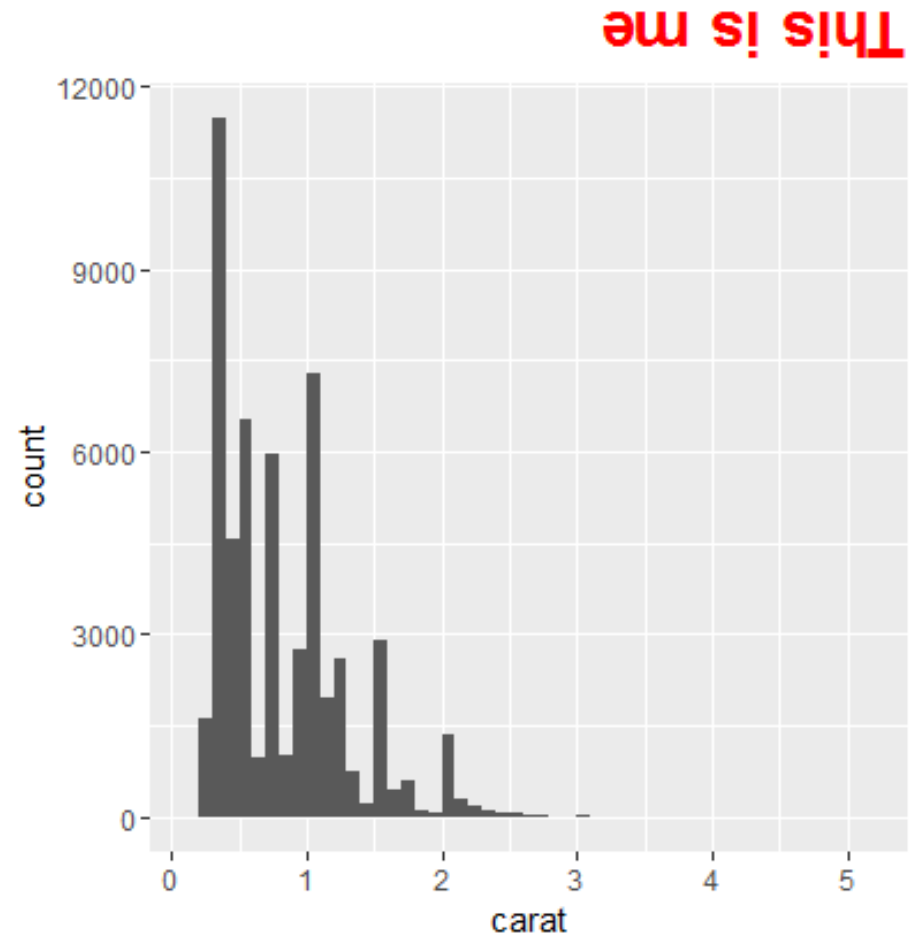
Theme element	Type	Description
axis.line	segment	line along axis
axis.text.x	text	x axis label
axis.text.y	text	y axis label
axis.ticks	segment	axis tick marks
axis.title.x	text	horizontal tick labels
axis.title.y	text	vertical tick labels
legend.background	rect	background of legend
legend.key	rect	background underneath legend keys
legend.text	text	legend labels
legend.title	text	legend name
panel.background	rect	background of panel
panel.border	rect	border around panel
panel.grid.major	line	major grid lines
panel.grid.minor	line	minor grid lines
plot.background	rect	background of the entire plot
plot.title	text	plot title
strip.background	rect	background of facet labels
strip.text.x	text	text for horizontal strips
strip.text.y	text	text for vertical strips

Puliendo los gráficos: Los temas.

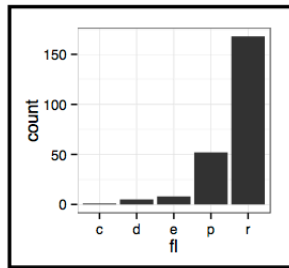
- La instanciación de los “temas” se realiza mediante la función ***theme()***:
 - Type es el tipo de función `element_xxx` que se precisa para instanciar dicho parámetro:
 - `xxx = {line|text|rect|...}`
- Ej:

```
g <- ggplot(movies, aes(rating))
+ geom_histogram(binwidth=1)
g <- g + ggtitle("This is me")

g + theme(plot.title =
  element_text(
    size=20,
    colour="red", hjust=0,
    face="bold", angle=180))
```

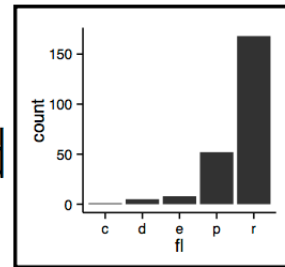


Puliendo los gráficos: Los temas.



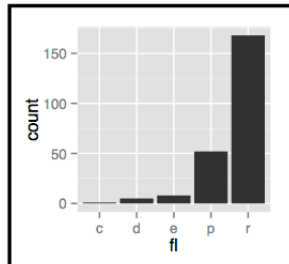
r + theme_bw()

White background
with grid lines



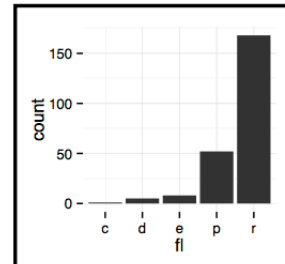
r + theme_classic()

White background
no gridlines



r + theme_grey()

Grey background
(default theme)

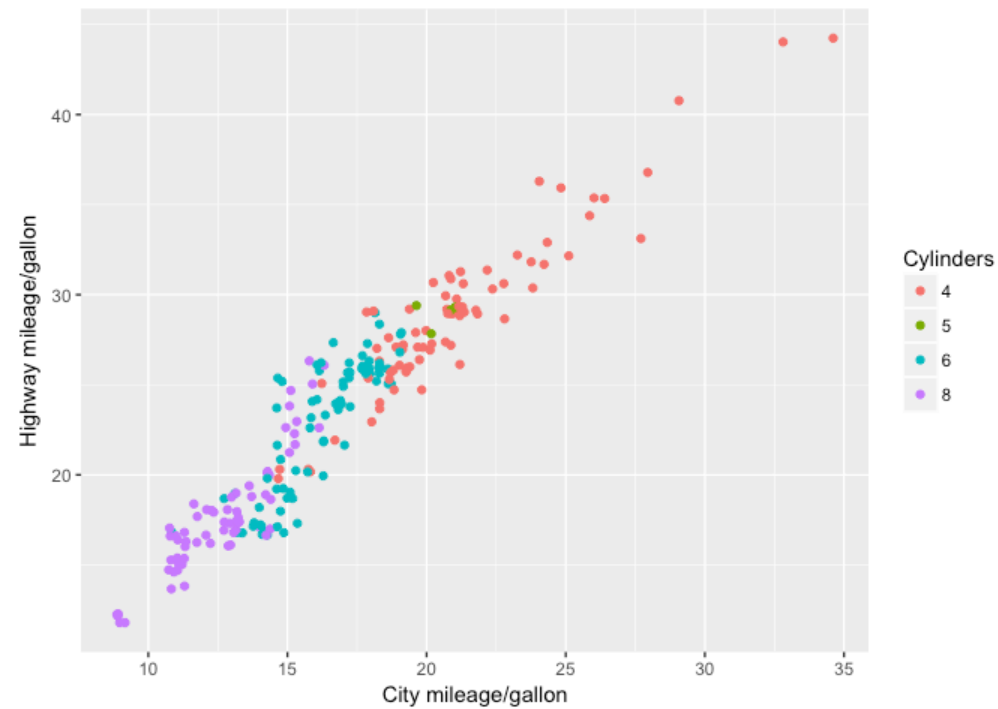


r + theme_minimal()

Minimal theme

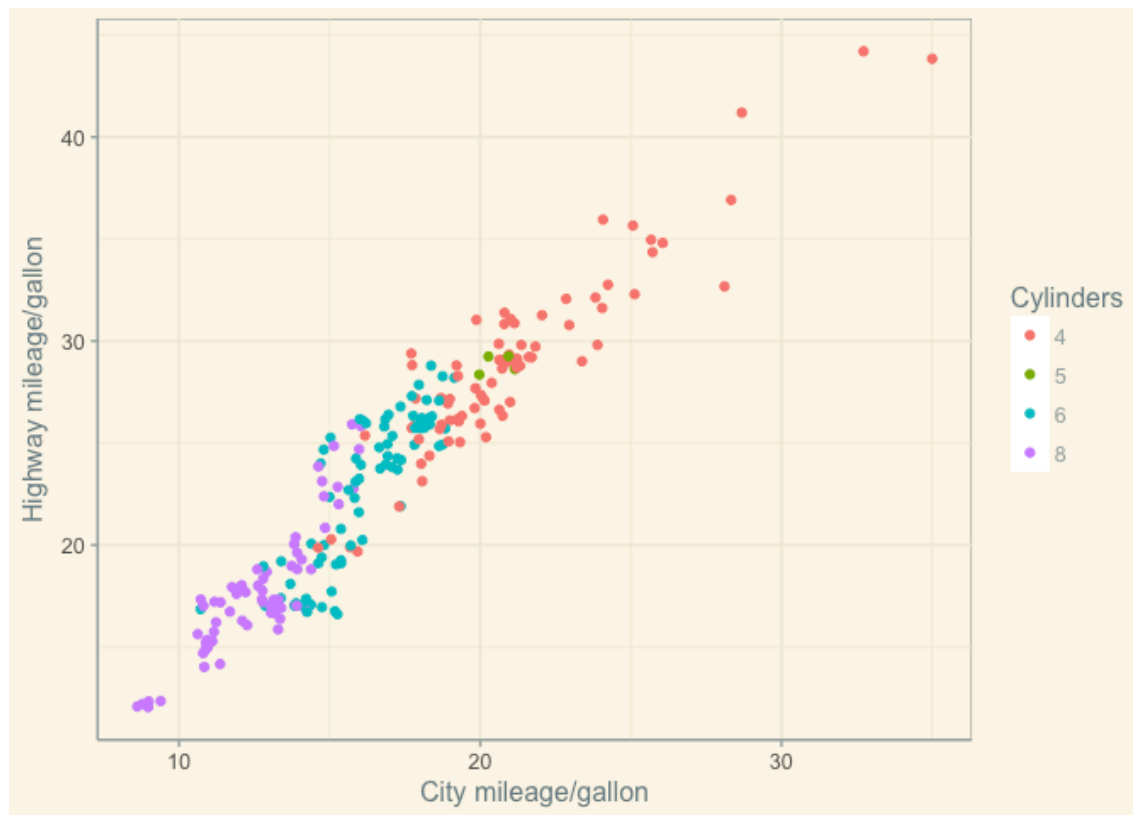
Puliendo los gráficos: Los temas.

```
p=ggplot(mpg, aes(x = cty, y = hwy, color = factor(cyl))) +  
  geom_jitter() +  
  labs(  
    x = "City mileage/gallon",  
    y = "Highway mileage/gallon",  
    color = "Cylinders"  
  )
```



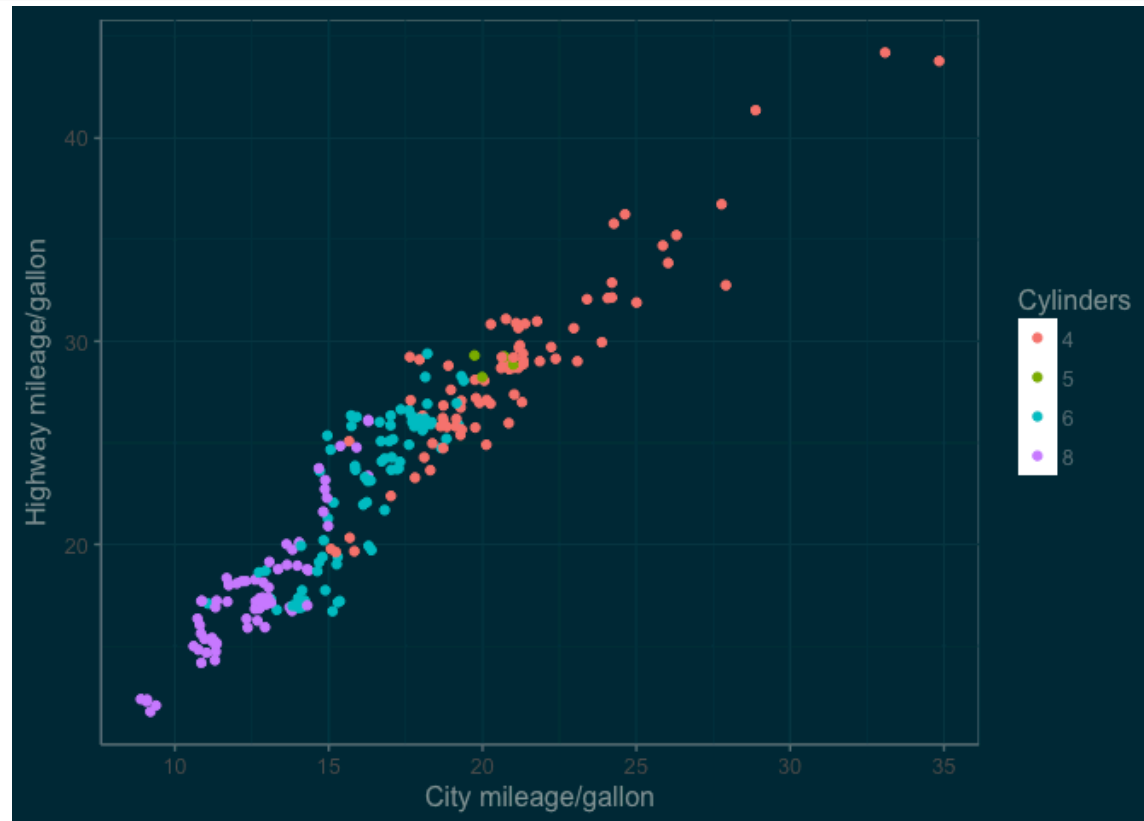
Puliendo los gráficos: Los temas.

```
p + theme_solarized()
```



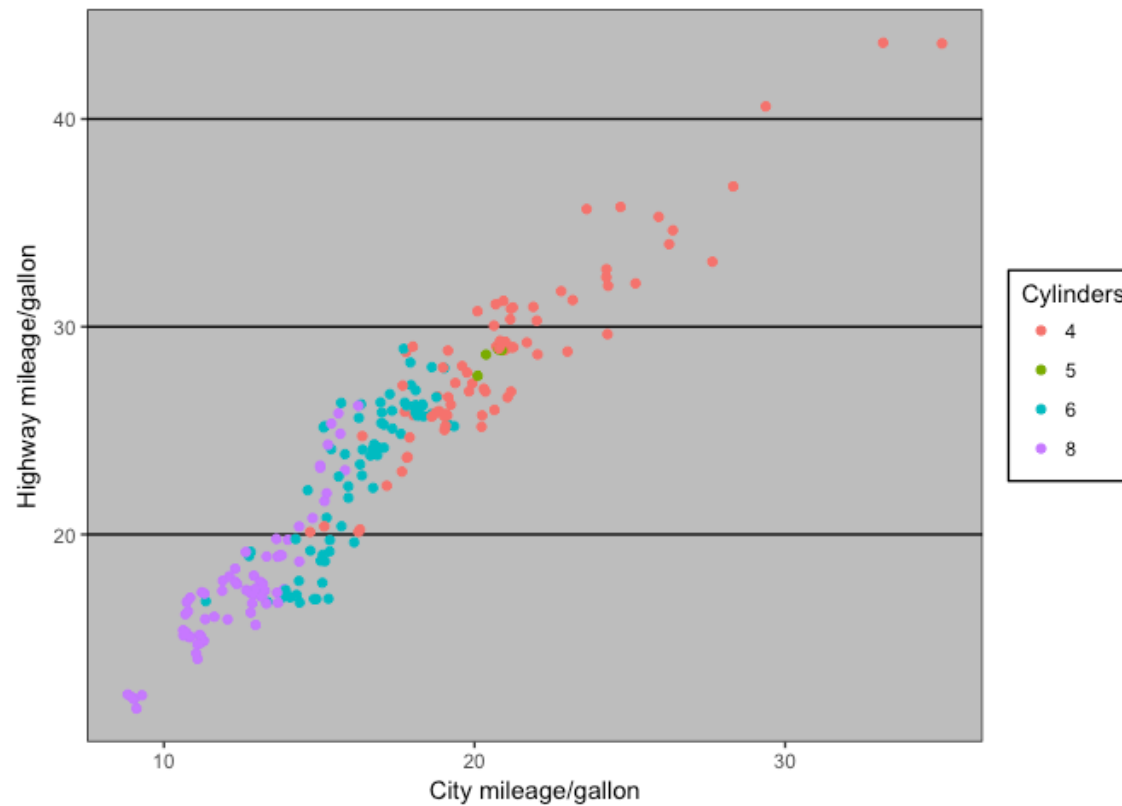
Puliendo los gráficos: Los temas.

```
p + theme_solarized(light=FALSE)
```



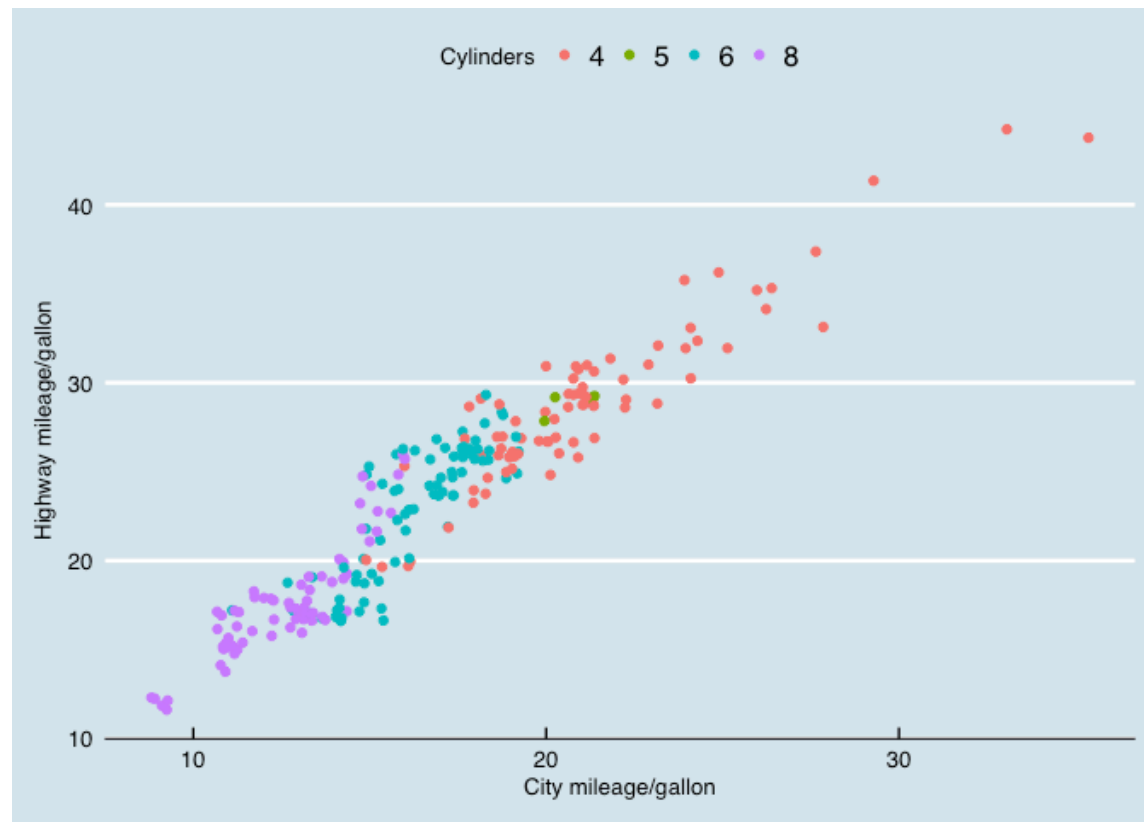
Puliendo los gráficos: Los temas.

```
p + theme_excel()
```



Puliendo los gráficos: Los temas.

```
p + theme_economist()
```



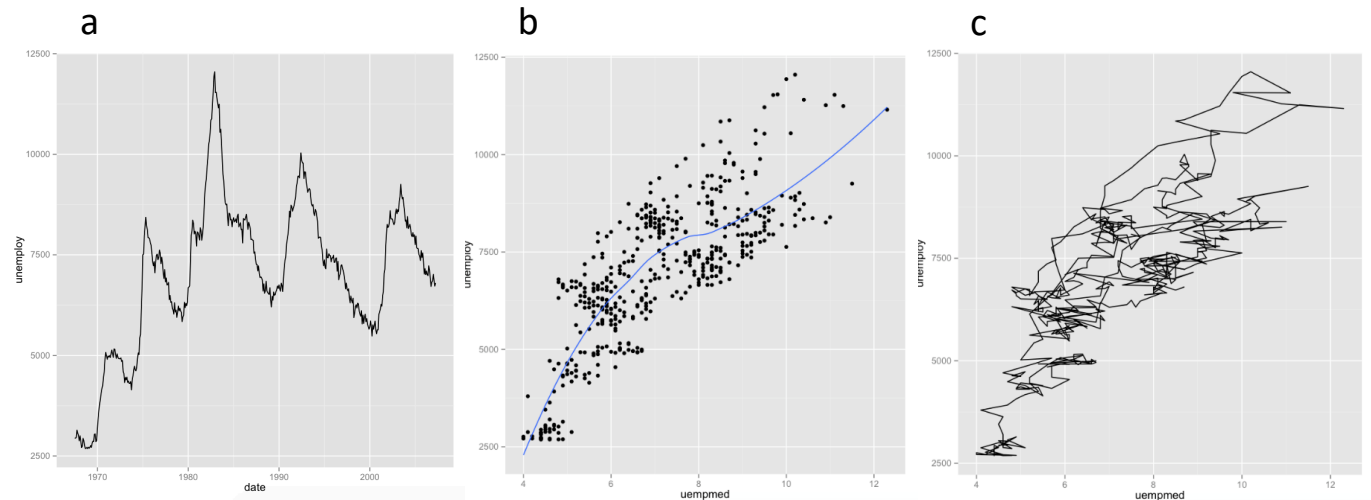
Puliendo gráficos: Múltiples gráficos.

- La colocación de gráficos en la misma página requiere dominar los conceptos de **grid** (rejilla guía de ventana) y **viewport** (subventana).
- El **viewport** por defecto es toda la ventana del gráfico.

```
a <- ggplot(economics, aes(date,
unemploy)) + geom_line()
```

```
b <- ggplot(economics, aes(uempmed,
unemploy)) + geom_point() +
geom_smooth(se = F)
```

```
c <- ggplot(economics, aes(uempmed,
unemploy)) + geom_path()
```

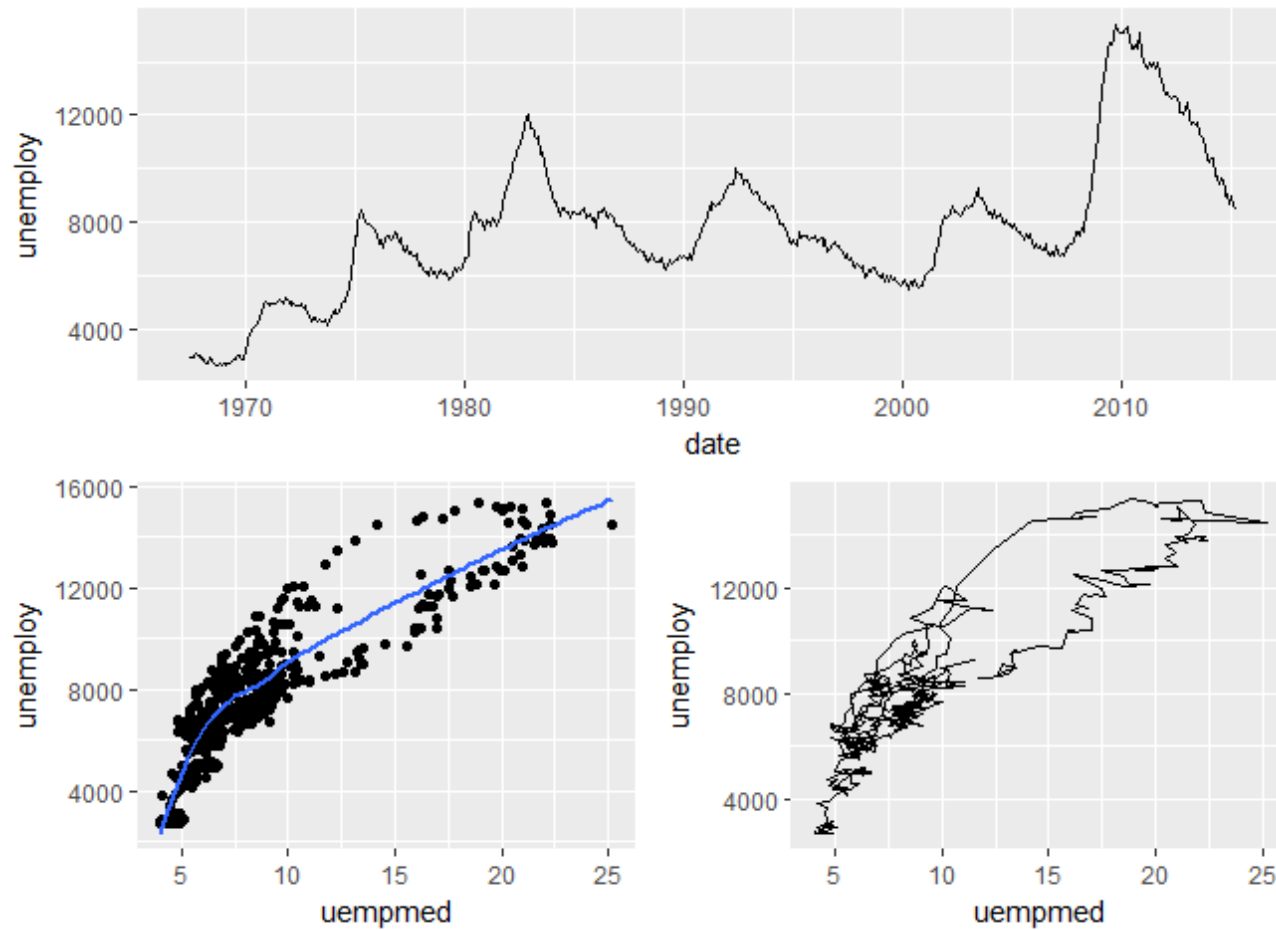


Puliendo gráficos: Múltiples gráficos.

- Para dibujar varios gráficos **sin superposición** se utiliza la función ***grid.Layout()***
- La función ***grid.Layout()*** permite **dividir la ventana en una rejilla regular de subventanas**.
- Para elegir subventana se indica fila y columna:

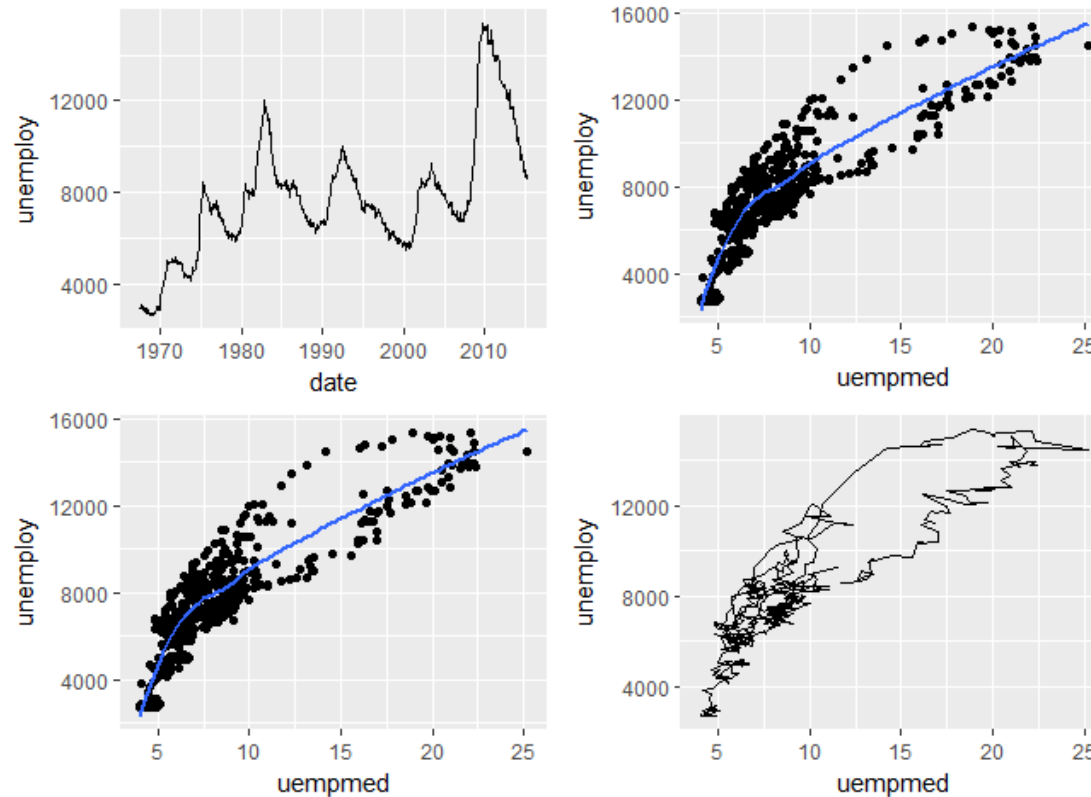
```
Library(grid)
grid.newpage()
pushViewport(viewport(layout=grid.Layout(2,2)))
print(a, vp = viewport(layout.pos.row=1, layout.pos.col=1:2))
print(b, vp = viewport(layout.pos.row=2, layout.pos.col=1))
print(c, vp = viewport(layout.pos.row=2, layout.pos.col=2))
```


Puliendo gráficos: Múltiples gráficos.



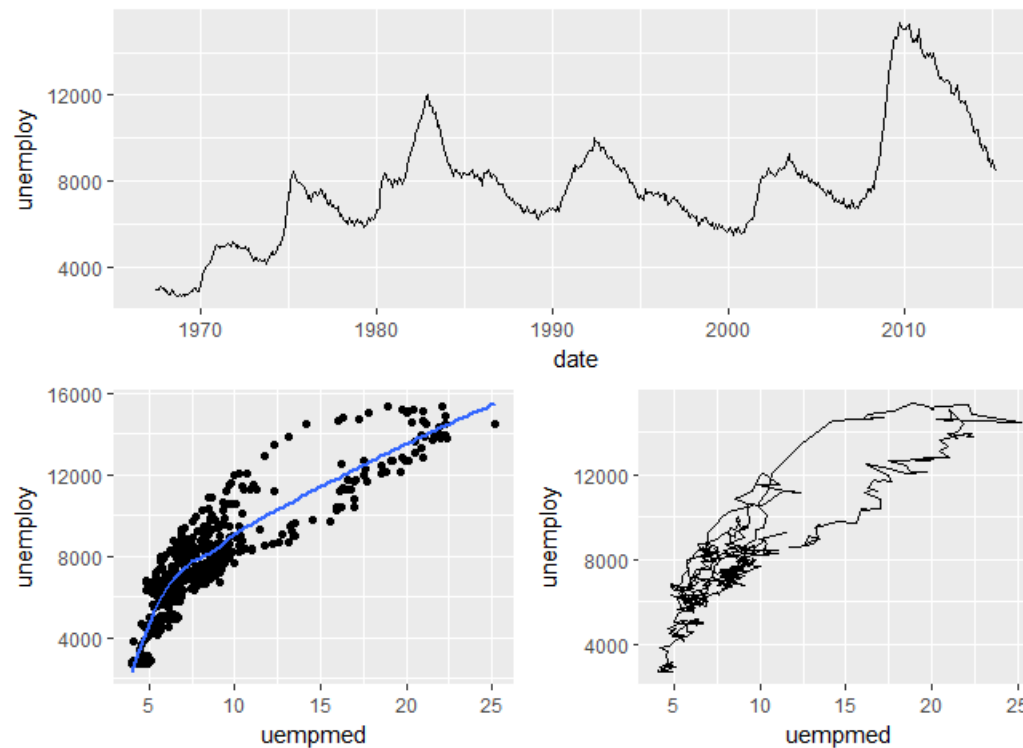
Puliendo gráficos: Múltiples gráficos.

```
library(gridExtra)  
grid.arrange(a, b, b, c, ncol=2)
```

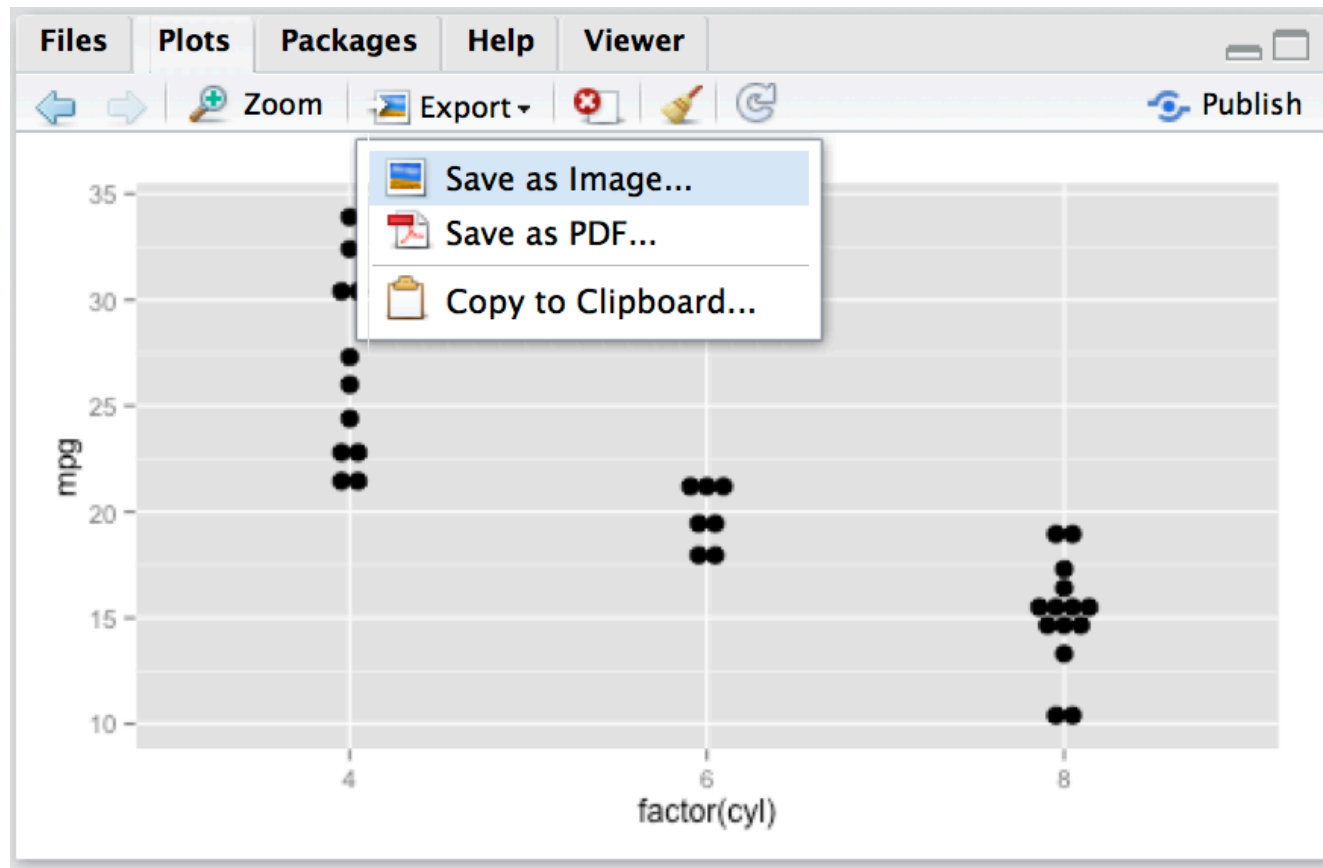


Puliendo gráficos: Múltiples gráficos.

```
library(gridExtra)
lay <- rbind(c(1,1), c(2,3))
grid.arrange(a, b, c, layout_matrix = lay)
```



Puliendo gráficos: Almacenamiento.



Puliendo gráficos: Almacenamiento.

```
ggsave(filename, plot = last_plot(), scale = 1, width, height)
```

- Almacena el último gráfico “renderizado”.
- Parámetros de ***ggsave()***:
 - ***file*** : especifica el nombre del fichero a almacenar. La extensión indica su formato (pdf o png).
 - ***path*** : especifica la carpeta en donde se almacenará el fichero.
 - ***width*** : especifica el ancho absoluto.
 - ***height*** : especifica el alto absoluto.
 - ***scale*** : especifica el tamaño relativo al que tiene en pantalla.
 - ***dpi*** : especifica el número de puntos por pulgada del gráficos (sólo en los de tipo bitmap).

Puliendo gráficos: Almacenamiento.

pdf() y *png()*:

- Permiten almacenar varios gráficos en el mismo fichero.
- Los parámetros son idénticos a los de ***ggsave()***

```
pdf(filename, width, height) # open device  
ggplot()                    # draw the plot(s)  
dev.off()                   # close the device
```

Puliendo gráficos

Ej5. Múltiples Gráficos.

- Visualización de gráficos de dispersión con diagramas de densidad en los márgenes.

